

On-Device Training Library

Complete Beginner's Reference

Every File · Every Line · Every Concept Explained

A comprehensive guide to the ODT C library for on-device training on the STM32 Nucleo-F746ZG microcontroller. Written for beginners — every line of source code is explained in plain language, with definitions for all C syntax and concepts.

Mohamed Atef | Master's Thesis | 2026

Table of Contents

Chapter	Title	Layer
1	Introduction — What is this project and why?	Overview
2	Layer 0 — Common.h: Debug Macros	Foundation
3	Layer 1 — DTypes.h/c: Byte-Level Data Types	Primitives
4	Layer 2 — Quantization.h/c: Quantization System	Data Model
5	Layer 3 — DataStorage.h/c and Tensor.h: The Tensor	Core Data
6	Layer 5 — Arithmetic Engine (all arithmetic files)	Computation
7	Layer 6 — Neural Network Layers (all layer files)	Neural Net
8	Layer 7 — Training Infrastructure (Loss, SGD, Loop)	Training
9	Support Modules (RNG, Serialize, DataLoader, CSV, UserAPI)	Support
10	What is Missing to Implement RigL — and Why	Analysis
11	How Everything Works Together on the STM32 MCU	Integration

Chapter 1

Introduction

What is this project and why does it exist?

1.1 The Problem: Machine Learning on a Tiny Chip

Modern machine learning models are trained on powerful servers with gigabytes of RAM and dedicated GPUs that cost thousands of dollars. But many real-world applications — wearable devices, industrial sensors, wildlife monitors, hearing aids — need to run AI locally, on a tiny chip with no internet connection and less RAM than a floppy disk.

The STM32 Nucleo-F746ZG is one such chip. It has a 32-bit ARM Cortex-M7 processor running at 216 MHz, 320 kilobytes of RAM, and 1 megabyte of flash storage. For comparison, a single image from your phone camera takes about 10 megabytes. Running even a tiny neural network on this chip is a serious engineering challenge.

This project — the On-Device Training (ODT) library — solves that problem. It is a C library (not Python, not TensorFlow) that can train and run neural networks entirely on the STM32, without any connection to a PC. The application is speaker identification from audio, as part of a Master's thesis combining two research ideas: FQT (Fixed-point Quantization Training) and RigL (sparse training).

1.2 What is FQT?

Floating-point numbers (like 0.12345678) need 32 bits each. That is too expensive for a chip with 320 KB of RAM. Fixed-point quantization replaces each 32-bit float with a smaller integer (say, an 8-bit or 16-bit number) plus a scale factor. For example, the float 0.5 might be stored as the integer 64 with a scale of 1/128. This is called SYM_INT32 quantization in the ODT library.

— ■ WORKED EXAMPLE

Example: storing the value 0.75 as SYM_INT32 with scale=0.01: mantissa = $\text{round}(0.75 / 0.01) = \text{round}(75.0) = 75$
Stored as: integer 75 (4 bytes) To recover: $75 \times 0.01 = 0.75$ With 8-bit storage (qMaxBits=8), values -127 to +127 are representable. With 16-bit storage (qMaxBits=16), values -32767 to +32767 are representable. FQT trains the network to be accurate even with this limited precision.

1.3 What is RigL?

Most neural networks are 'dense': every neuron is connected to every other neuron. But research (Evci et al., 2020 — RigL paper) shows that you can remove most connections (make the network 'sparse') and still get good accuracy. This is important on the STM32 because fewer connections means fewer multiplications, which means faster inference and less memory.

RigL works by periodically removing the smallest-magnitude weights and adding new connections where the gradients are largest. The ODT library has partial support for RigL — Chapter 10 explains exactly what is implemented and what is still missing.

1.4 The 8-Layer Architecture

The ODT library is organised in 8 layers, where each layer builds on the ones below. This document has one chapter per layer. Understanding the layers in order is the key to understanding the whole system.

Layer	Files	What it provides
0 — Foundation	Common.h	Debug macros — printing error/warning/debug messages
1 — Primitives	DTypes.h/.c	Read/write individual numbers as raw bytes
2 — Data Model	Quantization.h/.c	How numbers are stored: float, int, quantized
3 — Core Data	DataStorage.h, Tensor.h, TensorConversion.h	The tensor — the central data container
5 — Arithmetic	Arithmetic.h/.c + Add/Sub/Mul/Div/Matmul/etc.	All math operations on tensors
6 — Layers	Layer.h + Linear/Conv1d/Relu/Softmax/etc.	Neural network layers (forward + backward)
7 — Training	LossFunction.h + Optimizer.h + TrainingLoop	Loss functions, SGD optimizer, training loop
Support	RNG, Serialize, DataLoader, UserAPI	Random numbers, saving/loading, data pipeline

1.5 How to Read This Document

Each chapter follows the same structure: (1) what the file does and why it exists; (2) the complete source code with every line explained; (3) relations — which files depend on this file and which files this file uses; (4) what is missing for RIGL. C syntax concepts are explained in blue boxes whenever they first appear. Read the chapters in order — later chapters assume knowledge from earlier ones.

■ BEGINNER TIP

If you are new to C: start by reading Chapter 2 very carefully. It introduces the most important C concepts (preprocessor, macros, `#ifndef`). Later chapters build on these. Every time you see a 'C CONCEPT' box, stop and read it before continuing.

Chapter 2

Layer 0 — Common.h

The Debug Macro System

Section 2.1 — What This File Does

■ WHAT THIS FILE DOES

Common.h provides a debug printing system for the entire ODT library. It defines four macros: `PRINT_ERROR`, `PRINT_WARNING`, `PRINT_INFO`, and `PRINT_DEBUG`. Each macro calls `printf()` to output a message, but `PRINT_WARNING/INFO/DEBUG` are silenced in production builds (zero code size, zero runtime cost). `PRINT_ERROR` is always active because errors are fatal and must always be reported. Every .c file in the library includes Common.h as its first include.

Think of Common.h as the 'alarm system' of the library. During development, you can turn on verbose output to see exactly what is happening. When you compile for the final product (flashing to the STM32), all the verbose output is automatically removed by the compiler, so the firmware is as small and fast as possible.

Section 2.2 — Complete Source Code

```

// Common.h — src/common/include/Common.h
#ifndef COMMON_H // line 1
#define COMMON_H // line 2
#include <stdio.h> // line 4
#include <stdlib.h> // line 5
#include <stdbool.h> // line 6
#ifndef SOURCE_FILE // line 8
#define SOURCE_FILE "UNKNOWN" // line 9
#endif // line 10
#if defined DEBUG_MODE_DEBUG // line 12
#define DLEVEL 3 // line 13
#elif defined DEBUG_MODE_INFO // line 14
#define DLEVEL 2 // line 15
#elif defined DEBUG_MODE_WARNING // line 16
#define DLEVEL 1 // line 17
#else // line 18
#define DLEVEL 0 // line 19
#endif // line 20
#define PRINT_ERROR(str, ...) do { // line 22
printf("[%s] ERROR: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false) // line 24
#if DLEVEL >= 1 // line 26
#define PRINT_WARNING(str, ...) do { \
printf("[%s] WARNING: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false) // line 29
#else
#define PRINT_WARNING(str, ...) do { } while(false)
#endif
#if DLEVEL >= 2
#define PRINT_INFO(str, ...) do { \
printf("[%s] INFO: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_INFO(str, ...) do { } while(false)
#endif
#if DLEVEL >= 3
#define PRINT_DEBUG(str, ...) do { \
printf("[%s] DEBUG: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_DEBUG(str, ...) do { } while(false)
#endif
#endif // COMMON_H

```

Section 2.3 — Line-by-Line Explanation

■ C CONCEPT: #ifndef / #define / #endif — Header Guard

A header guard stops a .h file from being included more than once in the same compilation. Without it, if two .c files both include Common.h, the compiler would see all the definitions twice and give 'redefinition' errors. #ifndef COMMON_H → 'if the name COMMON_H is NOT yet defined...' #define COMMON_H → '...then define it now (as an empty symbol)' #endif → '...end of the conditional block' The first time Common.h is included, COMMON_H is not defined, so the block executes. The second time it is included, COMMON_H IS already defined, so the entire block is skipped. The name COMMON_H is arbitrary — it is the convention to use the filename in uppercase with dots replaced by underscores.

1	#ifndef COMMON_H	Start of the header guard. 'ifndef' = 'if not defined'. If the preprocessor symbol COMMON_H has not been defined yet, process everything until the matching #endif. This prevents the file from being processed twice.
---	------------------	--

2	<code>#define COMMON_H</code>	Define the guard symbol. After this line, <code>COMMON_H</code> is defined (its value is empty). If this file is included again later, line 1 will be false, and the whole file is skipped.
4	<code>#include <stdio.h></code>	Include the standard I/O header. This gives us access to the <code>printf()</code> function. Angle brackets <code>< ></code> mean this is a system header (part of the C standard library). Double quotes would mean a project-local file.
5	<code>#include <stdlib.h></code>	Include the standard library header. This gives us <code>exit()</code> , <code>malloc()</code> , <code>free()</code> , and more. We need <code>exit()</code> so that <code>PRINT_ERROR</code> can halt the program after a fatal error.
6	<code>#include <stdbool.h></code>	Include the boolean header. This defines the <code>bool</code> type (<code>true/false</code>). Without this, C has no boolean — it uses 0/1 integers instead. <code>stdbool.h</code> is a C99 addition; before C99, programmers had to define <code>bool</code> themselves.
8	<code>#ifndef SOURCE_FILE</code>	Each <code>.c</code> file defines <code>SOURCE_FILE</code> as its filename before including <code>Common.h</code> : <code>#define SOURCE_FILE "RELU"</code> . If a file forgets to do this, this guard provides the fallback <code>'UNKNOWN'</code> .
9	<code>#define SOURCE_FILE "UNKNOWN"</code>	Fallback value for <code>SOURCE_FILE</code> . The debug messages will show <code>[UNKNOWN]</code> as the source, which is a signal to the developer to add a proper definition.
12	<code>#if defined DEBUG_MODE_DEBUG</code>	Compile-time conditional: check if the compiler was given <code>-DDEBUG_MODE_DEBUG</code> . <code>'#if defined X'</code> is equivalent to <code>'#ifdef X'</code> . This starts a chain of three levels of debug verbosity.
13	<code>#define DLEVEL 3</code>	If <code>DEBUG_MODE_DEBUG</code> is defined, set <code>DLEVEL</code> to 3 (most verbose). <code>DLEVEL</code> controls which print macros are active: level 3 enables ALL of them.
14	<code>#elif defined DEBUG_MODE_INFO</code>	<code>'elif' = 'else if'</code> . Only checked if line 12 was false. If <code>DEBUG_MODE_INFO</code> is defined (but not <code>DEBUG_MODE_DEBUG</code>), set <code>DLEVEL=2</code> .
16	<code>#elif defined DEBUG_MODE_WARNING</code>	If only <code>DEBUG_MODE_WARNING</code> is defined, set <code>DLEVEL=1</code> (warnings only).
18	<code>#else</code>	No debug flag was defined — this is the default production build.
19	<code>#define DLEVEL 0</code>	<code>DLEVEL=0</code> means: only <code>PRINT_ERROR</code> is active. All other macros expand to empty code. The compiler sees <code>'do { } while(false)'</code> and removes it entirely. This is how the library achieves zero overhead in production.
22	<code>#define PRINT_ERROR(str, ...) do { \</code>	Define the <code>PRINT_ERROR</code> macro. It takes a format string <code>'str'</code> plus variadic arguments <code>'...'</code> (zero or more extra parameters, like <code>printf</code>). The <code>\</code> at the end continues the macro definition to the next line. <code>PRINT_ERROR</code> has NO <code>DLEVEL</code> guard — it is ALWAYS active.
23	<code>printf("[%s] ERROR: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \</code>	<code>'[%s] ERROR: '</code> and <code>'str'</code> are two string literals next to each other. The C preprocessor concatenates adjacent string literals automatically. So <code>PRINT_ERROR("value is %d", x)</code> becomes <code>printf("[RELU] ERROR: value is %d\n", x)</code> . <code>##__VA_ARGS__</code> expands to the variadic arguments; <code>##</code> removes the leading comma if there are no extra arguments (prevents a syntax error for <code>PRINT_ERROR("msg")</code>).

24	<code>} while(false)</code>	The <code>do { } while(false)</code> wrapper is a safe macro pattern. It makes the macro behave like a single statement in all contexts: <code>if (err) PRINT_ERROR("bad!");</code> ← works correctly with the wrapper Without the wrapper, the macro expands to two statements, and only the first is guarded by the <code>if</code> . The <code>do-while</code> forces both into one compound statement.
26	<code>#if DLEVEL >= 1</code>	<code>PRINT_WARNING</code> is only compiled into the binary if <code>DLEVEL</code> is 1, 2, or 3. The preprocessor evaluates <code>DLEVEL >= 1</code> at compile time — this is not a runtime check.

■ C CONCEPT: The Preprocessor — What It Does

The C preprocessor runs BEFORE the compiler. It handles all lines starting with `#`. It performs three main jobs: 1. File inclusion (`#include`) — copies the entire contents of a header file in place 2. Macro expansion (`#define`) — replaces macro names with their defined text 3. Conditional compilation (`#if`, `#ifdef`, `#ifndef`) — includes or excludes blocks of code The preprocessor sees the source code as raw text. It does not understand C syntax — it only does text substitution. The result of preprocessing is given to the C compiler, which then does the actual translation to machine code.

■ C CONCEPT: Variadic Macros — ... and `__VA_ARGS__`

A variadic macro accepts a variable number of arguments, like `printf()`. `#define MY_PRINT(fmt, ...) printf(fmt, __VA_ARGS__)` The `...` in the parameter list means 'zero or more extra arguments'. Inside the macro body, `__VA_ARGS__` expands to those extra arguments. The `##` before `__VA_ARGS__` is a special GNU extension: it removes the comma before `__VA_ARGS__` when there are no extra arguments. Without it: `MY_PRINT("hello")` → `printf("hello", )` ← syntax error! With `##`: `MY_PRINT("hello")` → `printf("hello")` ← correct

Section 2.4 — Relations With Other Files

■ RELATIONS WITH OTHER FILES

`Common.h` is the only file with NO dependencies — it is Layer 0. Every other `.c` file in the library includes `Common.h` as its first include. This makes `PRINT_ERROR`, `PRINT_WARNING` etc. available everywhere.

File	Direction	Why
<code>Common.h</code>	→ (no dependencies)	Layer 0: depends on nothing
Every <code>.c</code> file in the library	← includes <code>Common.h</code>	All <code>.c</code> files need <code>PRINT_ERROR</code> for error reporting
<code>stdio.h</code> (C standard library)	← includes (system)	Provides <code>printf()</code> which <code>PRINT_ERROR</code> uses
<code>stdlib.h</code> (C standard library)	← includes (system)	Provides <code>exit()</code> for fatal error termination
<code>stdbool.h</code> (C standard library)	← includes (system)	Provides <code>bool</code> type for <code>do-while(false)</code> safety pattern

2.5 The SOURCE_FILE Convention

Every `.c` file in the library starts with the line:

```
#define SOURCE_FILE "FILENAME"
```

This line must appear BEFORE the `#include "Common.h"` line, because `Common.h` uses `SOURCE_FILE` in its macro definitions. When `PRINT_ERROR` is called from `Relu.c`, the output shows `[RELU]` so the developer immediately knows which file triggered the error. On a UART terminal connected to the STM32, this is often the

only debugging information available.

— ■ WORKED EXAMPLE

How a .c file starts (example from Relu.c): `#define SOURCE_FILE "RELU"` ← must come FIRST `#include <stdlib.h>`
`#include <string.h>` `#include "Common.h"` ← `SOURCE_FILE` must be defined before this `#include "Relu.h"` When
you see: `[RELU] ERROR: Unknown QType!` You immediately know: the error came from Relu.c, the switch
statement fell through to default because an unexpected quantization type was used.

Section 2.5 — What Is Missing for RigL in This File

■ RigL GAP IN THIS FILE

Common.h has no RigL gap. It is a utility file that provides debug printing. RigL does not require any changes here.
However, when implementing RigL operations (pruning, regrowing), the developer will use `PRINT_DEBUG` to log
sparsity changes during training — so Common.h will naturally be used by the new RigL code.

Chapter 3

Layer 1 — DTypes.h / DTypes.c

Byte-Level Data Types and Raw Memory Access

Section 3.1 — What This File Does

■ WHAT THIS FILE DOES

DTypes.h and DTypes.c provide the most fundamental operations in the library: reading and writing individual numbers (int32_t and float) as raw bytes. All tensor data is stored as a raw array of uint8_t (unsigned bytes). To read or write a value, you need to assemble/disassemble its bytes. DTypes.c provides safe functions to do this using memcpy. Every arithmetic operation in the library eventually calls these functions.

Why store data as bytes? Because uint8_t is the universal currency of memory in C. A tensor might hold float32 values today and int32 values tomorrow. By storing everything as raw bytes, the tensor_t struct does not need to know what type of data it holds — only the functions that read/write it need to know. This is the foundation of the library's type flexibility.

■ C CONCEPT: What is uint8_t?

uint8_t is an 8-bit unsigned integer — a single byte, holding values 0 to 255. The 'u' stands for unsigned (no negative values). '8' is the bit width. 't' is a C convention for type aliases (typedef). Data type widths: uint8_t = 1 byte = 8 bits (values 0 to 255) int32_t = 4 bytes = 32 bits (values -2,147,483,648 to +2,147,483,647) float = 4 bytes = 32 bits (IEEE 754 floating point) size_t = 4 or 8 bytes (platform-dependent, always unsigned) These types are defined in <stdint.h> (for int32_t, uint8_t) and <stddef.h> (for size_t). Using them instead of 'int' or 'long' guarantees exact sizes on all platforms.

Section 3.2 — DTypes.h: The Header File

```
// DTypes.h — src/tensor/include/DTypes.h
#ifndef ENV5_RUNTIME_DTYPES_H
#define ENV5_RUNTIME_DTYPES_H
#include <stdint.h>
#include <stddef.h>
// Read functions
int32_t readBytesAsInt32(const uint8_t *bytes);
float readBytesAsFloat(const uint8_t *bytes);
// Write functions
void writeInt32ToByteArray(int32_t value, uint8_t *bytes);
void writeFloatToByteArray(float value, uint8_t *bytes);
// Bit-level operations (for BOOL/sparse types)
void writeBitsToByteArray(uint8_t *byteArray, size_t startBit,
size_t endBit, uint8_t data);
uint8_t readBitsFromByteArray(const uint8_t *byteArray, size_t startBit,
size_t endBit);
#endif // ENV5_RUNTIME_DTYPES_H
```

The header declares 6 functions. The first four are the workhorses used everywhere. The last two (writeBitsToByteArray, readBitsFromByteArray) handle sub-byte operations for BOOL tensors (where each element takes only 1 bit) and sparse tensors. Every file that needs to read or write tensor values includes DTypes.h.

Section 3.3 — DTypes.c: The Implementation

```
// DTypes.c – src/tensor/DTypes.c
#define SOURCE_FILE "DTYPES"
#include <string.h> // for memcpy
#include <stdint.h>
#include "Common.h"
#include "DTypes.h"
int32_t readBytesAsInt32(const uint8_t *bytes) {
    int32_t value;
    memcpy(&value, bytes, sizeof(int32_t));
    return value;
}
float readBytesAsFloat(const uint8_t *bytes) {
    float value;
    memcpy(&value, bytes, sizeof(float));
    return value;
}
void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {
    memcpy(bytes, &value, sizeof(int32_t));
}
void writeFloatToByteArray(float value, uint8_t *bytes) {
    memcpy(bytes, &value, sizeof(float));
}
void writeBitsToByteArray(uint8_t *byteArray, size_t startBit,
    size_t endBit, uint8_t data) {
    size_t byteIndex = startBit / 8;
    size_t bitStart = startBit % 8;
    size_t bitEnd = endBit % 8;
    uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;
    byteArray[byteIndex] = (byteArray[byteIndex] & ~mask) | ((data << bitStart) & mask);
}
uint8_t readBitsFromByteArray(const uint8_t *byteArray, size_t startBit,
    size_t endBit) {
    size_t byteIndex = startBit / 8;
    size_t bitStart = startBit % 8;
    size_t bitEnd = endBit % 8;
    uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;
    return (byteArray[byteIndex] & mask) >> bitStart;
}
```

Section 3.4 — Line-by-Line Explanation

3.4.1 readBytesAsInt32

■ C CONCEPT: memcpy — The Safe Way to Reinterpret Memory

`memcpy(destination, source, size)` copies 'size' bytes from source to destination. Why not just cast? You might think: why not write `*(int32_t*)bytes`? This would tell the CPU: 'treat the memory at address bytes as an `int32_t`'. But this is UNDEFINED BEHAVIOUR in C when the pointer is misaligned or the pointer types violate strict aliasing rules (the compiler can assume pointers to different types never point to the same memory). `memcpy` is the CORRECT way: it copies the raw bytes without any type assumptions. The compiler knows `memcpy` semantics and can optimize it to a single 32-bit load instruction on ARM Cortex-M7 anyway. Example: `uint8_t bytes[4] = {0x00, 0x00, 0x80, 0x3F};` // IEEE 754 for 1.0f float value; `memcpy(&value, bytes, sizeof(float));` // value is now 1.0f

1-3 `int32_t readBytesAsInt32(const uint8_t *bytes) {`

Function signature. '`const uint8_t *bytes`' means: a pointer to bytes that we promise not to modify (`const`). The '`const`' is important for correctness and lets the compiler pass read-only memory safely.

2	<code>int32_t value;</code>	Declare a local <code>int32_t</code> variable. It is uninitialized — its value is garbage until <code>memcpy</code> fills it on the next line. We do NOT initialize it to 0 because <code>memcpy</code> immediately overwrites all 4 bytes anyway.
3	<code>memcpy(&value, bytes, sizeof(int32_t));</code>	Copy <code>sizeof(int32_t) = 4</code> bytes from 'bytes' into 'value'. '&value' takes the address of the local variable — gives <code>memcpy</code> a pointer to write to. <code>sizeof(int32_t)</code> ensures we copy exactly 4 bytes regardless of the platform. After this line, 'value' holds the integer stored in <code>bytes[0..3]</code> .
4	<code>return value;</code>	Return the assembled integer. The caller receives it as a regular <code>int32_t</code> .

— ■ WORKED EXAMPLE

Example: reading the integer 305 from a byte array. 305 in binary = `0x00000131` = bytes `[0x31, 0x01, 0x00, 0x00]` (little-endian ARM) `uint8_t data[4] = {0x31, 0x01, 0x00, 0x00}; int32_t result = readBytesAsInt32(data); // result = 305`
 Little-endian means: the LEAST significant byte comes first in memory. ARM Cortex-M7 (and most modern CPUs) are little-endian. `0x31` is the least significant byte of 305; `0x00` is the most significant byte.

3.4.2 readBytesAsFloat

■ C CONCEPT: IEEE 754 — How Floats are Stored

A 32-bit float (float in C) is stored in IEEE 754 format: Bit 31: sign (0 = positive, 1 = negative) Bits 30-23: exponent (8 bits, biased by 127) Bits 22-0: mantissa (23 bits, fraction part) The value = $(-1)^{\text{sign}} \times 2^{(\text{exponent}-127)} \times 1.\text{mantissa}$
 Example: 1.0 in IEEE 754: Sign=0, Exponent=127 (stored as `01111111`), Mantissa=0 Bytes: `0x00, 0x00, 0x80, 0x3F` (little-endian) This is why `readBytesAsFloat` uses `memcpy` — we need to take the 4 raw bytes and have the CPU interpret them as a float WITHOUT changing their bit pattern.

1-3	<code>float readBytesAsFloat(const uint8_t *bytes) {</code>	Identical pattern to <code>readBytesAsInt32</code> , but returns float instead of <code>int32_t</code> .
2	<code>float value;</code>	Local float variable — 4 bytes on the stack, same as <code>int32_t</code> .
3	<code>memcpy(&value, bytes, sizeof(float));</code>	Copy 4 bytes from the array into the float variable. The CPU does NOT convert any values — it copies the exact bit pattern. If bytes holds the IEEE 754 representation of 0.5, value will be 0.5f.

3.4.3 writeInt32ToByteArray and writeFloatToByteArray

1	<code>void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {</code>	Reverse operation: write an <code>int32_t</code> into a byte array. 'void' means no return value — the result is written through the pointer 'bytes'. The caller is responsible for ensuring 'bytes' points to at least 4 valid bytes.
2	<code>memcpy(bytes, &value, sizeof(int32_t));</code>	Copy 4 bytes FROM 'value' (taking its address with &) INTO 'bytes'. After this, <code>bytes[0..3]</code> hold the little-endian representation of 'value'. For <code>value=305</code> : bytes becomes <code>[0x31, 0x01, 0x00, 0x00]</code> .

3.4.4 writeBitsToByteArray — Bit-Level Operations

■ C CONCEPT: Bit Manipulation — Masks, Shifts, OR, AND

A bitmask is an integer with specific bits set to 1. Combining a mask with bitwise operations lets you set, clear, or read individual bits. Operators: & (AND) — `1 & 1 = 1`; anything & 0 = 0 → clears bits | (OR) — `0 | 1 = 1`; anything | 1 = 1 → sets bits ~ (NOT) — inverts all bits → complement << (left shift) — multiply by 2 per shift → moves bits left >> (right shift) — divide by 2 per shift → moves bits right Example: isolate bits 2-4 of byte `0b10110110`: mask = $((1 << (4-2)) - 1) << 2 = (4-1) << 2 = 3 << 2 = 0b00001100$ result = $(0b10110110 \& 0b00001100) >> 2 = 0b00000001$

2	<code>size_t byteIndex = startBit / 8;</code>	Find which byte contains the bit we want. Bits are numbered 0..N. Bit 13 is in byte $13/8 = \text{byte } 1$ (integer division).
3	<code>size_t bitStart = startBit % 8;</code>	Offset within the byte. Bit 13 is at position $13\%8 = 5$ within byte 1.
5	<code>uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;</code>	Build a bitmask covering exactly the bits we want to write. '1u' is unsigned 1 to prevent signed overflow on shift. $(1u << \text{width}) - 1$ creates width consecutive 1-bits starting at bit 0. Shifting left by bitStart moves them to the correct position. Example: startBit=2, endBit=5: <code>mask = ((1<<3)-1) << 2 = 0b00011100</code>
6	<code>byteArray[byteIndex] = (byteArray[byteIndex] & ~mask) ((data << bitStart) & mask);</code>	Three steps in one line: 1. 'byteArray[byteIndex] & ~mask' — clear the target bits (AND with inverted mask) 2. '(data << bitStart) & mask' — place the new data in the correct bit positions 3. '... ...' — OR them together to get the final byte with the new bits set This is the standard 'clear-then-set' bit manipulation pattern.

Section 3.5 — Relations With Other Files

File	Direction	Why
DTypes.h/.c	→ depends on	<string.h> for memcpy, Common.h for SOURCE_FILE
Arithmetic.c	← uses DTypes	All element read/write: readBytesAsInt32/Float, writeXxx
Matmul.c	← uses DTypes	Reading matrix elements, writing results
TensorConversion.c	← uses DTypes	Reading/writing during type conversion
Sgd.c	← uses DTypes	Reading/writing weight and gradient values
Serialize.c / Deserialize.c	← uses DTypes	Writing tensor bytes to/from byte stream
Log.c / Sum.c / MinMax.c	← uses DTypes	Reading element values for reduction ops
Bernoulli.c / Distributions.c	← uses DTypes	Writing sampled values to tensors

■ RigL GAP IN THIS FILE

DTypes.h/.c has no direct RigL gap. However, RigL uses a sparsity mask (a tensor of booleans — one bit per weight). `writeBitsToByteArray` and `readBitsFromByteArray` are exactly the functions needed to work with this mask. They already exist! The gap is not in DTypes but in the higher-level RigL logic that uses them.

Chapter 4

Layer 2 — Quantization.h / Quantization.c

The Quantization Type System

Section 4.1 — What This File Does

■ WHAT THIS FILE DOES

Quantization.h defines HOW numbers are stored in every tensor. It provides three things: (1) the `qtype_t` enum — six possible storage formats; (2) config structs — parameters for each format (scale, zero-point, bit depth); (3) init functions — create and configure quantization objects. Every tensor carries a `quantization_t` that says 'my values are stored as X'. This is Layer 2 because all higher layers need to know the storage format before they can read or write tensor values.

Understanding quantization is essential for understanding the whole library. A tensor does not just store raw numbers — it stores numbers in a compressed format. The `quantization_t` attached to each tensor tells you how to decode those numbers back to their real values.

■ C CONCEPT: What is Quantization? (Plain Language)

Quantization is the process of representing a continuous value (like 0.12345) as a small integer (like 12) plus a scale factor (0.01). Real value = integer × scale $0.12 = 12 \times 0.01$ Why bother? Because integers use less memory and are faster to compute: float (32 bits) × float (32 bits) → 32-bit result (needs FPU, 1 cycle) int8 (8 bits) × int8 (8 bits) → 16-bit result (integer mult, ~0.5 cycles) The price you pay is precision loss. 0.12345 becomes 0.12 (rounded to 2 decimals). FQT (Fixed-point Quantization Training) trains the model to be accurate despite this precision loss.

Section 4.2 — Quantization.h: The Header File

[illegible]

Section 4.3 — Line-by-Line: The `qtype_t` Enum

■ C CONCEPT: typedef enum — Defining an Enumeration Type

An enum (enumeration) is a type whose values are named integer constants. `typedef enum myEnum { A, B, C } myEnum_t;` A=0, B=1, C=2 automatically (or you can assign values manually: A=5). 'typedef' creates a type alias so you can write 'myEnum_t x' instead of 'enum myEnum x'. Enums are useful because they let you write readable code: `if (q->type == FLOAT32) { ... } ← clear` if `(q->type == 1) { ... } ← confusing` — what does 1 mean?

INT32	INT32 — raw 32-bit integer	The value IS the integer, no scale factor. Used for: raw index tensors, intermediate accumulation buffers. Memory: 4 bytes per element. No extra config (int32QConfig_t is empty).
FLOAT32	FLOAT32 — IEEE 754 float	The standard floating-point format. Used for: weights during FLOAT32 training, gradients, activations when using float paths. Memory: 4 bytes per element. No extra config (float32QConfig_t is empty). This is the simplest and most precise option, but uses the most memory.
SYM_INT32	SYM_INT32 — symmetric fixed-point, 32-bit mantissa	Real value = mantissa × scale. The mantissa is stored as int32_t (4 bytes). Scale is a float stored in symInt32QConfig_t. qMaxBits limits the range: if qMaxBits=16, mantissas stay in [-32767, +32767]. Used for: FQT training — both weights and activations. Centred at zero — no zero-point needed.
SYM	SYM — symmetric, sub-32-bit mantissa	Same as SYM_INT32 but qMaxBits can be 8, 4, 2, etc. Allows even more aggressive compression. Used for inference-only deployments.
ASYM	ASYM — asymmetric fixed-point	Real value = (mantissa - zeroPoint) × scale. The zeroPoint allows the range to be shifted: e.g. ReLU outputs are always ≥ 0, so ASYM can map integer 0 to real 0.0 and cover [0, 2×scale×qMax] efficiently. Used for activations after ReLU in pure-integer inference.
BOOL	BOOL — 1-bit boolean	Each element is 0 or 1, stored as 1 bit (8 elements per byte). Used for: sparsity masks (which weights are active), Bernoulli dropout masks. Memory: (N + 7) / 8 bytes for N elements (ceiling division).

4.3.1 The quantization_t Struct

■ C CONCEPT: void* — The Generic Pointer

`void*` is a pointer to 'nothing in particular' — it can point to any type. You cannot dereference a `void*` directly (the compiler doesn't know the size). You must cast it to the correct type first: `void *ptr = &myCfg; // store any address`
`symInt32QConfig_t *cfg = ptr; // retrieve as specific type` In `quantization_t`, `qConfig` is `void*` because it might point to any of six config struct types depending on `q->type`. The code always checks `q->type` first, then casts `qConfig` to the appropriate type. This is a C polymorphism pattern.

Field	Type	Meaning
type	qtype_t	Which storage format: INT32, FLOAT32, SYM_INT32, SYM, ASYM, or BOOL
qConfig	void*	Pointer to the format-specific config struct. Must be cast to the right type based on 'type'. NULL for INT32 and FLOAT32 (they need no config).

— ■ WORKED EXAMPLE

How to read a SYM_INT32 tensor's scale: `quantization_t *q = tensor->quantization; // get quantization` if `(q->type == SYM_INT32)` { // check the type FIRST `symInt32QConfig_t *cfg = q->qConfig;` // cast `void*` to correct type `float scale = cfg->scale;` // now safe to read scale } Why check the type first? Because if `q->type` were `FLOAT32`, `qConfig` would point to a `float32QConfig_t` (an empty struct), and reading scale from it would give garbage.

4.3.2 symInt32QConfig_t — Configuration for SYM_INT32

Field	Type	Meaning
scale	float	The scale factor. $\text{real_value} = \text{mantissa} \times \text{scale}$. Updated by requantisation.
qMaxBits	uint8_t	Bit depth of the mantissa. e.g. 16 $\rightarrow$ mantissa in $[-32767, 32767]$. 8 $\rightarrow$ mantissa in $[-127, 127]$.
roundingMode	uint8_t	How to round when converting: HALF_AWAY (standard) or SR_HALF_AWAY (stochastic, reduces bias during FQT).

4.3.3 calcNumberOfBytesForData — Memory Size Calculator

This function tells you how many bytes a tensor needs based on its quantization type and number of elements. It is used when allocating tensor data buffers.

```
size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements) {
    switch(q->type) {
        case INT32: return numberOfElements * sizeof(int32_t); // 4N bytes
        case FLOAT32: return numberOfElements * sizeof(float); // 4N bytes
        case SYM_INT32: return numberOfElements * sizeof(int32_t); // 4N bytes
        case BOOL: return (numberOfElements + 7) / 8; // ceil(N/8) bytes
        // SYM/ASYM: depends on qMaxBits...
        default: PRINT_ERROR("Unknown qtype"); exit(1);
    }
}
```

4N byte s for I NT3 2/FL OA T32/ SY M_I NT3 2	All three use 4 bytes per element (32-bit storage).	Despite different semantics (raw int, float, scaled int), they all store one 32-bit value per element. The difference is only in how you interpret those bits.
(N+ 7)/8 for BO OL	Ceiling division: pack 8 booleans per byte.	(N+7)/8 is the ceiling of N/8. For N=10: (10+7)/8 = 17/8 = 2 bytes. Bit 0 of byte 0 is element 0; bit 7 of byte 0 is element 7; bit 0 of byte 1 is element 8.

■ WORKED EXAMPLE

Memory comparison for a 1000-element tensor: FLOAT32: $1000 \times 4 = 4000$ bytes (3.9 KB) SYM_INT32: $1000 \times 4 = 4000$ bytes (3.9 KB) — same space as FLOAT32! SYM (8-bit): $1000 \times 1 = 1000$ bytes (1.0 KB) — 4× smaller BOOL: $(1000+7)/8 = 125$ bytes (0.1 KB) — 32× smaller than float SYM_INT32 saves memory over FLOAT32 ONLY if $q\text{MaxBits} < 32$, or if the precision loss allows fewer bits. Otherwise the memory is the same but you gain in compute speed on integer-only hardware.

Section 4.4 — The Init Functions

Every quantization_t must be initialised before use. The init functions set the type field and configure the qConfig pointer. Here is the implementation of the most important ones:

```
// From Quantization.c:
void initInt32Quantization(quantization_t *q) {
    q->type = INT32;
    q->qConfig = NULL; // INT32 needs no config
}
void initFloat32Quantization(quantization_t *q) {
    q->type = FLOAT32;
    q->qConfig = NULL; // FLOAT32 needs no config
}
void initSymInt32QConfig(roundingMode_t rm, symInt32QConfig_t *cfg) {
    cfg->scale = 1.0f; // neutral scale: real value = mantissa × 1 = mantissa
    cfg->qMaxBits = 16; // default: 16-bit mantissa range
    cfg->roundingMode = rm;
}
void initSymInt32Quantization(quantization_t *q, symInt32QConfig_t *cfg) {
    q->type = SYM_INT32;
    q->qConfig = cfg; // store pointer to caller-provided config
}
```

q->type = INT32;	Set the type field to indicate raw integer storage.	This tells any code reading this tensor: 'read these bytes as int32_t, no conversion.'
q->qConfig = NULL;	NULL means 'no config struct'. INT32 and FLOAT32 need no extra parameters.	Any code that checks q->type first and only reads qConfig for SYM_INT32 will never accidentally dereference this NULL pointer.
cfg->scale = 1.0f;	Initialize scale to 1.0 (neutral). With scale=1.0, real = mantissa × 1 = mantissa.	The scale will be updated to a meaningful value after the first data is loaded or after the first requantisation pass. Starting at 1.0 is safe.
cfg->qMaxBits = 16;	Default: 16-bit mantissa range ([-32767, 32767]).	$qMax = 2^{(qMaxBits-1)} - 1 = 2^{15} - 1 = 32767$. This is a good balance: small enough to save memory, large enough for training precision.
q->qConfig = cfg;	Store the pointer to the caller's config struct.	The quantization_t does NOT own the config — it just stores the address. The caller must ensure cfg stays valid as long as the tensor exists.

Section 4.5 — Relations With Other Files

File	Direction	Why
Quantization.h/.c	→ depends on	DTypes.h (indirectly), stdint.h, stddef.h
Tensor.h	← includes Quantization.h	tensor_t has a quantization_t* field
TensorConversion	← uses Quantization	Reads scale, qMaxBits, type to do type conversion
Arithmetic.c / Matmul.c	← uses Quantization	Reads scale to convert int accumulations to real
Sgd.c	← uses Quantization	Checks q->type to dispatch float vs SYM_INT32 update

LinearApi.c / Conv1dApi.c	← uses Quantization	Creates quantization_t objects for each tensor slot
LossFunction.h / MSE.c	← uses Quantization	Checks tensor type for forward/backward dispatch

■ RigL GAP IN THIS FILE

Quantization.h needs no changes for basic RigL. However, the BOOL type (sparsity mask) is critical for RigL: each weight's 'active' bit is stored as a BOOL tensor. The existing BOOL infrastructure in Quantization.h/DTypes.c is sufficient. What is MISSING for RigL is not in this file but in the arithmetic layer: the mask-aware forward pass (skip multiplications for inactive weights) and the gradient tracking for inactive weights are not yet implemented.

Chapter 5

Layer 3 — DataStorage.h and Tensor.h / Tensor.c

The Central Data Abstraction

Section 5.1 — DataStorage.h: The Dataset Container

■ WHAT THIS FILE DOES

DataStorage.h defines how raw datasets (training examples) are stored in memory. It provides two structs: `dataEntry_t` (one item in the dataset) and `dataStorage_t` (a collection of items). The `DataLoader` uses `dataStorage_t` to iterate over training data. `DataStorage.c` is currently empty — the structs are the interface; the `DataLoader` implements the logic.

```
// DataStorage.h – src/data_loader/include/DataStorage.h
#ifndef DATASTORAGE_H
#define DATASTORAGE_H
#include <stddef.h>
#include <stdint.h>
typedef struct dataEntry {
    uint8_t *dataPTR; // pointer to raw bytes for this entry
    size_t numberOfElements; // how many elements the entry contains
} dataEntry_t;
typedef struct dataStorage {
    uint8_t *data; // flat byte buffer holding all entries
    size_t size; // total bytes in the buffer
    dataEntry_t *entries; // array of entry descriptors
    size_t numberOfEntries; // how many entries are in this dataset
} dataStorage_t;
#endif // DATASTORAGE_H
```

5.1.1 dataEntry_t — One Training Example

Field	Type	Meaning
dataPTR	uint8_t*	Pointer to this entry's raw bytes inside the flat buffer. This is NOT a copy — it points into the <code>dataStorage_t</code> 's data buffer.
numberOfElements	size_t	How many elements this entry contains. e.g. for a 1D signal of length 128, <code>numberOfElements = 128</code> .

5.1.2 dataStorage_t — The Full Dataset

Field	Type	Meaning
data	uint8_t*	The flat byte buffer. All entries are packed here consecutively. <code>dataEntry_t::dataPTR</code> points into this buffer.
size	size_t	Total byte count of the data buffer.
entries	dataEntry_t*	Array of <code>numberOfEntries</code> <code>dataEntry_t</code> descriptors. <code>entries[i].dataPTR = data + i * bytesPerEntry</code> .
numberOfEntries	size_t	The dataset size. The <code>DataLoader</code> iterates <code>[0, numberOfEntries)</code> .

■ C CONCEPT: Pointer-into-buffer Pattern

Instead of allocating memory separately for each training example, all data lives in ONE flat buffer. Entry descriptors then hold pointers into that buffer. This is memory-efficient (no malloc per entry) and cache-friendly (all data is contiguous in memory). Example: 4 entries, each 10 bytes: data = [e0_bytes... e1_bytes... e2_bytes... e3_bytes...] // 40 bytes total entries[0].dataPTR = &data[0]; // points to start of entry 0 entries[1].dataPTR = &data[10]; // points to start of entry 1 entries[2].dataPTR = &data[20]; // ... The DataLoader then passes entries[i].dataPTR to the getSample callback which reads it and writes into the batch tensor.

Section 5.2 — Tensor.h: The Central Data Structure

■ WHAT THIS FILE DOES

Tensor.h defines the most important struct in the entire library: `tensor_t`. A tensor is an N-dimensional array of numbers. In ML, every piece of data — inputs, weights, biases, gradients, activations — is stored as a tensor. Tensor.h also defines `shape_t` (the dimensions), `parameter_t` (a tensor pair for weight + gradient), `sparsity_t` (future sparse support), and all utility functions that work on tensors.

■ C CONCEPT: What is a Tensor?

A tensor is a generalization of arrays: Scalar (0D): a single number — 42.0 Vector (1D): [1.0, 2.0, 3.0] — shape [3] Matrix (2D): [[1,2],[3,4],[5,6]] — shape [3,2] 3D tensor: a batch of matrices — shape [batch, rows, cols] In neural networks: Input batch: shape [batch_size, input_features] Linear weights: shape [output_features, input_features] Conv1d weights: shape [out_channels, in_channels, kernel_size] In memory, all tensors are stored as a FLAT 1D array of bytes. The `shape_t` tells you how to interpret the flat array as N-dimensional.

```
// Tensor.h — src/tensor/include/Tensor.h (key structs)
typedef struct shape {
    size_t numberOfDimensions; // e.g. 2 for a matrix
    size_t *dimensions; // e.g. {3, 4} for a 3x4 matrix
    size_t *orderOfDimensions; // permutation for transposed views
} shape_t;
typedef struct tensor {
    uint8_t *data; // flat byte array
    shape_t *shape; // how to interpret the data
    quantization_t *quantization; // how values are encoded
    sparsity_t *sparsity; // sparse pattern (future use)
} tensor_t;
typedef struct parameter {
    tensor_t *param; // the weight tensor
    tensor_t *grad; // its gradient tensor (same shape)
} parameter_t;
```

Section 5.3 — shape_t in Detail

Field	Type	Meaning
numberOfDimensions	size_t	The number of axes. 1=vector, 2=matrix, 3=3D tensor, etc.
dimensions	size_t*	Array of size numberOfDimensions. dimensions[i] = length along axis i. For shape [3,4]: dimensions={3,4}, totalElements=3*4=12.
orderOfDimensions	size_t*	Permutation of [0..N-1]. Default: {0,1,2,...}. After transposeTensor(t,0,1), orderOfDimensions={1,0}. This enables O(1) transpose without moving data.

■ C CONCEPT: O(1) Transpose — No Data Movement

Transposing a matrix traditionally means moving all elements. For a 1000×1000 matrix, that's 1,000,000 copies. ODT uses a smarter approach: store a permutation of axis indices. When you transpose axes 0 and 1, instead of moving data, swap the axis order in `orderOfDims`. Accessing element `[i][j]` of the transposed matrix then maps to `[j][i]` of the original data. The indexing function applies the permutation. This is O(1): swapping two numbers regardless of tensor size. Essential for on-device training where moving data wastes memory and time.

5.3.1 `setShape` — Initialize a `shape_t`

```
// setShape fills in shape_t from caller-provided arrays
void setShape(shape_t *shape, size_t *dims, size_t numberOfDims,
size_t *orderOfDims) {
    shape->numberOfDimensions = numberOfDims;
    shape->dimensions = dims;
    shape->orderOfDimensions = orderOfDims;
}
// Typical usage for a 3x4 matrix:
size_t dims[2] = {3, 4};
size_t order[2] = {0, 1}; // row-major (no transpose)
shape_t myShape;
setShape(&myShape, dims, 2, order);
```

■ NOTE

`setShape` does NOT allocate memory. It stores the pointers you provide. The caller is responsible for keeping `dims[]` and `orderOfDims[]` arrays alive. In practice they are declared as static or stack arrays in the user's setup code.

5.3.2 `setOrderOfDimsForNewTensor` — Default Order

```
// Fill orderOfDimensions with the identity permutation {0, 1, 2, ...}
void setOrderOfDimsForNewTensor(size_t numberOfDimensions,
size_t *orderOfDimensions) {
    for (size_t i = 0; i < numberOfDimensions; i++) {
        orderOfDimensions[i] = i;
    }
}
```

for loop	for (size_t i = 0; i < numberOfDimensions; i++)	Iterate over all dimension indices, 0 to N-1.
body	orderOfDimensions[i] = i;	Set each axis to point to itself: no reordering. For a 3D tensor: <code>orderOfDimensions = {0,1,2}</code> means 'dimension 0 is still first, dimension 1 is still second, etc.'

Section 5.4 — `tensor_t` Field by Field

Field	Type	Meaning
data	uint8_t*	Raw bytes. For a FLOAT32 tensor of 12 elements: 48 bytes. For SYM_INT32: also 48 bytes (int32_t per element). For BOOL: ceil(N/8) bytes. This field is NEVER NULL for a valid tensor.
shape	shape_t*	Pointer to the shape descriptor. Gives dimensions and axis order. May be shared across multiple tensors (view semantics).
quantization	quantization_t*	Pointer to quantization. Tells you type (FLOAT32, SYM_INT32, etc.) and config (scale, qMaxBits, etc.). Must be cast correctly before use.
sparsity	sparsity_t*	Pointer to sparsity config. NULL for dense tensors (current state). Will hold sparsity mask location for RigL. Currently unused.

— ■ WORKED EXAMPLE

Full tensor setup for a 3x4 FLOAT32 weight matrix: // 1. Allocate the data buffer (12 floats * 4 bytes = 48 bytes) static uint8_t weightData[48]; // 2. Set up quantization static quantization_t weightQ; initFloat32Quantization(&weightQ); // type=FLOAT32, qConfig=NULL // 3. Set up shape static size_t dims[2] = {3, 4}; static size_t order[2] = {0, 1}; static shape_t weightShape; setShape(&weightShape, dims, 2, order); // 4. Assemble the tensor static tensor_t weightTensor; setTensorValues(&weightTensor, weightData, &weightShape, &weightQ, NULL); // Now weightTensor.data points to weightData (48 bytes of floats) // weightTensor.shape->dimensions[0] = 3, [1] = 4 // weightTensor.quantization->type = FLOAT32

Section 5.5 — Key Functions in Tensor.c

5.5.1 calcNumberOfElementsByShape

```
size_t calcNumberOfElementsByShape(shape_t *shape) {
    size_t numElem = 1;
    size_t numberOfDimensions = shape->numberOfDimensions;
    size_t *dimensions = shape->dimensions;
    for (size_t i = 0; i < numberOfDimensions; i++) {
        numElem *= dimensions[i];
    }
    return numElem;
}
```

size_t numElem = 1;	Start with 1 (multiplicative identity).	We multiply all dimension sizes together. Starting at 0 would give 0.
for loop	for (size_t i = 0; i < numberOfDimensions; i++)	Iterate over all dimensions.
numElem *= dimensions[i];	Multiply running product by this dimension's size.	For shape {3,4,5}: $1 \times 3 = 3 \rightarrow 3 \times 4 = 12 \rightarrow 12 \times 5 = 60$ elements total.
return numElem;	Return total element count.	A [3,4] matrix has 12 elements. A [2,3,4] cube has 24 elements.

5.5.2 calcBytesPerElement — Type-Aware Size

```

size_t calcBytesPerElement(quantization_t *quantization) {
switch (quantization->type) {
case INT32: return sizeof(int32_t); // 4 bytes
case FLOAT32: return sizeof(float); // 4 bytes
case SYM_INT32: return sizeof(int32_t); // 4 bytes
case SYM: {
symQConfig_t *cfg = quantization->qConfig;
return (size_t)ceilf((float)cfg->qBits / 8.0f); // e.g. 8 bits = 1 byte
}
case ASYM: {
asymQConfig_t *cfg = quantization->qConfig;
return (size_t)ceilf((float)cfg->qBits / 8.0f);
}
case BOOL: return 1; // 1 byte even though each element is 1 bit
default: PRINT_ERROR("Unknown QType!"); exit(1);
}
}
}

```

■ C CONCEPT: switch Statement in C

switch(expression) { case VALUE: ...; break; } tests an integer expression against multiple constant values. switch (x) { case 0: printf("zero"); break; case 1: printf("one"); break; default: printf("other"); break; } 'break' exits the switch. Without break, execution 'falls through' to the next case (a common C gotcha). The { ... } after a case with local variables (like cfg) creates a new scope to avoid redeclaration errors.

5.5.3 transposeTensor — O(1) Axis Swap

```

void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index) {
size_t *order = tensor->shape->orderOfDimensions;
size_t *dims = tensor->shape->dimensions;
// Swap the two entries in orderOfDimensions
size_t tmp = order[dim0Index];
order[dim0Index] = order[dim1Index];
order[dim1Index] = tmp;
// Also swap the physical dimension sizes
size_t tmpDim = dims[dim0Index];
dims[dim0Index] = dims[dim1Index];
dims[dim1Index] = tmpDim;
}

```

size_t tmp = order[dim0Index];	Save the value we're about to overwrite (classic three-way swap).	In C, a = b; b = a; would lose b's value. We need a temporary variable tmp to hold one value while we swap.
order[dim0Index] = order[dim1Index];	Move dim1's axis number into dim0's slot.	

order[dim1index] = tmp;	Move the saved dim0 value into dim1's slot.	After these three lines, the two entries are swapped.
dim swap	Also swap the sizes in the dimensions array.	If you transpose a [3,4] matrix, the shape becomes [4,3]. Swapping dims[] reflects this without moving data.

5.5.4 copyTensor and tensorBoolGet/Set

```
// copyTensor: copy data bytes between two tensors of the same size
void copyTensor(tensor_t *dest, tensor_t *src) {
    size_t bytes = calcBytesPerTensor(src);
    memcpy(dest->data, src->data, bytes);
}
// tensorBoolGet: read a single 1-bit boolean from a BOOL tensor
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {
    size_t byteIdx = flatIndex / 8;
    size_t bitIdx = flatIndex % 8;
    return (tensor->data[byteIdx] >> bitIdx) & 1u;
}
// tensorBoolSet: write a single 1-bit boolean into a BOOL tensor
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {
    size_t byteIdx = flatIndex / 8;
    size_t bitIdx = flatIndex % 8;
    if (value)
        tensor->data[byteIdx] |= (1u << bitIdx); // set bit
    else
        tensor->data[byteIdx] &= ~(1u << bitIdx); // clear bit
}
```

■ WORKED EXAMPLE

Reading element 13 from a BOOL tensor: flatIndex = 13 byteIdx = 13 / 8 = 1 (element 13 is in byte 1) bitIdx = 13 % 8 = 5 (it is bit 5 of that byte) data[1] = 0b10100000 (example) data[1] >> 5 = 0b00000101 ... & 1 = 1 → element 13 is TRUE Writing FALSE to element 13: mask = ~(1u << 5) = ~0b00100000 = 0b11011111 data[1] &= 0b11011111 → bit 5 cleared, other bits unchanged

Section 5.6 — parameter_t: Weights and Gradients Together

A parameter_t bundles a weight tensor with its gradient tensor. In neural network training, every learnable weight has a corresponding gradient of the same shape. The optimizer (SGD) needs both: param to update and grad to read. Pairing them reduces the number of pointers you need to pass around.

```
typedef struct parameter {
    tensor_t *param; // the weight tensor w
    tensor_t *grad; // gradient dL/dw (same shape as param)
} parameter_t;
// Helper accessors (inline, never fail):
tensor_t *getParamFromParameter(parameter_t *p) { return p->param; }
tensor_t *getGradFromParameter(parameter_t *p) { return p->grad; }
```

— ■ WORKED EXAMPLE

A linear layer has two parameter_t objects: one for weights, one for biases. linearConfig_t has: parameter_t weights → weights.param = weight tensor, weights.grad = dL/dW parameter_t bias → bias.param = bias tensor, bias.grad = dL/db During backward pass: linearCalcWeightGrads writes into weights.grad linearCalcBiasGrads writes into bias.grad During optimizer step: sgdStep reads weights.grad and updates weights.param sgdStep reads bias.grad and updates bias.param

Section 5.7 — sparsity_t: The RigL Hook

```
typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t;
typedef struct sparsityConfig {
    // currently empty
} sparsityConfig;
typedef struct sparsity {
    sparsityType_t type;
    sparsityConfig *config;
} sparsity_t;
// In tensor_t:
// sparsity_t *sparsity; // NULL for dense tensors
```

■ RigL GAP IN THIS FILE

The sparsity_t field in tensor_t is the planned integration point for RigL. Currently it is an empty stub — the config struct has no fields and is never read. To implement RigL you would: 1. Add a 'mask' field to sparsityConfig: a tensor_t* of type BOOL, same shape as the weight tensor. mask[i]=1 means weight i is active; 0 = pruned. 2. Add a 'sparsityRatio' float field: the target fraction of pruned weights. 3. In linearForward, check sparsity != NULL and skip elements where mask=0. 4. In the training loop, add a RigL update step that sorts weights by magnitude, prunes the smallest, and re-activates weights where the gradient is largest. The hook (sparsity_t *sparsity field) already exists in tensor_t. Only the mask logic, the arithmetic fast-path, and the RigL update step are missing.

Section 5.8 — TensorConversion.h / TensorConversion.c

■ WHAT THIS FILE DOES

TensorConversion provides the convertTensor function and the requantSymInt32Tensor functions. convertTensor converts a tensor from one quantization type to another (e.g. FLOAT32 → SYM_INT32 or SYM_INT32 → FLOAT32). This is essential for the FQT training loop: forward pass works in SYM_INT32, but intermediate buffers may need to be converted for loss computation or debug output.

```
// TensorConversion.h key declarations
// Function pointer type for a single type conversion
typedef void (*conversionFunction_t)(tensor_t *input, tensor_t *output);
// The dispatch table: conversionMatrix[fromType][toType]
extern conversionFunction_t conversionMatrix[6][6];
// Main entry point:
void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor);
// Requantize SYM_INT32 → SYM_INT32 with fresh dynamic scale (two-pass):
void requantSymInt32Tensor(tensor_t *input, tensor_t *output);
// Requantize SYM_INT32 → SYM_INT32 into a caller-set target scale:
void requantSymInt32TensorToScale(tensor_t *input, tensor_t *output);
```

■ C CONCEPT: 2D Function Pointer Table — `conversionMatrix[6][6]`

Instead of a giant if/else chain for every pair of types, `TensorConversion` uses a 6×6 table indexed by (inputType, outputType). `conversionMatrix[SYM_INT32][FLOAT32] = &symInt32ToFloat;`
`conversionMatrix[FLOAT32][SYM_INT32] = &floatToSymInt32;` ... `convertTensor` then does:
`conversionMatrix[input->quantization->type][output->quantization->type](input, output);` This is O(1) dispatch with no branching. Adding a new type only requires filling in the relevant row/column of the table.

5.8.1 `requantSymInt32Tensor` — The Two-Pass Algorithm

`requantSymInt32Tensor` is the most important conversion function. It re-encodes a `SYM_INT32` tensor into a new `SYM_INT32` tensor with an automatically computed scale. The algorithm is the core of FQT.

■ C CONCEPT: Two-Pass Requantisation (FQT, Deutel 2025)

Pass A — Find the scale: $\text{absMax} = \max \text{ over all } i \text{ of } |\text{mantissa}_i \times \text{inScale}|$ $\text{newScale} = \text{absMax} / \text{qMax}$ (where $\text{qMax} = 2^{(\text{qMaxBits}-1)} - 1$) e.g.: $\text{absMax}=0.95$, $\text{qMaxBits}=16 \rightarrow \text{qMax}=32767 \rightarrow \text{newScale}=0.95/32767 \approx 2.9\text{e-}5$ Pass B — Re-encode: for each i : $\text{out}_i = \text{clamp}(\text{round}(\text{realValue}_i / \text{newScale}), -\text{qMax}, \text{qMax})$ where $\text{realValue}_i = \text{mantissa}_i \times \text{inScale}$ Why two passes? Because you need to know the maximum value (Pass A) BEFORE you can compute the scale, and you need the scale BEFORE you can re-encode any value (Pass B). You cannot do this in one pass. IN-PLACE SAFE: both in and out can point to the same tensor, because Pass A only reads and Pass B reads-then-writes in the same order.

Section 5.9 — Relations With Other Files

File	Direction	Why
<code>Tensor.h/.c</code>	→ depends on	<code>Quantization.h</code> , <code>DTypes.h</code> , <code>Common.h</code> , <code>MinMax.h</code>
<code>Arithmetic.c / Matmul.c</code>	← uses Tensor	All operations take <code>tensor_t*</code> arguments
All Layer .c files	← uses Tensor	forward/backward operate on <code>tensor_t*</code>
<code>Sgd.c / LossFunction</code>	← uses Tensor	Read/write weights and gradients
<code>TensorConversion</code>	← uses Tensor	Converts <code>tensor_t</code> from one type to another
<code>DataLoader.c</code>	← uses Tensor + <code>DataStorage</code>	Loads data into batch <code>tensor_t</code>
<code>Serialize.c</code>	← uses Tensor	Saves <code>tensor_t</code> fields to byte stream

Chapter 6

Layer 4 — The Arithmetic Engine

Arithmetic.h/c, Add, Sub, Mul, Div, Matmul, MinMax, Sum, Log, Rounding

Section 6.1 — Overview of the Arithmetic Layer

■ WHAT THIS FILE DOES

The Arithmetic layer is the computational heart of the library. It provides element-wise operations (add, subtract, multiply, divide), matrix multiplication (matmul), reductions (min, max, absMax, sum), and utilities (log, rounding, clamping). All operations are type-dispatched: separate implementations exist for FLOAT32 and INT32/SYM_INT32, so the library can run in either mode. The key design is that all functions take `tensor_t*` arguments — they do not care how many elements there are or what shape the tensor has.

Module	Key Functions	Purpose
Arithmetic.c	int32PointWiseArithmetic, floatPointWiseArithmetic, calcIndicesByRawIndex, calcTensorIndexByIndices	Generic element-wise engine with index calculations
Add.h/.c	addInt32Tensors, addFloat32Tensors, addSymInt32Tensors, addInt32ElementWithInt32Tensor	Tensor addition (point-wise and element-scalar)
Sub.h/.c	subInt32Tensors, subFloat32Tensors, subSymInt32Tensors	Tensor subtraction
Mul.h/.c	mulInt32Tensors, mulFloat32Tensors, mulSymInt32Tensors	Tensor multiplication
Div.h/.c	divInt32Tensors, divFloat32Tensors	Tensor division
Matmul.h/.c	matmulInt32Tensors, matmulFloat32TensorsWithBias, matmulSymInt32TensorsWithBias	Matrix multiply ($O(N^3)$ loop)
MinMax.h/.c	findAbsMaxFloat, findMaxInt32, findMinFloat	Reduction: find extreme values
Sum.h/.c	sumint32, sumFloat	Reduction: sum all elements
Rounding.h/.c	roundByMode, clamp	Rounding modes + range clamp
Log.h/.c	logFloat	Natural logarithm of a float
Comparison.h/.c	gteFloatValue, gteSymInt32Zero	Greater-than-or-equal operations

Section 6.2 — Arithmetic.c: Index Calculation Functions

Before element-wise operations can work on multi-dimensional tensors, the library needs a way to convert between flat memory indices and multi-dimensional (row, column, ...) indices. Arithmetic.c provides three functions for this.

6.2.1 calcTensorIndexByIndices — Indices to Flat Index

```

// Given per-dimension indices, compute the flat memory offset.
// Example: shape [3,4], indices [1,2] → element at row 1, col 2
// flat index = 1*4 + 2 = 6 (row-major order)
size_t calcTensorIndexByIndices(size_t numberOfDimensions,
size_t *dimensions,
size_t *indices) {
size_t index = indices[numberOfDimensions - 1];
size_t offset = dimensions[numberOfDimensions - 1];
for (int i = (int)numberOfDimensions - 2; i >= 0; i--) {
index += indices[i] * offset;
offset *= dimensions[i];
}
return index;
}

```

■ C CONCEPT: Row-Major Memory Layout

In row-major (C) order, the LAST dimension changes fastest in memory. Matrix A [3 rows, 4 cols] in memory: A[0][0] A[0][1] A[0][2] A[0][3] A[1][0] A[1][1] ... flat: 0 1 2 3 4 5 ... Formula: flat = i*cols + j for a 2D matrix [rows, cols] For 3D [D,H,W]: flat = i*(H*W) + j*W + k The algorithm generalises this: starting from the last dimension, it accumulates the contribution of each index weighted by the product of all following dimensions.

1	size_t index = indices[numberOfDimensions - 1];	Start with the innermost index directly. For [3,4], indices=[1,2]: index = indices[1] = 2
2	size_t offset = dimensions[numberOfDimensions - 1];	Track the stride for the next outer dimension. For [3,4]: offset = dimensions[1] = 4 (we have 4 columns)
3	for (int i = ...; i >= 0; i--)	Walk backwards through dimensions from second-to-last to first. Note: i is int (not size_t) so i >= 0 works correctly (size_t is unsigned and would wrap to a huge number when it hits -1).
4	index += indices[i] * offset;	Add this dimension's contribution: index × stride. For i=0: index += indices[0] * 4 = 1 * 4 = 4. index = 2+4 = 6. Correct!
5	offset *= dimensions[i];	Grow the stride by this dimension's size for the next outer dimension.

6.2.2 calcIndicesByRowIndex — Flat Index to Indices

```

// Reverse: given flat index, compute per-dimension indices.
// Example: shape [3,4], rowIndex=6 → indices [1,2]
void calcIndicesByRowIndex(size_t numberOfDims, size_t *dims,
size_t rowIndex, size_t *indices) {
size_t offset = 1;
for (size_t i = 0; i < numberOfDims; i++) {
offset *= dims[i]; // offset = total elements = 3*4 = 12
}
size_t restIndex = rowIndex;
for (size_t i = 0; i < numberOfDims; i++) {
offset /= dims[i]; // each step: stride for this dimension
indices[i] = restIndex / offset;
restIndex -= indices[i] * offset;
}
}

```

— ■ WORKED EXAMPLE

Trace for shape [3,4], rowIndex=6: Phase 1: offset = 1 × 3 × 4 = 12 (total elements) Phase 2: i=0: offset /= 3 → offset=4; indices[0] = 6/4 = 1; restIndex = 6 - 1*4 = 2 i=1: offset /= 4 → offset=1; indices[1] = 2/1 = 2; restIndex = 2 - 2*1 = 0 Result: indices = [1, 2] → element at row 1, column 2 ✓ Intuition: In a 3×4 grid, flat element 6: Row = 6 ÷ 4 = 1 (integer division) Col = 6 mod 4 = 2

6.2.3 int32PointWiseArithmetic — The Generic Engine

This is the most important function in Arithmetic.c. It applies any binary operation (passed as a function pointer) to every pair of corresponding elements in two tensors, respecting the orderOfDimensions for transposed views.

```
void int32PointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor) {
// 1. Validate shapes match
if (!doDimensionsMatch(aTensor, bTensor)) {
PRINT_ERROR("Dimensions don't match!"); exit(1);
}
size_t numberOfElements = calcNumberOfElementsByTensor(aTensor);
size_t numberOfDims = aTensor->shape->numberOfDimensions;
// 2. Pre-compute ordered dims for both tensors
size_t aOrderedDims[numberOfDims]; // VLA: stack-allocated
size_t bOrderedDims[numberOfDims];
orderDims(aTensor, aOrderedDims);
orderDims(bTensor, bOrderedDims);
// 3. For each element in flat order:
for (size_t i = 0; i < numberOfElements; i++) {
size_t aIndices[numberOfDims];
size_t bIndices[numberOfDims];
// Convert flat index i → multi-dim indices for A (using A's ordered dims)
calcIndicesByRowIndex(numberOfDims, aOrderedDims, i, aIndices);
// Convert those indices to a physical memory offset (respecting transpose)
size_t aPhysical = calcElementIndexByIndices(numberOfDims,
aTensor->shape->dimensions, aIndices,
aTensor->shape->orderOfDimensions);
// Same for B
calcIndicesByRowIndex(numberOfDims, bOrderedDims, i, bIndices);
size_t bPhysical = calcElementIndexByIndices(numberOfDims,
bTensor->shape->dimensions, bIndices,
bTensor->shape->orderOfDimensions);
// Read both values, apply op, write result
int32_t aVal = readBytesAsInt32(aTensor->data + aPhysical * sizeof(int32_t));
int32_t bVal = readBytesAsInt32(bTensor->data + bPhysical * sizeof(int32_t));
int32_t result = arithmeticFunc(aVal, bVal);
// Output physical index (same as A's for same-shape tensors)
size_t outPhysical = calcElementIndexByIndices(numberOfDims,
outputTensor->shape->dimensions, aIndices,
outputTensor->shape->orderOfDimensions);
writeInt32ToByteArray(result,
outputTensor->data + outPhysical * sizeof(int32_t));
}
}
```

■ C CONCEPT: Function Pointers as Arguments (Higher-Order Functions)

int32ElementArithmeticFunc_t is defined as: typedef int32_t (*int32ElementArithmeticFunc_t)(int32_t a, int32_t b);

This is a pointer to any function that takes two int32_t and returns int32_t. When you call

int32PointWiseArithmetic(..., addInt32s, ...), the function addInt32s is passed as the arithmeticFunc argument. Inside the loop, arithmeticFunc(aVal, bVal) calls addInt32s(aVal, bVal) — it adds the values. If you instead pass mulInt32s, the same loop multiplies. One loop handles all binary operations: add, sub, mul, div, max, etc. This is the C equivalent of a 'higher-order function' in functional languages.

■■ IMPORTANT

VLAs (Variable Length Arrays): size_t aIndices[numberOfDims] uses a runtime value for the array size. This allocates memory on the STACK, not the heap. The advantage is no malloc/free needed. The risk: if numberOfDims is large and nested calls are deep, you can overflow the stack. On STM32, the stack is typically 4-8 KB. This is safe here because numberOfDims is small (2-4 for typical tensors).

Section 6.3 — Add.h/.c: Tensor Addition

Add.c implements tensor addition by wrapping `int32PointWiseArithmetic` with the `addInt32s` lambda. It also provides special functions for mixed-type additions (e.g. adding an `INT32` gradient to a `SYM_INT32` weight).

```
// The primitive element operation:
int32_t addInt32s(int32_t a, int32_t b) { return a + b; }
float addFloat32s(float a, float b) { return a + b; }
// Point-wise add of two INT32 tensors:
void addInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out) {
    int32PointWiseArithmetic(a, b, addInt32s, out);
}
// Point-wise add of two FLOAT32 tensors (inplace version):
void addFloat32TensorsInplace(tensor_t *a, tensor_t *b) {
    floatPointWiseArithmeticInplace(a, b, addFloat32s);
}
// Mixed: add an INT32 gradient tensor to a SYM_INT32 weight tensor
void addInt32TensorToSymInt32TensorInplace(tensor_t *symInt32T, tensor_t *int32T) {
    size_t n = calcNumberOfElementsByTensor(symInt32T);
    for (size_t i = 0; i < n; i++) {
        int32_t sym = readBytesAsInt32(symInt32T->data + i*4);
        int32_t raw = readBytesAsInt32(int32T->data + i*4);
        writeInt32ToByteArray(sym + raw, symInt32T->data + i*4);
    }
}
```

6.3.1 Why addInt32TensorToSymInt32TensorInplace?

During FQT training, gradients can accumulate as raw `INT32` values (sum of many products), while the weight tensor is `SYM_INT32` (scaled). Adding them directly — mantissa + gradient — works because the gradient has already been divided by the weight's scale before this call. The scale does not change from this addition; only the mantissa changes. This is a key optimization: it avoids re-scaling both tensors to float first.

— ■ WORKED EXAMPLE

Gradient accumulation in FQT: Weight tensor: `SYM_INT32`, scale=0.001, mantissa=1500 (real value = 1.5) Gradient: `INT32`, mantissa=-25 (already computed as delta-mantissa units) `addInt32TensorToSymInt32TensorInplace(weight, gradient)`: new_mantissa = 1500 + (-25) = 1475 real value = 1475 × 0.001 = 1.475 (weight updated by gradient)

Section 6.4 — Matmul.h/.c: Matrix Multiplication

■ WHAT THIS FILE DOES

Matrix multiplication is THE core operation of neural networks. Every linear layer forward pass is a `matmul`. `Matmul.c` provides three variants: `INT32`, `FLOAT32`, and `SYM_INT32`, plus bias-addition variants (`matmulXxxWithBias`). The algorithm is a classic three-nested-loop implementation ($O(N^3)$).

■ C CONCEPT: Matrix Multiplication

$C = A \times B$ where A is $[M, K]$ and B is $[K, N]$: $C[i][j] = \text{sum over } k \text{ of } A[i][k] \times B[k][j]$ For i in $[0..M)$, for j in $[0..N)$, for k in $[0..K)$. In a linear layer: Input X has shape $[\text{batch}, \text{in_features}]$ Weights W have shape $[\text{out_features}, \text{in_features}]$ ($K = \text{in_features}$) Output Y has shape $[\text{batch}, \text{out_features}]$ $Y = X \times W^T$ (transpose of W , so $[\text{in_features}, \text{out_features}]$) Adding bias: $Y[i][j] += \text{bias}[j]$ for all i in batch, j in `out_features`

```
// matmulFloat32TensorsWithBias – the most common forward path
void matmulFloat32TensorsWithBias(tensor_t *A, tensor_t *B,
    tensor_t *out, tensor_t *bias) {
    size_t M = getDimensionsByIndex(A, 0); // batch size
    size_t K = getDimensionsByIndex(A, 1); // in_features
    size_t N = getDimensionsByIndex(B, 0); // out_features
    for (size_t i = 0; i < M; i++) {
        for (size_t j = 0; j < N; j++) {
            float sum = 0.0f;
            for (size_t k = 0; k < K; k++) {
                float aVal = readBytesAsFloat(A->data + (i*K + k)*4);
                float bVal = readBytesAsFloat(B->data + (j*K + k)*4);
                sum += aVal * bVal;
            }
            // Add bias[j]
            float biaVal = readBytesAsFloat(bias->data + j*4);
            writeFloatToByteArray(sum + biaVal, out->data + (i*N + j)*4);
        }
    }
}
```

M, K, N	getDimensionsByIndex: extract dimensions.	M = rows of A (batch size), K = cols of A = rows of B (in_features), N = cols of B (out_features). The output shape is [M, N].
outer loops	for i in [0,M), for j in [0,N)	We compute each output element out[i][j].
inner loop	for k in [0,K): sum += A[i][k] * B[j][k]	Note B is accessed as B[j][k] (weight matrix is stored in [out, in] order, so B[j][k] gives the k-th input weight for output neuron j).
bias	sum + bias[j]	Add the bias for output neuron j. The bias is the same for all batch elements.

6.4.1 matmulSymInt32TensorsWithBias — The FQT Variant

The SYM_INT32 matmul accumulates in a 64-bit integer (int64_t) to avoid overflow. A and B have scales (scaleA and scaleB). The accumulator holds mantissa products; at the end, the real-value sum = mantissa_sum × scaleA × scaleB. This gets requantised to the output's SYM_INT32 format.

```
// Key section from matmulSymInt32TensorsWithBias:
int64_t acc = 0;
for (size_t k = 0; k < K; k++) {
    int32_t aVal = readBytesAsInt32(A->data + (i*K+k)*4);
    int32_t bVal = readBytesAsInt32(B->data + (j*K+k)*4);
    acc += (int64_t)aVal * (int64_t)bVal; // 64-bit accumulate
}
// Convert accumulation to real value, then to output scale
float realSum = (float)acc * scaleA * scaleB;
realSum += readBytesAsFloat(bias->data + j*4) * biasScale;
// Requantise: out[i][j] = round(realSum / outScale)
int32_t quantised = roundByMode(clamp(realSum/outScale, qMin, qMax), rm);
writeInt32ToByteArray(quantised, out->data + (i*N+j)*4);
```

■ C CONCEPT: Why int64_t for Accumulation?

A SYM_INT32 mantissa can be up to 32767 (for qMaxBits=16). Multiplying two 32767 values gives $32767^2 = \sim 1.07$ billion — still fits in int32_t. BUT if you add K of these products (K = in_features, e.g. 128), you get $128 \times 1.07B = 137B$ — this OVERFLOWS int32_t (max $\sim 2.1B$). int64_t holds values up to $\sim 9.2 \times 10^{18}$, so the accumulation is safe for any practical K. The cast (int64_t)aVal × (int64_t)bVal is critical: without the cast, the 32-bit multiplication would overflow before being assigned to the 64-bit accumulator.

Section 6.5 — MinMax.h/.c: Finding Extremes

MinMax provides reduction functions: scan all elements and return the largest, smallest, or absolute largest value. These are used by requantSymInt32Tensor (Pass A) and by gradient clipping.

```
float findAbsMaxFloat(uint8_t *bytes, size_t numberOfElements) {
    float absMax = 0.0f;
    for (size_t i = 0; i < numberOfElements; i++) {
        float val = readBytesAsFloat(bytes + i * sizeof(float));
        if (val < 0) val = -val; // fabsf
        if (val > absMax) absMax = val;
    }
    return absMax;
}

int32_t findMaxInt32(uint8_t *bytes, size_t numberOfElements) {
    int32_t max = readBytesAsInt32(bytes); // first element as initial max
    for (size_t i = 1; i < numberOfElements; i++) {
        int32_t val = readBytesAsInt32(bytes + i * sizeof(int32_t));
        if (val > max) max = val;
    }
    return max;
}
```

float absMax = 0.0f;	Initialize to 0. The absolute maximum is always ≥ 0 .	If we initialized to a very large negative number, we'd need a different comparison.
if (val < 0) val = -val;	Manual absolute value. Equivalent to <code>fabsf(val)</code> .	<code>abs()</code> works on integers; <code>fabsf()</code> is for floats. The manual negation avoids an <code>#include <math.h></code> dependency here.
if (val > absMax) absMax = val;	Update the running maximum.	We want the largest magnitude (absolute value), not the largest signed value.
int32_t max = readBytesAsInt32(bytes);	Seed with the first element (not 0 or INT_MIN).	Seeding with INT32_MIN would work too, but using the actual first element is cleaner and handles tensors with all-negative values correctly.

Section 6.6 — Rounding.h/.c: roundByMode and clamp

```
// Rounding.h:
typedef enum roundingMode { HALF_AWAY, SR_HALF_AWAY } roundingMode_t;
int32_t roundByMode(float input, roundingMode_t roundingMode);
float clamp(float input, float min, float max);
// Implementation (Rounding.c):
int32_t roundByMode(float input, roundingMode_t mode) {
    if (mode == SR_HALF_AWAY) {
        // Stochastic rounding: add uniform noise in [-0.5, 0.5) then round
        float noise = rngNextFloat(&rng) - 0.5f; // in [-0.5, 0.5)
        input += noise;
    }
    return (int32_t)roundf(input); // HALF_AWAY: standard round()
}
float clamp(float input, float min, float max) {
    if (input < min) return min;
    if (input > max) return max;
    return input;
}
```

■ C CONCEPT: Stochastic Rounding (SR_HALF_AWAY) — Why It Matters for FQT

Standard rounding always rounds 0.5 in the same direction. During FQT training, this introduces a systematic BIAS: small gradients consistently round to zero and never accumulate. Stochastic rounding fixes this by adding random noise before rounding: $\text{Round}(0.3 + \text{noise})$ where $\text{noise} \sim \text{Uniform}(-0.5, 0.5)$. With probability 0.3: rounds to 1 (noise > 0.7). With probability 0.7: rounds to 0 (noise < 0.7). On average: $E[\text{result}] = 0.3 \times 1 + 0.7 \times 0 = 0.3$ (the true value!). Standard rounding would always give 0 (biased toward zero). Stochastic rounding is unbiased — it preserves the expected value even after thousands of rounding operations. This is why FQT converges while naive fixed-point training doesn't.

Section 6.7 — Comparison.h/.c: Threshold Operations

Comparison functions implement threshold-based operations: set $\text{output} = \text{input}$ if $\text{input} \geq \text{threshold}$, else $\text{output} = 0$. These are used by the ReLU activation layer.

```
// Comparison.h – key functions:
// Set all elements < threshold to 0, leave others unchanged
void gteFloatValue(tensor_t *input, float threshold, float belowVal,
    tensor_t *output);
// Same but for INT32 tensors
void gteInt32Value(tensor_t *input, int32_t threshold, int32_t belowVal,
    tensor_t *output);
// SYM_INT32 version: compare mantissa to 0 (zero is scale-independent)
void gteSymInt32Zero(tensor_t *input, tensor_t *output);
// Typical call from ReLU forward (FLOAT32 mode):
// gteFloatValue(input, 0.0f, 0.0f, output);
// →  $\text{output}[i] = (\text{input}[i] \geq 0) ? \text{input}[i] : 0$ 
```

— ■ WORKED EXAMPLE

$\text{ReLU}(x) = \max(0, x)$ implemented via `gteFloatValue`: Input: [-1.2, 0.5, -0.3, 2.1, 0.0, -0.7] Call: `gteFloatValue(input, 0.0f, 0.0f, output)` Output: [0.0, 0.5, 0.0, 2.1, 0.0, 0.0] For SYM_INT32 (mantissa representation): Input mantissas: [-120, 500, -30, 2100, 0, -70] (scale = 0.001) `gteSymInt32Zero`: mantissa < 0 → set to 0 Output mantissas: [0, 500, 0, 2100, 0, 0] Real values: [0, 0.5, 0, 2.1, 0, 0] ✓

Section 6.8 — Relations With Other Files

File	Direction	Why
Arithmetic.h/.c	→ depends on	Tensor.h, DTypes.h, Common.h, Matmul.h
Add/Sub/Mul/Div	→ use Arithmetic.c	All call int32/floatPointWiseArithmetic with a lambda
Matmul.h/.c	→ depends on	Tensor.h, DTypes.h, Arithmetic.h (getDimensionsByIndex)
MinMax.h/.c	→ depends on	DTypes.h for readBytesAsFloat/Int32
Rounding.h/.c	→ depends on	RNG.h (for stochastic rounding), stdint.h
Layer/*.c (Linear, Conv1d, ...)	← uses Arithmetic engine	Forward and backward passes call Add, Mul, Matmul, etc.
TensorConversion.c	← uses MinMax + Rounding	requantSymInt32: findAbsMaxFloat → roundByMode → clamp
Sgd.c	← uses Add	Weight update: addInt32TensorToSymInt32TensorInplace for gradient accumulation

■ RigL GAP IN THIS FILE

RigL requires TWO new arithmetic operations not currently implemented: 1. Masked matmul: `matmulSymInt32TensorsWithBias` needs a mask tensor argument. If `mask[i][k]=0`, skip the multiplication $A[i][k] \times B[j][k]$ (treat as 0). This saves compute proportional to the sparsity ratio. 2. Gradient accumulation for masked weights: during the backward pass, even pruned (`mask=0`) weights need their gradient tracked (for the RigL regrow step). Currently `linearCalcWeightGrads` skips no elements. A new 'accumulate gradient into dead-weight buffer' function is needed.

Chapter 7

Layer 5 — Neural Network Layers

Layer.h, Linear, ReLU, Softmax, Conv1d, Dropout, Flatten and more

Section 7.1 — Layer.h: The Dispatch System

■ WHAT THIS FILE DOES

Layer.h is the registry for all neural network layers. It defines: (1) `layerType_t` — an enum of all supported layer types; (2) `layerConfig_t` — a union that holds a pointer to any layer's config; (3) `layer_t` — the base struct every layer shares; (4) `layerFunctions_t` — a struct of function pointers for forward/backward/calcOutputShape; (5) `layerFunctions[]` — a global dispatch table indexed by `layerType_t`. This is the glue that lets the training loop call any layer's forward pass with the same syntax regardless of which layer type it is.

■ C CONCEPT: The Dispatch Table Pattern

A dispatch table is an array of function pointers indexed by a type enum. Instead of a giant if/else chain: if (layer->type == LINEAR) linearForward(...); else if (layer->type == RELU) reluForward(...); else if ... The dispatch table allows: `layerFunctions[layer->type].forward(layer, input, output)`; One line, works for all layer types, O(1) dispatch. Adding a new layer only requires adding one row to the table and one enum value. No existing code needs to change.

```
// Layer.h – the enum of all layer types
typedef enum layerType {
    LINEAR, RELU, CONV1D, CONV1D_TRANSPOSED, MAXPOOL1D,
    AVGPOOL1D, SOFTMAX, FLATTEN, QUANTIZATION,
    ADAPTIVE_AVGPOOL1D, DROPOUT, LAYERNORM
} layerType_t;
// The union: one slot that can point to any layer config
typedef union layerConfig {
    linearConfig_t *linear;
    reluConfig_t *relu;
    conv1dConfig_t *conv1d;
    softmaxConfig_t *softmax;
    dropoutConfig_t *dropout;
    // ... (11 types total)
} layerConfig_t;
// The base layer struct
typedef struct layer {
    layerType_t type;
    layerConfig_t *config;
} layer_t;
// Function pointer types for forward/backward/shape
typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*);
typedef void (*backwardFn_t)(layer_t*, tensor_t*, tensor_t*, tensor_t*);
typedef void (*calcOutputShapeFn_t)(layer_t*, shape_t*, shape_t*);
// One row in the dispatch table
typedef struct layerFunctions {
    forwardFn_t forward;
    backwardFn_t backward;
    calcOutputShapeFn_t calcOutputShape;
} layerFunctions_t;
// The table – declared extern, defined in Layer.c
extern layerFunctions_t layerFunctions[];
```

```
// Layer.c — how the table is populated (simplified):
layerFunctions_t layerFunctions[] = {
[LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
[RELU] = { reluForward, reluBackward, reluCalcOutputShape },
[CONV1D] = { conv1dForward, conv1dBackward, conv1dCalcOutputShape },
[SOFTMAX] = { softmaxForward, softmaxBackward, softmaxCalcOutputShape },
[DROPOUT] = { dropoutForward, dropoutBackward, dropoutCalcOutputShape },
[FLATTEN] = { flattenForward, flattenBackward, flattenCalcOutputShape },
// ... all 12 types
};
// Training loop usage:
for (int l = 0; l < numLayers; l++) {
layerFunctions[layers[l]->type].forward(
layers[l], inputTensors[l], outputTensors[l]);
}
```

■ C CONCEPT: union layerConfig_t — One Pointer, Many Types

A union allocates memory for the LARGEST member, shared by all members. Only one member is valid at a time. `sizeof(layerConfig_t) = sizeof(pointer) = 4 or 8 bytes`. If `layer->type == LINEAR`: access `layer->config->linear` (`linearConfig_t*`) If `layer->type == RELU`: access `layer->config->relu` (`reluConfig_t*`) NEVER access the wrong union member — it would give a garbage pointer. Always check `layer->type` first. The dispatch table does this correctly by calling the right function for the right type.

Section 7.2 — Linear.h/c: The Fully-Connected Layer

■ WHAT THIS FILE DOES

The Linear layer implements a fully-connected (dense) neural network layer: $\text{output} = \text{input} \times \text{weights}^T + \text{bias}$. It is the most commonly used layer in speaker ID models (MLP architectures). It supports both `FLOAT32` (for easy training on PC) and `SYM_INT32` (for FQT on MCU). The backward pass computes three gradients: dL/dW (weight grads), dL/db (bias grads), and $dL/dinput$ (propagated loss for the layer below).

Field	Type	Meaning
weights	<code>parameter_t*</code>	Weight matrix + gradient. Shape: <code>[out_features, in_features]</code> .
bias	<code>parameter_t*</code>	Bias vector + gradient. Shape: <code>[out_features]</code> .
forwardQ	<code>quantization_t*</code>	Quantization type for the forward pass (<code>FLOAT32</code> or <code>SYM_INT32</code>).
weightGradQ	<code>quantization_t*</code>	Quantization type for weight gradients.
biasGradQ	<code>quantization_t*</code>	Quantization type for bias gradients.
propLossQ	<code>quantization_t*</code>	Quantization type for the propagated loss ($dL/dinput$).
ownsQuantizations	<code>bool</code>	If true, <code>free()</code> will also free <code>forwardQ</code> and all <code>qConfigs</code> . Prevents dangling pointers when the layer is torn down.

7.2.1 linearForwardFloat — The Core Forward Pass

```
void linearForwardFloat(tensor_t *w, tensor_t *b,
tensor_t *input, tensor_t *output) {
transposeTensor(w, 0, 1); // w: [out,in] → [in,out]
matmulFloat32TensorsWithBias(input, w, output, b); // output = input × w^T + b
transposeTensor(w, 0, 1); // restore w: [in,out] → [out,in]
}
```

transpose(w, 0, 1)	Swap axes 0 and 1 of weight tensor (O(1) operation).	Weights are stored as [out_features, in_features] = [N, K]. matmul expects B to be [K, N] for $Y = X \times B$. Transposing gives [K, N] = [in_features, out_features] without copying data.
matmulFloat32TensorsWithBias(..)	$Y = \text{input} \times w + b$ where w is now transposed.	input: [batch, in_features], w: [in_features, out_features] → output: [batch, out_features]. Bias b has shape [out_features] and is broadcast across the batch dimension.
transpose(w, 0, 1) again	Restore w to its original [out_features, in_features] shape.	We transpose back so the weight tensor remains in canonical form for the backward pass and optimizer step, which expect [out, in] layout.

7.2.2 linearBackward — The Gradient Computation

■ C CONCEPT: Backpropagation Through a Linear Layer

Given: forward was $Y = X @ W^T + b$ Received from above: dL/dY (the loss gradient w.r.t. output Y) Compute: $dL/dW = dL/dY^T @ X \rightarrow$ weight gradient (accumulate, not overwrite) $dL/db = \text{sum}(dL/dY, \text{axis}=0) \rightarrow$ bias gradient (sum across batch) $dL/dX = dL/dY @ W \rightarrow$ propagated loss (sent to layer below) These formulas come from the chain rule of calculus applied to matrix operations. They are derived once and implemented; no calculus required to USE the library.

```
// linearCalcWeightGradsFloat32 - accumulate dL/dw:
void linearCalcWeightGradsFloat32(tensor_t *loss, tensor_t *forwardInput,
    tensor_t *weightGrads) {
    // Allocate a temporary buffer on the stack (VLA):
    size_t N = calcNumberOfElementsByTensor(weightGrads);
    float tmpData[N];
    tensor_t tmp; quantization_t tmpQ;
    initFloat32Quantization(&tmpQ);
    setTensorValues(&tmp, (uint8_t*)tmpData, weightGrads->shape, &tmpQ, NULL);
    // dL/dw = loss^T @ forwardInput:
    transposeTensor(loss, 0, 1);
    matmulFloat32Tensors(loss, forwardInput, &tmp);
    transposeTensor(loss, 0, 1);
    // ACCUMULATE (not replace) into weightGrads:
    addFloat32TensorsInplace(weightGrads, &tmp);
}

// linearCalcBiasGradsFloat32 - accumulate dL/db:
// (sum loss across batch dimension)
void linearCalcBiasGradsFloat32(tensor_t *biasGrads, tensor_t *loss) {
    // For each output neuron j: biasGrads[j] += sum over batch of loss[i][j]
    // (implemented as addFloat32TensorsInplace along axis 0)
}
```

■ BEGINNER TIP

WHY ACCUMULATE? During a batch, you run backward for each sample in the batch and ADD each sample's gradient contribution. This is gradient accumulation. The optimizer then divides by batch size (or uses REDUCTION_MEAN in the loss function). That's why `sgdZeroGrad()` is called at the START of each batch: to reset the accumulated gradients to zero before the new batch.

7.2.3 linearCalcOutputShape

```
// The output shape depends on the weight tensor's shape:
// Input: [batch_size, in_features]
// Weights: [out_features, in_features]
// Output: [batch_size, out_features]
void linearCalcOutputShape(layer_t *layer, shape_t *inputShape, shape_t *outputShape) {
    linearConfig_t *cfg = layer->config->linear;
    tensor_t *w = getParamFromParameter(cfg->weights);
    // outputShape->dimensions[0] = inputShape->dimensions[0]; // batch
    // outputShape->dimensions[1] = w->shape->dimensions[0]; // out_features
}
```

■ RigL GAP IN THIS FILE

To add RigL sparsity to the Linear layer: 1. Add a 'mask' tensor field to `linearConfig_t`: a BOOL tensor of shape [out,in]. 2. In `linearForwardSymInt32`: before each $A[i][k] \cdot B[j][k]$ multiplication, check `mask[j][k]`. If `mask=0`, skip the multiply (zero contribution). 3. In `linearCalcWeightGradsSymInt32`: compute gradients for ALL weights (including masked ones) and store in a separate 'grad_all' buffer. This is needed because RigL regrowing uses gradient magnitude. 4. Add a `rigLUpdate()` function called every T steps that: (a) prunes the s% smallest active weights, (b) regrowing the s% inactive weights with largest |gradient|.

Section 7.3 — Relu.h/.c: Rectified Linear Unit

■ WHAT THIS FILE DOES

ReLU is the most common activation function: $f(x) = \max(0, x)$. It is placed after linear layers to add non-linearity to the network. Without non-linearities, a stack of linear layers collapses to a single linear layer and cannot learn complex functions like speaker identification. The backward pass of ReLU: $dL/dx = dL/dy$ if $y > 0$, else 0.

Field	Type	Meaning
forwardQ	quantization_t*	Quantization type for forward pass (FLOAT32 or ASYM).
backwardQ	quantization_t*	Quantization type for backward pass.
ownsQuantizations	bool	Whether free() should also free forwardQ/backwardQ.

```
// Relu.c – forward (FLOAT32 path):
void reluForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
    gteFloatValue(input, 0.0f, 0.0f, output);
    // gteFloatValue: output[i] = (input[i] >= 0) ? input[i] : 0
}
// Relu.c – forward (SYM_INT32 path):
void reluForwardSymInt32(layer_t *layer, tensor_t *input, tensor_t *output) {
    // Zero any element where mantissa < 0 (zero is scale-independent)
    gteSymInt32Zero(input, output);
    // Copy scale from input to output (ReLU doesn't change the scale)
    symInt32QConfig_t *inCfg = input->quantization->qConfig;
    symInt32QConfig_t *outCfg = output->quantization->qConfig;
    outCfg->scale = inCfg->scale;
}
// Relu.c – dispatch based on forwardQ->type:
void reluForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    reluConfig_t *cfg = layer->config->relu;
    switch (cfg->forwardQ->type) {
        case FLOAT32: reluForwardFloat(layer, input, output); break;
        case SYM_INT32: reluForwardSymInt32(layer, input, output); break;
        case ASYM: reluForwardAsym(layer, input, output); break;
        default: PRINT_ERROR("Unknown QType"); exit(1);
    }
}
// ReLU backward (FLOAT32):
void reluBackwardFloat(layer_t *layer, tensor_t *forwardInput,
    tensor_t *loss, tensor_t *propLoss) {
    float *inputArr = (float *)forwardInput->data; // direct cast: FLOAT32 is native
    size_t n = calcNumberOfElementsByTensor(loss);
    for (size_t i = 0; i < n; i++) {
        float inVal = readBytesAsFloat(forwardInput->data + i*4);
        float lossV = readBytesAsFloat(loss->data + i*4);
        // Gradient = 0 where input was <= 0, else pass through
        float propV = (inVal > 0.0f) ? lossV : 0.0f;
        writeFloatToByteArray(propV, propLoss->data + i*4);
    }
}
```

■ C CONCEPT: ReLU Backward — The Heaviside Step Function

During forward: $y = \max(0, x) \rightarrow y > 0$ iff $x > 0$ During backward, the derivative of ReLU is: $dy/dx = 1$ if $x > 0$ $dy/dx = 0$ if $x \leq 0$ By the chain rule: $dL/dx = dL/dy \times dy/dx = dL/dy$ if $x > 0 = 0$ if $x \leq 0$ So the backward pass just GATES the incoming gradient: pass it through where the input was positive, block it where it was zero or negative. This is why we need to save forwardInput — we need to know which inputs were positive.

■ NOTE

reluForwardSymInt32 copies the INPUT scale to the OUTPUT. This is correct because ReLU does not change the range of values — it only zeroes some of them. The maximum absolute value (which determines the scale) cannot increase through ReLU, so the input's scale is a safe output scale too.

Section 7.4 — Softmax.h/.c: The Output Layer

■ WHAT THIS FILE DOES

Softmax is always the last layer in a classifier. It converts raw network outputs (logits) into a probability distribution: all outputs are in [0,1] and sum to 1. For a speaker ID model with N speakers, softmax outputs [p(speaker_1), ..., p(speaker_N)]. The predicted speaker is argmax(output). The backward pass of softmax (combined with cross-entropy loss) simplifies to $dL/dinput = softmax_output - one_hot_target$, which is very efficient.

■ C CONCEPT: Softmax Formula

Given logits $z = [z_1, z_2, \dots, z_N]$: $softmax(z)_i = \exp(z_i) / \sum_j (\exp(z_j))$ Example: $z = [1.0, 2.0, 0.5]$ $\exp(z) = [e^1, e^2, e^{0.5}] = [2.718, 7.389, 1.649]$ $sum = 11.756$ $softmax = [0.231, 0.629, 0.140]$ (they sum to 1.0) Numerical stability trick (subtract max before exp): $z' = z - \max(z) = [-1, 0, -1.5]$ This prevents overflow in $\exp()$ without changing the result (since $\exp(z - \max) / \sum (\exp(z - \max)) == \exp(z) / \sum (\exp(z))$)

```
// softmaxForward – FLOAT32 path (simplified):
void softmaxForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    size_t batchSize = getDimensionsByIndex(input, 0);
    size_t numClasses = getDimensionsByIndex(input, 1);
    for (size_t i = 0; i < batchSize; i++) {
        float *inRow = (float*)input->data + i * numClasses;
        float *outRow = (float*)output->data + i * numClasses;
        // Step 1: find max for numerical stability
        float maxVal = inRow[0];
        for (size_t j = 1; j < numClasses; j++)
            if (inRow[j] > maxVal) maxVal = inRow[j];
        // Step 2: compute exp(z - max) and accumulate sum
        float sum = 0.0f;
        for (size_t j = 0; j < numClasses; j++) {
            outRow[j] = expf(inRow[j] - maxVal);
            sum += outRow[j];
        }
        // Step 3: normalize
        for (size_t j = 0; j < numClasses; j++)
            outRow[j] /= sum;
    }
}
```

Section 7.5 — Conv1d.h/.c: 1D Convolution

■ WHAT THIS FILE DOES

Conv1d performs 1D convolution: sliding a filter (kernel) over a 1D input sequence. For audio/speaker-ID: input is [batch, channels, time], kernel is [out_channels, in_channels, kernel_size]. At each position, the kernel computes a weighted sum of the local input window. Convolutions detect local patterns (like phoneme features) at any position.

Field	Type	Meaning
kernel	kernel_t	Convolution hyperparameters: size, stride, dilation, padding type.
weights	parameter_t*	Filter weights. Shape: [out_channels, in_channels, kernel_size].
bias	parameter_t*	Per-output-channel bias. Shape: [out_channels].
groups	size_t	Grouped convolution (1=standard, in_channels=depthwise).

forwardQ	quantization_t*	Forward quantization mode.
weightGradQ / biasGradQ / propLossQ	quantization_t*	Gradient quantizations.
ownsQuantizations	bool	Memory ownership flag.

■ C CONCEPT: 1D Convolution — Sliding Window

Input signal: [1, 2, 3, 4, 5, 6, 7, 8] (length 8) Kernel: [1, 0, -1] (size 3, detects edges) Stride=1, Valid padding (no padding): Output[0] = $1 \times 1 + 2 \times 0 + 3 \times (-1) = -2$ Output[1] = $2 \times 1 + 3 \times 0 + 4 \times (-1) = -2$ Output[2] = $3 \times 1 + 4 \times 0 + 5 \times (-1) = -2$... Output length = $(\text{input_length} - \text{kernel_size}) / \text{stride} + 1 = (8-3)/1+1 = 6$ With SAME padding (adds zeros): output_length = input_length / stride With EXPLICIT padding: caller specifies exact pad amount on each side

```
// Kernel.h – convolution hyperparameters:
typedef struct kernel {
    size_t size; // filter width (e.g. 3, 5, 7)
    paddingType_t paddingType; // VALID, SAME, or EXPLICIT
    size_t stride; // step between filter positions (default 1)
    size_t dilation; // spacing between kernel elements (default 1)
    size_t padding; // explicit padding amount (if EXPLICIT)
} kernel_t;
typedef enum paddingType {
    VALID, // no padding: output shorter than input
    SAME, // zero-pad so output_len = ceil(input_len / stride)
    EXPLICIT // caller specifies exact padding
} paddingType_t;
```

■ RigL GAP IN THIS FILE

Conv1d is more complex to sparsify than Linear. The weight tensor has 3 dimensions [out_ch, in_ch, kernel_size]. RigL for conv layers typically prunes individual filter weights within each [in_ch, kernel_size] slice. A BOOL mask of the same shape is needed. The inner loop in conv1dForward would check mask[out_c][in_c][k] before each multiply. This is not currently implemented.

Section 7.6 — Dropout.h/c: Regularisation

■ WHAT THIS FILE DOES

Dropout randomly zeros some activations during training. With probability p, each neuron is 'dropped' (set to zero). This forces the network to not rely on any single neuron, preventing overfitting. During evaluation (inference), all neurons are active and outputs are scaled by (1-p).

Field	Type	Meaning
p	float	Drop probability. p=0.5 means 50% of neurons are zeroed each forward pass.
training	bool	If false, dropout is bypassed (identity function). Must be set to true by training loop, false during evaluation.
mask	tensor_t*	Pre-allocated BOOL tensor, same shape as input. Stores which neurons are dropped (0=drop, 1=keep). Saved during forward to be reused in backward.
forwardQ / backwardQ	quantization_t*	Quantization modes.
ownsQuantizations	bool	Memory ownership.

```

// dropoutForward (simplified):
void dropoutForward(layer_t *dropoutLayer, tensor_t *input, tensor_t *output) {
    dropoutConfig_t *cfg = dropoutLayer->config->dropout;
    if (!cfg->training) {
        copyTensor(output, input); return; // identity in eval mode
    }
    size_t n = calcNumberOfElementsByTensor(input);
    float scale = 1.0f / (1.0f - cfg->p); // inverted dropout scaling
    for (size_t i = 0; i < n; i++) {
        bool keep = (rngNextFloat(&rng) >= cfg->p); // keep with prob (1-p)
        tensorBoolSet(cfg->mask, i, keep); // save to mask
        float val = readBytesAsFloat(input->data + i*4);
        writeFloatToByteArray(keep ? val * scale : 0.0f, output->data + i*4);
    }
}
// dropoutBackward: multiply incoming gradient by the same mask
// gradient passes where mask=1, blocked where mask=0

```

■ C CONCEPT: Inverted Dropout Scaling

When training with $p=0.5$, only 50% of neurons fire. If you don't scale up the kept neurons, the total activation is half as large. During evaluation (all neurons active), the total is TWICE as large — mismatch! Inverted dropout multiplies kept neurons by $1/(1-p) = 2$ during training. Now the expected total activation is the same in training and evaluation. No scaling needed during evaluation. This is the modern standard approach.

Section 7.7 — Flatten.h/.c: Shape Transformation

■ WHAT THIS FILE DOES

Flatten converts a multi-dimensional tensor to a 2D tensor: $[batch, d1, d2, \dots] \rightarrow [batch, d1*d2*\dots]$. This is needed between Conv1d and Linear layers: conv output is $[batch, channels, length]$ but linear expects $[batch, features]$. Flatten does NOT copy data — it just changes the shape metadata.

```

// flattenForward: reshape by changing shape metadata, not data
void flattenForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    size_t batchSize = getDimensionsByIndex(input, 0);
    size_t features = calcNumberOfElementsByTensor(input) / batchSize;
    // Reuse the input data buffer (no copy!):
    output->data = input->data;
    // Update shape to [batch, features]:
    output->shape->dimensions[0] = batchSize;
    output->shape->dimensions[1] = features;
    output->shape->numberOfDimensions = 2;
    // Quantization stays the same
    output->quantization = input->quantization;
}
// flattenBackward: reverse – reshape [batch, features] → [batch, channels, length]
// (just metadata changes, same data)
// flattenCalcOutputShape:
void flattenCalcOutputShape(layer_t *layer, shape_t *in, shape_t *out) {
    size_t total = 1;
    for (size_t i = 0; i < in->numberOfDimensions; i++)
        total *= in->dimensions[i];
    out->numberOfDimensions = 2;
    // out->dimensions[0] = batch; out->dimensions[1] = total/batch
}

```

■ NOTE

Flatten is a zero-copy operation: $output->data = input->data$ means both tensors share the SAME byte buffer. Only the shape interpretation changes. This is safe because flatten does not modify any values — it is purely a view.

Section 7.8 — Relations With Other Files

File	Direction	Why
All Layer .h/.c files	→ depend on	Layer.h (layer_t, layerConfig_t), Tensor.h, Common.h
Linear.c	→ uses	Matmul.h (forward), Add.h (grad accumulation), TensorConversion.h
Relu.c	→ uses	Comparison.h (gteFloatValue, gteSymInt32Zero)
Softmax.c	→ uses	MinMax.h (findMax for stability), Log.h (for cross-entropy)
Dropout.c	→ uses	RNG.h (rngNextFloat for sampling), Tensor.h (tensorBoolSet)
Conv1d.c	→ uses	Conv1dKernel.h, SlidingWindow1d.h, Kernel.h, Matmul.h
Flatten.c	→ uses	Tensor.h (shape manipulation only)
Training loop	← uses all layers	Calls layerFunctions[type].forward/backward
Layer.c	→ populates	layerFunctions[] dispatch table with all layer function pointers

Chapter 8

Layer 6 — Training Infrastructure

LossFunction, Optimizer/SGD, and the DataLoader

Section 8.1 — LossFunction.h: The Loss System

■ WHAT THIS FILE DOES

LossFunction.h defines how the network measures its own error. It provides: (1) `lossFuncType_t` — MSE or `CROSS_ENTROPY`; (2) `lossConfig_t` — per-experiment configuration; (3) `lossFunctions_t` — a struct of function pointers (forward, backward, meanScale); (4) `lossFunctions[]` — the global dispatch table indexed by `lossFuncType_t`. The training loop calls `lossFunctions[config.funcType].forward(...)` to compute the loss and `.backward(...)` to get the initial gradient.

Field	Type	Meaning
<code>funcType</code>	<code>lossFuncType_t</code>	MSE (regression) or <code>CROSS_ENTROPY</code> (classification). Speaker ID always uses <code>CROSS_ENTROPY</code> .
<code>backwardReduction</code>	<code>reduction_t</code>	<code>REDUCTION_MEAN</code> (divide by batch size) or <code>REDUCTION_SUM</code> . <code>MEAN</code> is the standard choice.
<code>classWeights</code>	<code>tensor_t*</code>	Optional per-class weights for imbalanced datasets. <code>NULL</code> = equal weights. Shape: <code>[numClasses]</code> .

■ C CONCEPT: Loss Functions — What They Measure

A loss function measures how **WRONG** the network's prediction is. MSE (Mean Squared Error) — for regression: $L = (1/N) \times \sum_i (\text{prediction}_i - \text{target}_i)^2$ Large errors are penalized quadratically. Good for continuous outputs. Cross-Entropy — for classification: $L = -(1/N) \times \sum_i \text{target}_i \times \log(\text{prediction}_i)$ Measures how surprised the model is by the true class. prediction = [0.231, 0.629, 0.140], true class = 1 (index 1): $L = -\log(0.629) = 0.463$ A perfect prediction ($p=1.0$ for true class): $L = -\log(1.0) = 0$ A terrible prediction ($p \rightarrow 0$): $L = -\log(0) \rightarrow \text{infinity}$ Combined with softmax, the backward gradient simplifies to: $dL/d\text{logit}_i = \text{softmax_output}_i - \text{one_hot_target}_i$

```
// LossFunction.h – the dispatch table structure
typedef struct lossFunctions {
    lossFwdFn_t forward; // compute scalar loss
    lossBwdFn_t backward; // compute per-element gradient
    computeMeanScaleFn_t computeMeanScale; // for REDUCTION_MEAN scaling
} lossFunctions_t;
extern lossFunctions_t lossFunctions[]; // defined in LossFunction.c
// LossFunction.c (simplified):
lossFunctions_t lossFunctions[] = {
    [MSE] = { mseForward, mseBackward, computeMeanScaleMSE },
    [CROSS_ENTROPY] = { crossEntropyForward, crossEntropySoftmaxBackward, computeMeanScaleCE },
};
```

8.1.1 crossEntropyForwardFloat — The CE Loss Computation

```
// crossEntropyForwardFloat (simplified):
float crossEntropyForwardFloat(tensor_t *softmaxOut, tensor_t *label,
reduction_t reduction) {
    size_t batch = getDimensionsByIndex(softmaxOut, 0);
    size_t numCl = getDimensionsByIndex(softmaxOut, 1);
    float totalLoss = 0.0f;
    for (size_t b = 0; b < batch; b++) {
        for (size_t c = 0; c < numCl; c++) {
            float pred = readBytesAsFloat(softmaxOut->data + (b*numCl+c)*4);
            float target = readBytesAsFloat(label->data + (b*numCl+c)*4);
            // Add -target * log(pred) to loss
            // log(0) guard: clamp pred to avoid -infinity
            if (pred > 0.0f && target > 0.0f)
                totalLoss -= target * logFloat(pred);
        }
    }
    if (reduction == REDUCTION_MEAN)
        totalLoss /= (float)batch;
    return totalLoss;
}
```

8.1.2 crossEntropySoftmaxBackward — The Combined Gradient

```
// Combined softmax + cross-entropy backward: dL/dlogit = softmax_out - target
// This is the most efficient backward formula for CE+softmax.
void crossEntropySoftmaxBackward(tensor_t *softmaxOut, tensor_t *label,
tensor_t *result) {
    size_t n = calcNumberOfElementsByTensor(softmaxOut);
    for (size_t i = 0; i < n; i++) {
        float pred = readBytesAsFloat(softmaxOut->data + i*4);
        float target = readBytesAsFloat(label->data + i*4);
        writeFloatToByteArray(pred - target, result->data + i*4);
    }
}
```

— ■ WORKED EXAMPLE

Backward gradient for a 3-class example: Softmax output: [0.231, 0.629, 0.140] True label (one-hot): [0, 1, 0] (class 1 is correct) $dL/dlogit = softmax - label$: [0.231-0, 0.629-1, 0.140-0] = [0.231, -0.371, 0.140] Interpretation: Logit 0 (class 0): was 0.231, should be 0 → push it DOWN (gradient +0.231) Logit 1 (class 1): was 0.629, should be 1 → push it UP (gradient -0.371) Logit 2 (class 2): was 0.140, should be 0 → push it DOWN (gradient +0.140) This gradient flows backward through all layers, updating weights toward making class 1 more likely.

Section 8.2 — Optimizer.h and Sgd.h/.c: Weight Updates

■ WHAT THIS FILE DOES

The optimizer uses computed gradients to update weights. Optimizer.h provides the generic optimizer_t wrapper with a dispatch table. Sgd.h/.c implements SGD (Stochastic Gradient Descent) and SGD+Momentum. SGD is the simplest optimizer: $new_weight = old_weight - lr \times gradient$. SGD+Momentum adds a velocity term that smooths updates and helps escape local minima.

■ C CONCEPT: Gradient Descent — The Learning Algorithm

The goal: minimize loss $L(W)$ by adjusting weights W . Gradient descent: move W in the direction opposite to the gradient. $W_new = W_old - learning_rate \times dL/dW$ learning_rate (lr): controls step size. Too large → overshoot. Too small → slow. Typical values: 0.001 to 0.1 Example: $W = 2.0$, $lr = 0.01$, gradient = 5.0 $W_new = 2.0 - 0.01 \times 5.0 = 2.0 - 0.05 = 1.95$ (moved slightly in the direction that reduces loss) SGD+Momentum: velocity = momentum $\times$ velocity + gradient $W = W - lr \times velocity$ Momentum (e.g. 0.9) accumulates past gradients, accelerating in consistent directions.

```
// Sgd.h – the SGD optimizer structs
typedef struct sgd {
float learningRate; // step size (e.g. 0.001)
float momentumFactor; // 0 = no momentum; 0.9 = standard momentum
float weightDecay; // L2 regularization (adds -wd × W to gradient)
} sgd_t;
void sgdInit(sgd_t *sgd, float learningRate, float momentumFactor,
float weightDecay);
void sgdStep(sgd_t *sgd, parameter_t *parameter, quantization_t *qtype);
void sgdStepM(sgd_t *sgd, parameter_t *parameter, quantization_t *qtype,
tensor_t *state); // with momentum
void sgdZeroGrad(parameter_t *parameter);
```

8.2.1 sgdStepFloat — Float32 Weight Update

```
// Sgd.c – sgdStepFloat (simplified):
static void sgdStepFloat(sgd_t *sgd, parameter_t *parameter) {
tensor_t *w = getParamFromParameter(parameter);
tensor_t *g = getGradFromParameter(parameter);
size_t n = calcNumberOfElementsByTensor(w);
float lr = sgd->learningRate;
float wd = sgd->weightDecay;
float *wArr = (float *)w->data; // direct cast: FLOAT32 data IS float array
float *gArr = (float *)g->data;
for (size_t i = 0; i < n; i++) {
float grad = gArr[i];
if (wd != 0.0f) grad += wd * wArr[i]; // L2 regularization
wArr[i] -= lr * grad; // gradient step
}
}
```

float *wArr = (float *)w->data;	Direct pointer cast from uint8_t* to float*.	When the tensor is FLOAT32, data holds IEEE 754 floats. Casting to float* allows direct indexed access wArr[i] without memcpy. This is safe for FLOAT32 but would be wrong for SYM_INT32 or other types.
if (wd != 0.0f) grad += wd * wArr[i];	Add L2 weight decay to the gradient.	Weight decay acts as a penalty for large weights. The effective gradient becomes $dL/dW + wd \times W$, pushing weights toward zero over time.
wArr[i] -= lr * grad;	Gradient descent: $W = W - lr \times \text{gradient}$.	The '-' is key: we subtract because we want to DECREASE the loss, which means moving AGAINST the gradient direction.

8.2.2 sgdStepSymInt32 — FQT Weight Update

```

// The SYM_INT32 SGD step uses a temporary float conversion:
static void sgdStepSymInt32(sgd_t *sgd, parameter_t *parameter) {
    tensor_t *w = getParamFromParameter(parameter);
    tensor_t *g = getGradFromParameter(parameter);
    size_t n = calcNumberOfElementsByTensor(w);
    // 1. Convert weight and gradient to float (VLA on stack):
    float wFloat[n], gFloat[n];
    tensor_t wTmp, gTmp;
    quantization_t wQ, gQ;
    initFloat32Quantization(&wQ); initFloat32Quantization(&gQ);
    setTensorValues(&wTmp, (uint8_t*)wFloat, w->shape, &wQ, NULL);
    setTensorValues(&gTmp, (uint8_t*)gFloat, g->shape, &gQ, NULL);
    convertTensor(w, &wTmp); // SYM_INT32 → float
    convertTensor(g, &gTmp); // SYM_INT32 gradient → float
    // 2. Apply gradient descent in float:
    for (size_t i = 0; i < n; i++)
        wFloat[i] -= sgd->learningRate * gFloat[i];
    // 3. Convert back to SYM_INT32 (requantise):
    convertTensor(&wTmp, w); // float → SYM_INT32 (updates scale)
}

```

■ C CONCEPT: Why Convert to Float for the SGD Step?

In SYM_INT32, weights have a SCALE. When we apply gradient descent, the update magnitude is $lr \times \text{gradient}$ (in real-value space). We could update mantissas directly, but we'd need to know how many mantissa units equal $lr \times \text{gradient}$. This requires knowing the scale. The float conversion approach is cleaner: 1. Convert weight mantissa to real float: $\text{real} = \text{mantissa} \times \text{scale}$ 2. Update in float: $\text{real} -= lr \times \text{gradReal}$ 3. Convert back: $\text{new_mantissa} = \text{round}(\text{real} / \text{newScale})$ Step 3 automatically recomputes the scale using `requantSymInt32Tensor` (two-pass: find new `absMax` → compute `newScale` → re-encode). VLAs (`float wFloat[n]`) are used for stack allocation to avoid heap `malloc`.

8.2.3 sgdZeroGrad — Clearing Gradients

```

void sgdZeroGrad(parameter_t *parameter) {
    tensor_t *g = getGradFromParameter(parameter);
    size_t bytes = calcBytesPerTensor(g);
    memset(g->data, 0, bytes); // set all gradient bytes to 0
    // For SYM_INT32: also reset the scale to 1.0 (a zero tensor has any scale)
    if (g->quantization->type == SYM_INT32) {
        symInt32QConfig_t *cfg = g->quantization->qConfig;
        cfg->scale = 1.0f;
    }
}

```

memset(g->data, 0, bytes)	Set all bytes of the gradient tensor to 0.	memset fills a block of memory with a specific byte value. Setting to 0 clears all mantissas to 0 (integer zero is all-zero bits). This MUST be called before each training batch to prevent gradient accumulation across batches.
cfg->scale = 1.0f	Reset the SYM_INT32 scale to 1.0.	After zeroing, all mantissas are 0 regardless of scale. But if scale is huge from the previous batch, the next gradient accumulation would be in the wrong unit. Reset scale=1.0 so the next add uses mantissa units.

■ ■ IMPORTANT

NEVER forget `sgdZeroGrad` at the start of each batch! If you accumulate gradients across batches, the updates will be proportional to the TOTAL gradient over all past batches, not just the current one. The model may fail to converge or may converge to a wrong minimum. The typical training loop pattern is always: `zero_grad` → `forward` → `backward` → `step`.

Section 8.3 — `DataLoader.h/.c`: The Data Pipeline

■ WHAT THIS FILE DOES

`DataLoader.h` defines the data pipeline for training. It wraps a dataset with: batch assembly (group N samples), optional shuffling (randomize sample order each epoch), optional transform (preprocess each sample), and optional `dropLast` (drop incomplete final batch). The training loop calls `getBatch()` in a loop until the dataset is exhausted.

Field	Type	Meaning
<code>getSample</code>	<code>getSampleFn_t</code>	Callback: given an index, return a <code>sample_t*</code> . The user implements this to load data from flash/UART/SRAM.
<code>getDatasetSize</code>	<code>getDatasetSizeFn_t</code>	Callback: return total number of samples.
<code>batchSize</code>	<code>uint16_t</code>	Number of samples per batch (e.g. 8, 16, 32).
<code>getBatch</code>	<code>getBatchFn_t</code>	Callback: assemble a full batch. Called by training loop.
<code>transform</code>	<code>transformFn_t</code>	Applied once after loading each sample. e.g. normalize audio, pad/truncate to fixed length.
<code>targetTransform</code>	<code>transformFn_t</code>	Applied to the label tensor. e.g. convert integer class ID to one-hot float vector.
<code>shuffle</code>	<code>bool</code>	If true, sample order is randomized each epoch.
<code>shuffleSeed</code>	<code>uint64_t</code>	RNG seed for reproducible shuffling.
<code>indices</code>	<code>size_t*</code>	Pre-allocated array of size <code>datasetSize</code> . Holds the shuffled access order. Caller provides the buffer.
<code>dropLast</code>	<code>bool</code>	If true, the last batch is dropped if it has fewer than <code>batchSize</code> samples.

■ C CONCEPT: Callbacks / Function Pointers as Dependency Injection

The `DataLoader` does not know HOW to load a sample — it just knows THAT it needs to call some function with a sample index. The user provides the implementation via function pointers: `dataLoader.getSample = &myGetSampleFromFlash`; This is the C equivalent of an interface or abstract method. The `DataLoader` code is reusable across any dataset that provides a `getSampleFn_t`. To switch from Flash to UART loading: just change the `getSample` pointer. No other `DataLoader` code changes.

```

// Typical DataLoader setup:
// 1. Provide your data-loading function:
sample_t *myGetSample(size_t id) {
// Load sample 'id' from wherever your data lives:
return &dataBuffer[id];
}
size_t myGetDatasetSize() { return 1000; } // 1000 training samples
// 2. Allocate shuffle index array:
static size_t shuffleIndices[1000];
// 3. Initialize the DataLoader:
dataLoader_t loader;
initDataLoader(&loader,
myGetSample, // getSample callback
myGetDatasetSize, // size callback
defaultGetBatch, // batch assembly (provided by library)
16, // batchSize = 16
NULL, // no transform
NULL, // no targetTransform
true, // shuffle = yes
42, // seed for reproducibility
shuffleIndices, // pre-allocated index array
true // dropLast
);
// 4. Training loop:
for (size_t epoch = 0; epoch < numEpochs; epoch++) {
rngShuffleIndices(shuffleIndices, myGetDatasetSize()); // shuffle
for (size_t b = 0; b * 16 < myGetDatasetSize(); b++) {
batch_t *batch = loader.getBatch(&loader, b);
// ... train on batch ...
}
}
}

```

Section 8.4 — The Complete Training Loop

Now that you understand every component, here is how they compose into a complete training iteration. This is the pattern used by `trainingEpochDefault` (if it exists in the project) or implemented directly by the user.

```

// Complete training loop pseudo-code:
for (int epoch = 0; epoch < NUM_EPOCHS; epoch++) {
// 1. Shuffle the dataset indices
rngShuffleIndices(shuffleIndices, datasetSize);
float epochLoss = 0.0f;
int correct = 0;
for (int b = 0; b * batchSize < datasetSize; b++) {
batch_t *batch = loader.getBatch(&loader, b);
// 2. Zero gradients for this batch
sgdZeroGrad(&weightParameter);
sgdZeroGrad(&biasParameter);
// 3. Forward pass through all layers
tensor_t *x = batch->input;
layerFunctions[LINEAR].forward(&linearLayer, x, &act1);
layerFunctions[RELU].forward(&reluLayer, &act1, &act2);
layerFunctions[LINEAR].forward(&linear2Layer, &act2, &act3);
layerFunctions[SOFTMAX].forward(&softmaxLayer, &act3, &output);
// 4. Compute loss (scalar value for monitoring)
float loss = lossFunctions[CROSS_ENTROPY].forward(
&output, batch->target, REDUCTION_MEAN);
epochLoss += loss;
// 5. Compute initial gradient: dL/dlogit = softmax_out - target
lossFunctions[CROSS_ENTROPY].backward(&output, batch->target, &grad0);
// 6. Backpropagate in REVERSE layer order
layerFunctions[SOFTMAX].backward(&softmaxLayer, &act3, &grad0, &grad1);
layerFunctions[LINEAR].backward(&linear2Layer, &act2, &grad1, &grad2);
layerFunctions[RELU].backward(&reluLayer, &act1, &grad2, &grad3);
layerFunctions[LINEAR].backward(&linearLayer, x, &grad3, NULL);
// 7. Scale gradients for REDUCTION_MEAN
float meanScale = lossFunctions[CROSS_ENTROPY].computeMeanScale(
batchSize, &output);
mulFloat32ElementWithTensor(&weightGrad, meanScale, &weightGrad);
// 8. Update weights
sgdStep(&sgd, &weightParameter, &forwardQ);
sgdStep(&sgd, &biasParameter, &forwardQ);
}
printf("Epoch %d: loss=%.4f\n", epoch, epochLoss / datasetSize);
}

```

Section 8.5 — Relations With Other Files

File	Direction	Why
LossFunction.h/.c	→ depends on	Tensor.h, Log.h (for cross-entropy)
CrossEntropy.h/.c	→ implements	lossFunctions[CROSS_ENTROPY] entries
MSE.h/.c	→ implements	lossFunctions[MSE] entries
Sgd.h/.c	→ depends on	Tensor.h, TensorConversion.h (float conversion), Add.h
Optimizer.h	← wraps Sgd	optimizer_t holds a union with sgd_t* for dispatch
DataLoader.h/.c	→ depends on	Dataset.h, Tensor.h, RNG.h (for shuffling)
Training loop	← uses all	LossFunction + Optimizer + DataLoader + LayerFunctions

■ RigL GAP IN THIS FILE

For RigL, the training loop needs a new outer step: Every T_{rigl} batches (e.g. every 100 batches): 1. For each sparse layer: a. Sort active weights by $|\text{mantissa} \times \text{scale}|$, prune the smallest $s\%$ b. Sort inactive weights by $|\text{gradient}|$, regrow the top $s\%$ c. Update the BOOL mask tensor 2. After the mask update, zero the newly-activated weights (they regrow from zero, guided by gradients) The current LossFunction and Optimizer code needs no changes. The RigL step is an ADDITION to the training loop, not a modification of existing code. The only new requirement: the backward pass must compute gradients for ALL weights (including currently-pruned ones) so the regrow decision can be made.

Chapter 9

Layer 7 — Support Modules

RNG, Serialization, Bernoulli, and NPY Loader

Section 9.1 — RNG.h/.c: The Random Number Generator

■ WHAT THIS FILE DOES

RNG.h provides a fast, deterministic pseudo-random number generator (PRNG) based on the xorshift32 algorithm. It is used for: (1) dataset shuffling (rngShuffleIndices), (2) Dropout mask generation (via Bernoulli.c), (3) stochastic rounding in FQT (via Rounding.c). The generator is seeded by rngSetSeed and maintains a single global state. Setting the same seed gives the same sequence every time (reproducibility).

```
// RNG.h – the public interface
typedef struct { uint32_t state; } rng32_t;
// Seed and query
void rngSetSeed(uint32_t seed);
uint32_t rngGetSeed(void);
// Generate a uniform float in [0, 1)
float rngNextFloat(void);
// Fisher-Yates shuffle of an index array
void rngShuffleIndices(size_t *indices, size_t n);
```

9.1.1 xorshift32 — How Random Numbers Are Generated

```
// RNG.c – the internal state and xorshift32 step:
static rng32_t rng = { .state = 1 }; // module-level, persistent
static uint32_t rngNext(rng32_t *rng) {
    uint32_t x = rng->state;
    x ^= x << 13; // step 1: XOR with left-shifted self
    x ^= x >> 17; // step 2: XOR with right-shifted result
    x ^= x << 5; // step 3: XOR with left-shifted result
    rng->state = x;
    return x;
}
```

■ C CONCEPT: XOR-Shift PRNG — Why It Works

A PRNG (Pseudo-Random Number Generator) produces a deterministic sequence that LOOKS random but is fully determined by the initial seed. XOR (exclusive-or, ^): bit-by-bit, output 1 if exactly one input is 1. $0 \wedge 0 = 0$, $0 \wedge 1 = 1$, $1 \wedge 0 = 1$, $1 \wedge 1 = 0$. $x \wedge x \ll 13$ means: $x = x \text{ XOR } (x \text{ shifted left by 13 bits})$. This mixes bits: each bit becomes a function of multiple original bits. Three shifts (13, 17, 5) are carefully chosen constants that make the generator cycle through all $2^{32} - 1$ non-zero values before repeating (maximal period). The generator is very fast: 3 XOR + 3 shift = 6 instructions, 1 cycle on ARM. This is significantly faster than rand() from the C standard library, which is important on a microcontroller where every cycle counts.

— ■ WORKED EXAMPLE

Trace with seed=1 (state = 0x00000001 = binary 0000...0001): $x = 0x00000001$ $x \wedge x \ll 13$: $x = 0x00000001 \text{ XOR } 0x00002000 = 0x00002001$ $x \wedge x \gg 17$: $x = 0x00002001 \text{ XOR } 0x00000000 = 0x00002001$ (17-bit shift of small number = 0) $x \wedge x \ll 5$: $x = 0x00002001 \text{ XOR } 0x00040020 = 0x00042021$ state = 0x00042021 = 270369 decimal. Next call: state = 270369, another round of 3 XOR-shifts → new number. Every call produces a different 32-bit integer.

9.1.2 rngBounded — Fair Bounded Range

```
// Get a random uint32_t in [0, bound) with NO modulo bias:
static uint32_t rngBounded(uint32_t bound) {
// Compute largest multiple of bound that fits in uint32_t:
uint32_t limit = UINT32_MAX - (UINT32_MAX % bound);
uint32_t x;
// Rejection sampling: discard values above 'limit'
do {
x = rngNext(&rng);
} while (x >= limit);
return x % bound;
}
```

■ C CONCEPT: Rejection Sampling — Avoiding Modulo Bias

Why not just use 'rngNext() % bound'? Example: bound=3, range=[0,1,2]. $UINT32_MAX+1 = 4,294,967,296$. $4,294,967,296 / 3 = 1,431,655,765$ remainder 1. So values 0 and 1 appear 1 more time than value 2 — NOT uniform! Rejection sampling: discard any generated value above 'limit' (the largest multiple of bound that fits). The remaining values are uniformly distributed when divided by bound. For bound=3: limit = $4,294,967,296 - (4,294,967,296 \% 3) = 4,294,967,295$. We reject nothing — by luck limit = $UINT32_MAX+1$ for bound=3. For bound=2: limit = $UINT32_MAX+1 - 0 = 4,294,967,296$. Even here, values 0..MAX/2-1 map to 0, others to 1 — perfectly balanced.

9.1.3 rngShuffleIndices — Fisher-Yates Shuffle

```
// rngShuffleIndices: Fisher-Yates shuffle of indices [0..n-1]
void rngShuffleIndices(size_t *indices, size_t n) {
// Initialise to identity permutation if needed:
// (caller typically calls setOrderOfDimsForNewTensor to pre-fill)
for (size_t i = n - 1; i >= 1; i--) {
// Pick a random j in [0, i] (inclusive):
size_t j = (size_t)rngBounded((uint32_t)(i + 1));
// Swap indices[i] and indices[j]:
size_t tmp = indices[i];
indices[i] = indices[j];
indices[j] = tmp;
}
}
```

■ C CONCEPT: Fisher-Yates Shuffle — Why This Algorithm?

The Fisher-Yates shuffle produces an UNBIASED permutation: all $N!$ orderings are equally likely. Algorithm (working from end to beginning): for i from $N-1$ down to 1: pick random j in $[0, i]$ swap elements i and j Example: [A, B, C, D] ($N=4$): $i=3$: $j=\text{random}[0..3]=1 \rightarrow$ swap [3] and [1] $\rightarrow$ [A, D, C, B] $i=2$: $j=\text{random}[0..2]=2 \rightarrow$ swap [2] and [2] $\rightarrow$ [A, D, C, B] (no-op) $i=1$: $j=\text{random}[0..1]=0 \rightarrow$ swap [1] and [0] $\rightarrow$ [D, A, C, B] Result: [D, A, C, B] — one of 24 equally likely orderings. Why not a naive shuffle (pick random pair and swap N times)? Naive is biased: some permutations are more likely than others.

9.1.4 rngNextFloat — Uniform Float in [0, 1)

```
float rngNextFloat(void) {
// Take top 24 bits of the 32-bit random number:
uint32_t raw = rngNext(&rng) >> 8; // 24-bit value in [0, 2^24 - 1]
return (float)raw / (float)(1 << 24); // divide by 2^24  $\rightarrow$  [0, 1)
}
```

■ C CONCEPT: 24-Bit Float Resolution

A float32 mantissa has 23 bits of precision (24 bits including the implicit 1). Using more bits would be wasted — they can't be represented exactly in float32. `raw >> 8`: discard the bottom 8 bits, keep the top 24. Divide by 2^{24} = 16,777,216 gives a value in [0, 1). Example: `raw = 0x3A7B12C4 = 979763908 >> 8 → 0x003A7B12 = 3833618 / 16777216 = 0.22848...` (a float in [0, 1))

9.1.5 rngSetSeed — Reproducibility

```
void rngSetSeed(uint32_t seed) {  
    // xorshift32 cannot have state=0 (0 XOR anything = 0 forever)  
    rng.state = (seed == 0) ? UINT32_MAX : seed;  
}
```

■ NOTE

Always set the seed before any training run for reproducibility. With the same seed, the same shuffle order, the same dropout masks, and the same stochastic rounding decisions are made. Different seeds → different results even with identical training data and code. Publishing the seed used in your experiments lets others reproduce your results exactly.

Section 9.2 — Serialize.h/.c and Deserialize.h/.c

■ WHAT THIS FILE DOES

Serialization saves a trained model (tensor weights) to a byte stream. Deserialization reads it back. On the STM32, you train the model, then serialize weights to UART (PC receives them). Later, deserialize from flash/UART to restore the model for inference on the MCU. The format is raw bytes — each tensor is written as its metadata followed by its data.

```
// Serialize.h – save model to a FILE stream:  
void serializeTensor(tensor_t *tensor, FILE *f);  
void serializeParameter(parameter_t *parameter, FILE *f);  
void serializeModel(layer_t **model, size_t sizeModel, FILE *f);  
// Deserialize.h – restore model from a FILE stream:  
void deserializeTensor(tensor_t *tensor, FILE *f);  
void deserializeParameter(parameter_t *parameter, FILE *f);  
void deserializeModel(layer_t **model, size_t sizeModel, FILE *f);
```

■ C CONCEPT: Serialization — Writing Structs to a Byte Stream

`serializeTensor` writes in order: 1. `tensor->quantization->type` (4 bytes: `qtype_t`) 2. `qConfig` fields (scale, `qMaxBits`, etc.) 3. shape: `numberOfDimensions`, `dimensions[]`, `orderOfDimensions[]` 4. data: the raw weight bytes
`deserializeTensor` reads in the same order and fills the struct. This only works if both writer and reader use the SAME struct layout — true on one architecture (ARM Cortex-M7 → same ARM PC) but potentially broken across platforms with different endianness or struct packing. The library uses `fwrite/fread` for simplicity. On the MCU, `FILE*` maps to UART or flash.

— ■ WORKED EXAMPLE

Serializing a `SYM_INT32` tensor with shape [3,4] and 12 elements: Bytes written: [0-3]: `qtype = SYM_INT32 = 2` (`uint32_t`, 4 bytes) [4-7]: `scale = 0.001f` (float, 4 bytes = 0x6F126E3A in IEEE754) [8]: `qMaxBits = 16` (`uint8_t`, 1 byte) [9]: `roundingMode = 0` (`uint8_t`, 1 byte) [10-13]: `numberOfDimensions = 2` (`size_t`, 4 bytes) [14-17]: `dimensions[0] = 3` (4 bytes) [18-21]: `dimensions[1] = 4` (4 bytes) [22-25]: `orderOfDimensions[0] = 0` (4 bytes) [26-29]: `orderOfDimensions[1] = 1` (4 bytes) [30-77]: $12 \times 4 = 48$ bytes of `int32_t` mantissas Total: 78 bytes for this tensor's weights.

Section 9.3 — Bernoulli.h/.c: Sampling for Dropout

■ WHAT THIS FILE DOES

Bernoulli.c provides Bernoulli sampling: for a given probability p , generate a BOOL (0 or 1) where $P(1) = p$. Used by Dropout to generate the random drop mask: each mask element is 0 (drop, prob p) or 1 (keep, prob $1-p$). Relies on RNG.c's `rngNextFloat`.

```
// Bernoulli.h:
void bernoulliSampleTensor(tensor_t *mask, float p);
// Fills all elements of 'mask' (BOOL tensor):
// mask[i] = 1 with probability p
// mask[i] = 0 with probability 1-p
// Implementation (Bernoulli.c):
void bernoulliSampleTensor(tensor_t *mask, float p) {
    size_t n = calcNumberOfElementsByTensor(mask);
    for (size_t i = 0; i < n; i++) {
        bool val = (rngNextFloat() < p); // true with prob p
        tensorBoolSet(mask, i, val);
    }
}
```

— ■ WORKED EXAMPLE

Generating a dropout mask with $p=0.5$ for 8 neurons: `rngNextFloat()` calls: [0.73, 0.12, 0.61, 0.48, 0.92, 0.33, 0.55, 0.09] Compare to $p=0.5$: [F, T, F, T, F, T, F, T] ($T = < 0.5 = \text{keep}$) Wait — Dropout DROPS with prob $p=0.5$, KEEPS with prob 0.5 . So Dropout calls `bernoulliSampleTensor(mask, 1-p=0.5)` for the KEEP mask: mask = [0, 1, 0, 1, 0, 1, 0, 1] Activations multiplied by mask: [0, a1, 0, a3, 0, a5, 0, a7] The kept activations are also scaled by $1/(1-p) = 2$ (inverted dropout).

Section 9.4 — NPYLoader.h/.c: Loading NumPy Data Files

■ WHAT THIS FILE DOES

NPYLoader reads NumPy .npy files. NumPy is the standard Python array library. Training data prepared on a PC (audio features, spectrograms) is saved as .npy files and loaded onto the MCU via UART or stored in flash. NPYLoader parses the .npy header (array shape and dtype) and fills a `tensor_t` with the array data.

■ C CONCEPT: The .npy File Format

A .npy file has: Bytes 0-5: magic string 0x93NUMPY Byte 6: major version (1 or 2) Byte 7: minor version (0) Bytes 8-9: header length (uint16_t, little-endian) Bytes 10..(10+header_len-1): Python dict as ASCII text, e.g.: {'descr': '<f4', 'fortran_order': False, 'shape': (1000, 128), } After header: raw array data (C contiguous, little-endian) '<f4' means little-endian float32. '<i4' means little-endian int32. NPYLoader parses this header, extracts the shape and dtype, then reads the raw bytes directly into the tensor's data buffer.

Section 9.5 — Relations With Other Files

File	Direction	Why
RNG.h/.c	→ standalone	No project dependencies — uses only <code>stdint</code> , <code>stddef</code>
Bernoulli.h/.c	→ uses RNG + Tensor	<code>rngNextFloat</code> for sampling, <code>tensorBoolSet</code> to write mask
Dropout.c	← uses Bernoulli	Calls <code>bernoulliSampleTensor</code> to generate the drop mask
Rounding.c	← uses RNG	<code>rngNextFloat</code> for stochastic rounding noise
DataLoader.c	← uses RNG	<code>rngShuffleIndices</code> for epoch-wise shuffle

Serialize.h/.c	← uses Tensor + Layer	Iterates model layers and serializes each tensor
Deserialize.h/.c	← uses Tensor + Layer	Reads bytes and fills tensor_t fields
NPYLoader.h/.c	← uses Tensor	Fills tensor_t.data and tensor_t.shape from .npz bytes
Training loop	← uses all support modules	DataLoader + RNG + Serialize complete the training pipeline

■ RigL GAP IN THIS FILE

RNG.c already provides everything needed for RigL's stochastic elements. RigL itself is deterministic (prune smallest, grow largest gradient) — no extra randomness needed. The Bernoulli sampler is NOT needed for basic RigL (RigL doesn't randomly prune). However, a variant called 'Random pruning + gradient regrow' (simpler than RigL) would use Bernoulli to select which weights to prune randomly.

Chapter 10

What Is Missing for RigL?

A Complete Gap Analysis for Implementing Evci et al. 2020

Section 10.1 — What Is RigL?

■ C CONCEPT: RigL — Rigging the Lottery (Evci et al., NeurIPS 2020)

RigL (Rigging the Lottery) is a sparse training algorithm. Instead of training a FULL dense network and then pruning, RigL trains a SPARSE network from scratch — most weights are always zero. The key insight: during training, periodically change WHICH weights are zero: 1. PRUNE: identify the $s\%$ of active weights with the smallest absolute value. Set them to zero (deactivate them). They are no longer trained. 2. GROW: identify the $s\%$ of currently-ZERO weights with the largest gradient magnitude. Re-activate them (set mask=1, initialize weight=0). This is done every T steps (e.g. every 100 batches). Over time, the sparse structure evolves to concentrate weights where they matter most. Result: a model with, say, 80% of weights permanently zero, but with almost the same accuracy as the full dense model. On MCU: 80% fewer multiplications in the forward pass = 5× speedup. The sparsity ratio s stays fixed throughout training. Only the PATTERN changes.

Section 10.2 — What Already Exists in the Library

Significant infrastructure for RigL already exists in the ODT codebase. The following components are in place and need no changes:

Component	Status	Why It Helps RigL
tensor_t::sparsity	EXISTS (stub)	The hook for a sparsity mask is already in tensor_t. Only the sparsityConfig fields need to be filled.
BOOL quantization type	EXISTS (full)	Sparsity masks need 1 bit per weight. BOOL tensors (DTypes.c, Tensor.c) handle this.
tensorBoolGet/Set	EXISTS (full)	Reading and writing individual mask bits works.
writeBitsToByteArray / readBitsFromByteArray	EXISTS (full)	Low-level mask bit access.
rngShuffleIndices / rngNextFloat	EXISTS (full)	RigL uses no randomness, but stochastic variants do. Dropout already works.
findAbsMaxFloat / findMaxInt32	EXISTS (full)	Used in Pass A of requantisation; RigL prune step uses the same 'sort by magnitude' logic.
SYM_INT32 weight tensors	EXISTS (full)	FQT weights are already quantized — RigL needs to work with SYM_INT32 mantissas.
parameter_t (weight + grad together)	EXISTS (full)	RigL needs both weight and gradient; parameter_t already bundles them.
Training loop structure	EXISTS (partial)	The per-batch loop exists; a RigL update call can be inserted every T steps.

Section 10.3 — Gap #1: The Sparsity Mask Infrastructure

The sparsity_t struct in tensor_t is currently an empty stub. It needs to be filled with actual content to hold the mask.

```

// CURRENT state of Tensor.h (stub):
typedef struct sparsityConfig {
// EMPTY – nothing here
} sparsityConfig;
typedef struct sparsity {
sparsityType_t type;
sparsityConfig *config;
} sparsity_t;
// WHAT NEEDS TO BE ADDED:
typedef struct sparsityConfig {
tensor_t *mask; // BOOL tensor, same shape as weight tensor
// mask[i] = 1 → weight i is ACTIVE
// mask[i] = 0 → weight i is PRUNED (= 0)
float sparsityRatio; // e.g. 0.8 = 80% of weights pruned
size_t updateStep; // how many batches between RigL updates (T)
size_t batchCount; // internal counter: when >= updateStep, do update
} sparsityConfig_t;
// Allocation: caller provides mask data buffer:
// uint8_t maskData[(N+7)/8]; // ceil(N/8) bytes for N weight bits
// tensor_t maskTensor;
// setTensorValues(&maskTensor, maskData, weightShape, &boolQ, NULL);
// sparsityConfig.mask = &maskTensor;

```

■ RigL GAP IN THIS FILE

WORK TO DO: Extend sparsityConfig with mask, sparsityRatio, and updateStep. Add initSparsityConfig() and initSparsity() functions. Add a BOOL quantization_t to linearConfig_t for the mask. MEMORY COST: 1 bit per weight. A 128×64 linear layer has 8192 weights → 1024 bytes of mask.

Section 10.4 — Gap #2: Masked Forward Pass

The current linearForwardSymInt32 multiplies EVERY weight with EVERY input. For RigL, pruned weights (mask=0) must be skipped. This is both a correctness requirement (pruned weights should not contribute) and the MAIN performance benefit of sparsity.

```

// CURRENT linearForwardSymInt32 (simplified):
void linearForwardSymInt32(tensor_t *w, tensor_t *b,
tensor_t *input, tensor_t *output) {
transposeTensor(w, 0, 1);
matmulSymInt32TensorsWithBias(input, w, output, b);
transposeTensor(w, 0, 1);
}
// WHAT NEEDS TO CHANGE – matmulSymInt32TensorsWithBias inner loop:
// Current:
for (size_t k = 0; k < K; k++) {
int32_t bVal = readBytesAsInt32(B->data + (j*K+k)*4);
acc += (int64_t)aVal * (int64_t)bVal;
}
// With sparse mask:
for (size_t k = 0; k < K; k++) {
// Check if weight [j][k] is active:
bool active = tensorBoolGet(sparsityMask, j*K + k); // ← NEW
if (!active) continue; // ← NEW: skip pruned
int32_t bVal = readBytesAsInt32(B->data + (j*K+k)*4);
acc += (int64_t)aVal * (int64_t)bVal;
}

```

■ RigL GAP IN THIS FILE

WORK TO DO: Modify `matmulSymInt32TensorsWithBias` to accept an optional sparsity mask tensor. If `mask != NULL`, check `tensorBoolGet` before each multiply. Also modify `matmulFloat32TensorsWithBias` for the float training path. The API change: add `'tensor_t *mask'` parameter (`NULL` = dense, use all weights).

Section 10.5 — Gap #3: Full Gradient Computation for Inactive Weights

The RigL GROW step selects inactive weights by their gradient magnitude. To do this, we need the gradient of the LOSS with respect to inactive (pruned) weights. Currently, the backward pass skips pruned weights (they contribute zero to the forward, so their gradient is zero). But for RigL, we need to compute what their gradient WOULD BE if they were active — a counterfactual gradient.

■ C CONCEPT: Gradient of a Pruned Weight

For a linear layer: $Y = X @ W$ (ignoring bias) Gradient of loss w.r.t. W : $dL/dW = dL/dY^T @ X$ This gradient exists for ALL weight positions, even if $W[i][j] = 0$. The gradient tells us: 'if we activated this weight, how much would the loss change per unit of weight?' Currently, `linearCalcWeightGradsFloat32` computes dL/dW for ALL positions regardless of the mask (because the mask is not implemented yet). So the gradient computation is actually ALREADY CORRECT for RigL once the mask is added! The issue is: we must NOT update the pruned weights in `sgdStep`. We must also track the gradient separately so the RigL grow-step can read it.

```
// CURRENT linearCalcWeightGradsFloat32 - computes gradient for ALL weights:
void linearCalcWeightGradsFloat32(tensor_t *loss, tensor_t *forwardInput,
    tensor_t *weightGrads) {
    // ... matmul: dL/dW = loss^T @ forwardInput ...
    addFloat32TensorsInplace(weightGrads, &intermediateGrad); // accumulate
}
// FOR RIGL: this is fine as-is. The gradient is computed for all weights.
// The RigL grow step then reads weightGrads to find the best inactive weight.
// BUT: sgdStep must NOT update pruned weights:
// CURRENT sgdStep:
wArr[i] -= lr * gArr[i]; // updates ALL weights
// MODIFIED sgdStep for RigL:
if (mask == NULL || tensorBoolGet(mask, i)) { // only if active
    wArr[i] -= lr * gArr[i];
}
// Pruned weights stay at 0; their gradient is computed but not applied.
```

■ RigL GAP IN THIS FILE

WORK TO DO: Modify `sgdStep` and `sgdStepM` to accept an optional mask tensor. Skip the weight update for positions where `mask=0`. The gradient accumulation in `linearCalcWeightGrads` already computes gradients for all positions — no change needed there for basic RigL.

Section 10.6 — Gap #4: The RigL Update Function

The core of RigL is a periodic update function that modifies the sparsity mask. This function does not exist anywhere in the codebase. It is the main thing to implement.

```

// PSEUDOCODE for riglUpdate – to be implemented in RigL.h/.c:
void riglUpdate(parameter_t *parameter, tensor_t *mask, float sparsityRatio) {
    tensor_t *w = getParamFromParameter(parameter);
    tensor_t *g = getGradFromParameter(parameter);
    size_t N = calcNumberOfElementsByTensor(w);
    size_t numPruned = (size_t)(sparsityRatio * N); // e.g. 80% pruned
    // === PRUNE STEP ===
    // Find active weights sorted by |w| ascending:
    // (use a temporary index array + sort by |w[i]|)
    size_t activeIndices[N];
    size_t numActive = 0;
    for (size_t i = 0; i < N; i++)
        if (tensorBoolGet(mask, i)) activeIndices[numActive++] = i;
    // Sort activeIndices by |w[activeIndices[i]]| ascending (insertion sort or qsort):
    sortByAbsWeight(activeIndices, numActive, w);
    // Prune the smallest s% of active weights:
    size_t numToPrune = (size_t)(sparsityRatio * numActive);
    for (size_t i = 0; i < numToPrune; i++) {
        size_t idx = activeIndices[i];
        tensorBoolSet(mask, idx, false); // deactivate
        writeInt32ToByteArray(0, w->data + idx*4); // set weight to 0
    }
    // === GROW STEP ===
    // Find inactive weights sorted by |g| descending:
    size_t inactiveIndices[N];
    size_t numInactive = 0;
    for (size_t i = 0; i < N; i++)
        if (!tensorBoolGet(mask, i)) inactiveIndices[numInactive++] = i;
    sortByAbsGradDescending(inactiveIndices, numInactive, g);
    // Regrow the top numToPrune inactive weights:
    for (size_t i = 0; i < numToPrune && i < numInactive; i++) {
        size_t idx = inactiveIndices[i];
        tensorBoolSet(mask, idx, true); // activate
        // Weight starts at 0 (it was pruned); will grow via future gradients
    }
    // Zero gradients after mask change (stale gradients don't apply to new topology)
    sgdZeroGrad(parameter);
}

```

■ RigL GAP IN THIS FILE

WORK TO DO: Create RigL.h and RigL.c with riglUpdate(). Sort logic requires either: (a) a simple insertion sort ($O(N^2)$, fine for $N < 10000$), or (b) qsort() with a comparison callback. The sort needs temporary index arrays of size N. For a 128×64 layer: $N=8192$. $8192 \times \text{sizeof}(\text{size_t}) = 32\text{KB}$ on the stack — too large! Must use static/global arrays or heap malloc for the sort buffers. Alternative: partial selection (find the k-th smallest, no full sort) saves memory.

Section 10.7 — Gap #5: Initialization and Training Loop Integration

Even with the above components, RigL needs to be initialized and called at the right point in the training loop. Here is the complete integration plan:

```
// === INITIALIZATION (once, before training) ===
// 1. Create the BOOL mask tensor for each sparse layer:
uint8_t linearMaskData[(IN*OUT + 7)/8]; // ceil(N/8) bytes
tensor_t linearMask;
quantization_t boolQ;
initBoolQuantization(&boolQ, &boolCfg);
setTensorValues(&linearMask, linearMaskData, linearWeightShape, &boolQ, NULL);
// 2. Initialize mask: set 20% active, 80% pruned
// (randomly select 20% to start active):
riglInitMask(&linearMask, 0.8f); // 0.8 = sparsityRatio = 80% zeros
// 3. Initialize sparsityConfig for this layer:
sparsityConfig_t spCfg = {
    .mask = &linearMask,
    .sparsityRatio = 0.8f,
    .updateStep = 100, // update topology every 100 batches
    .batchCount = 0,
};
// Attach to linear layer's weight tensor:
linearLayer.config->linear->weights->param->sparsity = &sparsity;
sparsity.config = &spCfg;
// === TRAINING LOOP MODIFICATION ===
for (int b = 0; ...; b++) {
    // ... normal forward / backward / step ...
    sgdZeroGrad(&weightParam);
    // forward pass (uses mask in matmul)
    // backward pass (computes gradients for all weights including inactive)
    // optimizer step (updates only active weights)
    // RigL update step (every T batches):
    if (b % T_RIGL == 0 && b > 0) {
        riglUpdate(&weightParam, &linearMask, 0.8f);
    }
}
```

Section 10.8 — Gap #6: Memory Budget Analysis

On the STM32 Nucleo-F746ZG (320 KB RAM, 1 MB flash), RigL adds memory overhead. Here is the breakdown for a typical speaker-ID model:

Component	Size (example)	Notes
Dense linear weights [128,64]	$8192 \times 4 = 32$ KB	SYM_INT32 baseline
RigL mask for [128,64]	$8192 / 8 = 1$ KB	BOOL tensor, packed bits
Gradient buffer [128,64]	$8192 \times 4 = 32$ KB	Must hold all gradients even for pruned
Sort buffer (riglUpdate)	$8192 \times 4 = 32$ KB	Temporary, can reuse gradient buffer
Extra vs dense training	1 KB mask + sort temporary	Sort buffer reuses existing memory
Saved computation (80% sparse)	~5× fewer mults in forward pass	Direct MCU speedup
Flash savings (sparse model)	Store only 20% active weights = 6.4 KB	Sparse serialisation needed

■ NOTE

The sort buffer (for riglUpdate) can reuse the activation buffer from the forward pass since riglUpdate runs BETWEEN batches (no active forward/backward computation at that moment). This avoids needing a separate 32 KB allocation.

Section 10.9 — Complete RigL Implementation Checklist

Summary of all changes needed, in implementation order:

Step	File to Modify/Create	What To Do
1	Tensor.h	Add fields to sparsityConfig: mask (tensor_t*), sparsityRatio (float), updateStep (size_t), batchCount (size_t)
2	Tensor.c	Add initSparsityConfig(), initSparsity() functions
3	Matmul.c	Add 'tensor_t *mask' parameter to matmulSymInt32TensorsWithBias and matmulFloat32TensorsWithBias. Skip k when tensorBoolGet(mask,j*K+k)=0.
4	Linear.h	Add 'tensor_t *mask' field to linearConfig_t
5	Linear.c	Pass mask to matmul in linearForwardFloat and linearForwardSymInt32
6	Sgd.c	Add mask check in sgdStepFloat and sgdStepSymInt32: skip update where mask=0
7	RigL.h	CREATE: declare riglUpdate(), riglInitMask(), rigLConfig_t
8	RigL.c	CREATE: implement riglUpdate() with prune+grow steps; sorting by w and g
9	Training loop	Call riglUpdate() every T batches for each sparse layer
10	Serialize.c	Add sparse serialisation: only write active weights and the mask

Chapter 11

How Everything Works Together on the MCU

STM32 Nucleo-F746ZG — Memory, Execution, and the Complete Data Flow

Section 11.1 — The Target Hardware

■ WHAT THIS FILE DOES

The STM32 Nucleo-F746ZG is an ARM Cortex-M7 microcontroller development board. It runs at 216 MHz, has 320 KB of SRAM and 1 MB of flash. There is no operating system (bare-metal), no virtual memory, no malloc (or very limited). All memory must be statically declared. The ODT library is designed specifically for these constraints.

Resource	Amount	What It Means for ODT
SRAM	320 KB	All tensors, activations, gradients, stack — everything live in RAM. No swap space.
Flash	1 MB	Code + read-only data (e.g. dataset in flash). Trained weights saved here.
CPU	ARM Cortex-M7 @ 216 MHz	Integer ops: ~216M/s. FP (with FPU): ~216M single-precision ops/s.
FPU	Double-precision optional	Single-precision floats run in 1 cycle on the Cortex-M7 FPU.
UART	Up to 4.5 Mbps	Used for: loading training data, sending trained weights to PC.
No OS	Bare-metal	No threads, no malloc, no printf (unless routed to UART). All functions are called directly.

Section 11.2 — Memory Layout on the STM32

On a bare-metal embedded system, you manage memory manually. Everything is a statically declared array or struct. Here is how a typical ODT model's memory is laid out in SRAM:


```

// MCU startup sequence (conceptual):
// HARDWARE INIT (generated by CubeMX or written manually):
SystemClock_Config(); // set CPU to 216 MHz
MX_USART3_UART_Init(); // configure UART for data transfer
// ... other peripheral inits ...
// ODT LIBRARY INIT:
// 1. Seed the random number generator (use a fixed seed for reproducibility):
rngSetSeed(42);
// 2. Initialize quantization configs for all tensors:
initSymInt32QConfig(HALF_AWAY, &w1QCfg); // scale=1.0, qMaxBits=16
initSymInt32QConfig(SR_HALF_AWAY, &g1QCfg); // stochastic rounding for grads
initFloat32Quantization(&outQ); // softmax output = float
// 3. Initialize quantization_t wrappers:
initSymInt32Quantization(&w1Q, &w1QCfg);
initFloat32Quantization(&outQObj);
// 4. Initialize shapes:
size_t w1Dims[2] = {128, 64};
size_t w1Order[2]; setOrderOfDimsForNewTensor(2, w1Order);
setShape(&w1Shape, w1Dims, 2, w1Order);
// 5. Link data buffers into tensors:
setTensorValues(&w1Tensor, w1Data, &w1Shape, &w1Q, NULL);
setTensorValues(&g1Tensor, g1Data, &w1Shape, &g1Q, NULL);
setParameterValues(&w1Param, &w1Tensor, &g1Tensor);
// 6. Initialize weights (e.g. random from PC, or He init):
// (NPYLoader loads pre-computed initial weights via UART)
loadWeightsFromUART(&w1Tensor);
// 7. Initialize layer configs:
linearInitConfig(&linear1Cfg, &w1Param, &b1Param, &symInt32Q, &gradQ, &gradQ, &lossQ);
initLayer(&linear1Layer, LINEAR, &linear1LayerConfig);
// 8. Initialize optimizer:
sgdInit(&sgd, 0.001f, 0.9f, 0.0001f); // lr=0.001, momentum=0.9, wd=0.0001
// 9. Initialize DataLoader:
initDataLoader(&loader, myGetSample, myGetDatasetSize, defaultGetBatch, ...);

```

Section 11.4 — The Complete Forward Pass (Step by Step)

During each training iteration, the forward pass computes the network's prediction given the current weights. Here is how data flows through every layer and every function call for a 2-layer FQT MLP:

```

// INPUT: batchInput tensor [1, 128] – a single 128-feature speech frame
// (in SYM_INT32: mantissas in [-32767, 32767], scale ≈ feature normalization factor)
// === FORWARD PASS ===
// LAYER 1: Linear (128 → 64)
layerFunctions[LINEAR].forward(&linear1Layer, &batchInput, &act1);
// Inside linearForward:
// 1. transposeTensor(w1, 0, 1) [128,64] → [64,128] (O(1))
// 2. matmulSymInt32TensorsWithBias(batchInput, w1, act1, b1)
// for each output neuron j in [0,64):
// acc = 0 (int64)
// for each input k in [0,128):
// acc += input[k] × w1[j][k] (int32×int32 → int64)
// realVal = acc × scaleInput × scaleW1 + bias[j] × scaleBias
// act1[j] = round(clamp(realVal / scaleAct1, qMin, qMax))
// (scale act1 is chosen to cover the full output range)
// 3. transposeTensor(w1, 0, 1) [64,128] → [128,64] (O(1) restore)
// LAYER 2: ReLU
layerFunctions[RELU].forward(&reluLayer, &act1, &act2);
// Inside reluForwardSymInt32:
// for each i: act2[i] = (act1[i] > 0) ? act1[i] : 0
// Copy act1's scale to act2 (ReLU doesn't change the scale)
// LAYER 3: Linear (64 → N_SPEAKERS)
layerFunctions[LINEAR].forward(&linear2Layer, &act2, &act3);
// Same matmul as layer 1 but [64 → N_SPEAKERS]
// LAYER 4: Softmax (N_SPEAKERS → probabilities)
layerFunctions[SOFTMAX].forward(&softmaxLayer, &act3, &output);
// Note: softmax requires FLOAT32 – if act3 is SYM_INT32, it gets converted
// Inside softmaxForward (FLOAT32):
// 1. Find max logit for numerical stability
// 2. exp(z - max) for each class
// 3. Divide by sum: output = probability distribution
// OUTPUT: output tensor [1, N_SPEAKERS] – FLOAT32 probabilities summing to 1.0
// e.g. output = [0.02, 0.01, 0.91, 0.03, 0.03] (speaker 2 most likely)

```

Section 11.5 — The Complete Backward Pass (Step by Step)

```

// === BACKWARD PASS (reverse order!) ===
// LOSS GRADIENT: dL/dlogit = softmax_output - one_hot_target
lossFunctions[CROSS_ENTROPY].backward(&output, &target, &grad0);
// grad0 = [0.02, 0.01, -0.09, 0.03, 0.03] (correct class gets -0.09)
// LAYER 4 BACKWARD: Softmax (combined with CE already done above)
// No separate softmax backward needed (CE + softmax combined gradient = above)
// LAYER 3 BACKWARD: Linear2 backward
layerFunctions[LINEAR].backward(&linear2Layer, &act2, &grad0, &grad1);
// Inside linearBackward:
// linearCalcWeightGradsFloat32(grad0, act2, w2.grad):
// dL/dw2 = grad0^T @ act2 [added to w2.grad]
// linearCalcBiasGradsFloat32(b2.grad, grad0):
// dL/db2 = sum(grad0, axis=0) [added to b2.grad]
// linearCalcPropLossFloat32(w2, grad0, grad1):
// grad1 = grad0 @ W2 [propagate loss to layer below]
// LAYER 2 BACKWARD: ReLU backward
layerFunctions[RELU].backward(&reluLayer, &act1, &grad1, &grad2);
// Inside reluBackwardFloat:
// for i: grad2[i] = (act1[i] > 0) ? grad1[i] : 0
// Gates the gradient: block where input was <= 0
// LAYER 1 BACKWARD: Linear1 backward
layerFunctions[LINEAR].backward(&linear1Layer, &batchInput, &grad2, NULL);
// grad propagated to input layer (NULL = no output needed, first layer)
// linearCalcWeightGradsFloat32(grad2, batchInput, w1.grad): dL/dw1
// linearCalcBiasGradsFloat32(b1.grad, grad2): dL/db1
// OPTIMIZER STEP: update all weights
sgdStep(&sgd, &w1Param, &forwardQ); // W1 -= lr * w1.grad
sgdStep(&sgd, &b1Param, &forwardQ); // b1 -= lr * b1.grad
sgdStep(&sgd, &w2Param, &forwardQ); // W2 -= lr * w2.grad
sgdStep(&sgd, &b2Param, &forwardQ); // b2 -= lr * b2.grad
// After all batches in epoch: requantize weights if needed
// (requantSymInt32Tensor updates scales to current value range)

```

Section 11.6 — Memory Flow Diagram

This diagram shows where data lives in SRAM at each stage of one training iteration. Each arrow represents a function call that reads from source buffers and writes to destination buffers.

```

SRAM CONTENTS during one forward + backward pass:
[w1Data] [g1Data] [b1Data] [w2Data] [g2Data] [b2Data] <- WEIGHTS (persistent)
| <- forward reads w1
v
[inputBuf] → [linearForward] → [act1Buf] [act1Buf stores: 64 SYM_INT32]
|
[reluForward] <- gteSymInt32Zero
|
[act2Buf] [act2Buf stores: 64 SYM_INT32]
|
[linear2Forward] <- reads w2
|
[act3Buf] [act3Buf: N_SPEAKERS floats]
|
[softmaxForward]
|
[outBuf] [N_SPEAKERS probabilities]
|
[CE.backward] <- computes loss
|
[grad0] [= output - target]
|
[linear2.backward] -> [g2Data] [b2 grad updated]
|
[grad1] [64-dim gradient]
|
[relu.backward] <- gate by act1
|
[grad2] [64-dim gated gradient]
|
[linear1.backward] -> [g1Data] [b1 grad updated]
|
[sgdStep × 4] -> [w1Data, b1Data, w2Data, b2Data]
Note: grad0, grad1, grad2 buffers can OVERLAP if they are different sizes
and backward is computed sequentially (only one gradient needed at a time).
This is a key memory optimization for embedded systems.

```

Section 11.7 — Performance Considerations on Cortex-M7

■ C CONCEPT: ARM Cortex-M7 Performance Characteristics

The Cortex-M7 has: - Double-issue pipeline: can execute 2 instructions per cycle in ideal cases - Harvard architecture: separate instruction and data caches - 16 KB L1 instruction cache + 16 KB L1 data cache - FPU (Floating-Point Unit): single-precision multiply in 1 cycle - TCM (Tightly Coupled Memory): zero-wait-state memory access Impact on ODT: - float multiply (matmulFloat32): 1 cycle per element (FPU MAC) - int32 multiply: also 1 cycle (MUL instruction) - memcpy, memset: uses DMA or optimised LDMIA instructions - The inner loop of matmul (K multiplies per output): runs in the cache as long as the weight row fits in 16 KB data cache For a [128, 64] matmul: inner row is $128 \times 4 = 512$ bytes (fits in 16 KB cache easily). The main bottleneck is memory bandwidth for large tensors that don't fit in cache.

Operation	Cost (approx)	Bottleneck
matmulFloat32 [1,128]×[128,64]	$128 \times 64 = 8192$ MACs ≈ 8192 cycles	FPU throughput
matmulSymInt32 [1,128]×[128,64]	8192 int64 MACs ≈ 8192 cycles	MUL/MULL throughput
requantSymInt32 (2 passes, 128×64)	2×8192 reads + 8192 writes $\approx 25K$ cycles	Memory bandwidth
reluForwardSymInt32 (64 elements)	64 comparisons ≈ 64 cycles	Trivial

sgdStep SYM_INT32 (128×64 weights)	2 convertTensor + VLA alloc + loop ≈ 100K cycles	Float conversion
softmaxForward (float, 64 classes)	64 exp() + 64 div ≈ 3000 cycles	exp() latency
Full forward pass (2-layer MLP)	~20K cycles ≈ 0.1 ms at 216 MHz	matmul dominant
Full backward pass	~3× forward ≈ 60K cycles ≈ 0.3 ms	3 matmuls (w1grad,w2grad,prop)
Full training step (batch=1)	~80K cycles ≈ 0.4 ms	Limited by matmul + SGD

— ■ WORKED EXAMPLE

Training speed estimate for 1000-sample dataset, 100 epochs: 1000 samples × 100 epochs × 0.4 ms/step = 40,000 ms = 40 seconds With RigL (80% sparse, 5× fewer matmul ops in forward): Forward: 0.02 ms instead of 0.1 ms Backward: still need full gradient → ~0.3 ms unchanged Total: ~0.32 ms/step Training time: 32 seconds (20% faster) The bigger win from RigL is INFERENCE speedup: after training, the sparse model runs 5× faster (no backward needed during inference).

Section 11.8 — The Layer Stack: How All 8 Layers Interact

Every function call in the training loop flows through all 8 architectural layers. Here is the complete chain of dependencies for one linearForward call:

```
// CALL: layerFunctions[LINEAR].forward(&layer, &input, &output)
// Layer 5 (Layers): dispatch
// → linearForward (Linear.c) [Layer 5]
// Layer 5 → Layer 4 (Arithmetic):
// → transposeTensor (Tensor.c) [Layer 3]
// orderOfDimensions swap – no lower layers called
// → matmulSymInt32TensorsWithBias (Matmul.c) [Layer 4]
// Layer 4 → Layer 3 (Tensor):
// → getDimensionsByIndex (Arithmetic.c) → tensor->shape
// → calcNumberOfElementsByTensor (Tensor.c)
// Layer 4 → Layer 1 (DTypes) for each element:
// → readBytesAsInt32 (DTypes.c) [Layer 1] – read A[i][k] and B[j][k]
// → writeInt32ToByteArray (DTypes.c) [Layer 1] – write output[i][j]
// Layer 4 → Layer 4 (Rounding) for quantised output:
// → roundByMode (Rounding.c) → clamp (Rounding.c)
// → roundByMode may call: rngNextFloat (RNG.c) [Layer 7]
// rngNextFloat → rngNext (private) – pure arithmetic
// Layer 4 → Layer 2 (Quantization) to read scales:
// → input->quantization->qConfig cast to symInt32QConfig_t*
// → read scale field (plain struct access, no function call)
// Layer 0 (Common) is active throughout:
// → PRINT_ERROR (Common.h) if any error occurs – wraps printf to UART
// → PRINT_DEBUG (Common.h) if DLEVEL >= DEBUG_LEVEL
// Total call depth: up to 4 levels
// Total functions touched: ~10 across 6 of the 8 layers
// Data touched: w1Data, g1Data buffers (weight) + act1Buf (output)
```

Section 11.9 — Debugging on the MCU

Debugging embedded code is harder than debugging on a PC. The ODT library provides several tools through Common.h:

```
// Common.h debugging tools:
// 1. DLEVEL: set debug verbosity at COMPILE TIME
// DLEVEL=0: silent (production) DLEVEL=3: very verbose
#define DLEVEL 2 // set in build system or CMakeLists.txt
// 2. PRINT_DEBUG: only prints if DLEVEL >= threshold
PRINT_DEBUG(2, "Layer 1 forward: scale=%.6f", scale);
// → uart output: "[TENSOR] Layer 1 forward: scale=0.000123"
// 3. PRINT_ERROR: always prints, then you can set a breakpoint
PRINT_ERROR("matmul: dimension mismatch %lu vs %lu", M, N);
// 4. printTensor: dump all tensor values to UART
printTensor(&act1Tensor);
// → prints all 64 mantissas and the current scale
// 5. printShape: dump shape information
printShape(tensor->shape);
// Common debugging workflow:
// Step 1: set DLEVEL=3, rebuild, flash to MCU
// Step 2: open UART terminal (115200 baud)
// Step 3: watch debug output as training runs
// Step 4: if a PRINT_ERROR fires, check which file/line
// Step 5: use SWD debugger (J-Link/OpenOCD) to set breakpoints
```

■ BEGINNER TIP

PRINT_ERROR uses `do { ... } while (false)` internally (see Chapter 2 on Common.h). This means you can safely put PRINT_ERROR inside an `if/else` without curly braces. The SOURCE_FILE macro is set at the top of each .c file so that error messages tell you exactly which module printed the error — very helpful when you see '[ARITHMETIC] Dimensions don't match' — you know to look in Arithmetic.c.

Section 11.10 — The End-to-End Speaker-ID Training Pipeline

Putting it all together: here is the complete pipeline from raw audio to a trained speaker-ID model on the STM32, using all components of the ODT library:

```

=== PHASE 1: DATA PREPARATION (on PC, Python) ===
1. Collect audio clips for N speakers.
2. Extract MFCC or filterbank features → numpy array [num_clips, 128].
3. Save as .npy file: np.save('features.npy', X)
4. Create labels: np.save('labels.npy', Y) (one-hot, shape [num_clips, N])
5. Transfer to STM32 via UART or embed in flash.
=== PHASE 2: INITIALIZATION (in main() on MCU) ===
6. SystemClock_Config() – set 216 MHz
7. MX_USART3_UART_Init() – set up UART
8. rngSetSeed(42) – reproducible training
9. Declare all static buffers, shapes, quantization configs
10. initLayer() for each layer
11. sgdInit() for optimizer
12. initDataLoader() with getSampleFromFlash callback
13. Load initial weights (random or pre-computed) via NPYLoader
=== PHASE 3: TRAINING (loop on MCU) ===
14. for epoch in [0..99]:
15. rngShuffleIndices(indices, datasetSize)
16. for batch in [0..datasetSize/batchSize):
17. sgdZeroGrad(all parameters)
18. load batch from DataLoader
19. linearForward(1) → reluForward → linearForward(2) → softmaxForward
20. crossEntropyForward → print loss
21. crossEntropySoftmaxBackward → linearBackward(2) → reluBackward
    → linearBackward(1)
22. scale gradients by 1/batchSize
23. sgdStep all parameters
24. print epoch loss to UART
=== PHASE 4: MODEL EXPORT (after training) ===
25. serializeModel(layers, numLayers, uartFile)
    → streams model weights to PC via UART
=== PHASE 5: INFERENCE (optional, back on MCU) ===
26. deserializeModel() from flash (pre-stored trained weights)
27. for each new audio clip:
28. extract features → fill inputTensor
29. forward pass (no backward needed)
30. predictedSpeaker = argmax(outputTensor)

```

Section 11.11 — Common Pitfalls and How to Avoid Them

■ ■ IMPORTANT

PITFALL 1: Forgetting `sgdZeroGrad` before each batch. Gradients accumulate across batches. The loss goes NaN. ALWAYS call `sgdZeroGrad` at the start of each batch.

■ ■ IMPORTANT

PITFALL 2: Mismatched quantization types. Calling `linearForwardFloat` with a `SYM_INT32` weight tensor will silently reinterpret the int32 mantissas as float bit patterns — garbage output. Always check that `forwardQ->type` matches the actual tensor type.

■ ■ IMPORTANT

PITFALL 3: Stack overflow from large VLAs. `sgdStepSymInt32` declares `'float wFloat[n]'` on the stack. For `n=8192` elements: 32 KB on the stack. The default ARM stack is often only 4-8 KB. Solution: increase stack size in the linker script, or use static buffers for large tensors.

■ ■ IMPORTANT

PITFALL 4: Not resetting scale after zeroing a `SYM_INT32` tensor. After `memset(grad, 0)`, the old scale is still set. The next gradient addition uses mantissa units relative to the old scale. `sgdZeroGrad` correctly resets the scale to 1.0 — always use `sgdZeroGrad`, never raw `memset` on a `SYM_INT32` gradient tensor.

■■ IMPORTANT

PITFALL 5: `transposeTensor` not called in pairs. `linearForwardFloat` transposes `w`, calls `matmul`, then transposes back. If an error occurs between the two transposes (e.g. dimension mismatch in `matmul`), the weight tensor stays transposed. The next forward pass uses the wrong layout. Use `PRINT_ERROR + return` (don't exit) if you need cleanup — or use `KeepTogether` for transpose-matmul-transpose as an atomic unit.

Chapter 12

Deep Dive — Arithmetic.c and Matmul.c

Every Function Explained Line by Line

Section 12.1 — getDimensionsByIndex: Handling Transposed Tensors

getDimensionsByIndex is subtle. It does NOT just return dimensions[index]. It searches the orderOfDimensions array to find which physical slot holds the requested logical dimension. This is needed because after transposeTensor, the mapping between logical and physical dimensions is scrambled.

```
size_t getDimensionsByIndex(tensor_t *tensor, size_t index) {
    size_t numberOfDims = tensor->shape->numberOfDimensions;
    for (size_t i = 0; i < numberOfDims; i++) {
        if (tensor->shape->orderOfDimensions[i] == index) {
            return tensor->shape->dimensions[i];
        }
    }
    PRINT_ERROR("Tensor doesn't have %lu dimensions!", index);
    exit(1);
}
```

■ C CONCEPT: Logical vs Physical Dimension Index

A freshly created [3,4] tensor has: dimensions[] = {3, 4} orderOfDimensions = {0, 1} (identity: logical 0 → physical 0)
After transposeTensor(t, 0, 1) (makes it a [4,3] transposed view): dimensions[] = {4, 3} (SWAPPED physically)
orderOfDimensions = {1, 0} (logical 0 → physical 1) getDimensionsByIndex(t, 0): Loops: orderOfDimensions[0]=1 ≠ 0; orderOfDimensions[1]=0 == 0 → returns dimensions[1] = 3 (logical dim 0 has size 3, which was rows before transpose) This correctly gives the LOGICAL dimension size, accounting for the transpose.

1	size_t numberOfDims = tensor->shape->numberOfDimensions;	Cache the number of dimensions to avoid repeated struct field access in the loop.
2-3	for (size_t i = 0; i < numberOfDims; i++) { if (orderOfDimensions[i] == index)	Search for the physical slot whose logical label matches 'index'. In the identity case this finds i==index immediately. After transpose, it may find a different i.
4	return tensor->shape->dimensions[i];	Return the physical dimension size at that slot. This is the correct SIZE for the requested logical dimension.
5-6	PRINT_ERROR(...); exit(1);	If no slot matched, the requested dimension index exceeds the tensor's rank. e.g. asking for dimension 2 of a 2D tensor (valid indices: 0,1).

Section 12.2 — orderDims: Materializing Logical Dimensions

```
void orderDims(tensor_t *tensor, size_t *orderedDims) {
    for (size_t i = 0; i < tensor->shape->numberOfDimensions; i++) {
        orderedDims[i] = getDimensionsByIndex(tensor, i);
    }
}
```

orderDims fills an array with the logical dimension sizes in order [0,1,2,...]. The caller provides a VLA array large enough for numberOfDimensions elements. After this call, orderedDims[0] = logical-size-of-dim-0, etc. This is

used by `int32PointWiseArithmetic` to compute indices correctly.

— ■ WORKED EXAMPLE

For a transposed [3,4] tensor viewed as [4,3]: `dimensions[] = {4, 3}` `orderOfDimensions = {1, 0}` `orderDims()` calls: `getDimensionsByIndex(t, 0) → looks for orderOfDims[i]==0 → i=1 → dimensions[1]=3` `getDimensionsByIndex(t, 1) → looks for orderOfDims[i]==1 → i=0 → dimensions[0]=4` `orderedDims = {3, 4}` — the LOGICAL sizes before transpose. This is used in `calcIndicesByRawIndex` so that flat-index-to-indices conversion uses the logical sizes, not the physical ones.

Section 12.3 — `doDimensionsMatch`: Shape Validation

```
bool doDimensionsMatch(tensor_t *a, tensor_t *b) {
    size_t aN = a->shape->numberOfDimensions;
    size_t bN = b->shape->numberOfDimensions;
    if (aN != bN) {
        PRINT_ERROR("Rank mismatch: %lu vs %lu", aN, bN);
        exit(1);
    }
    size_t aOrd[aN], bOrd[bN];
    orderDims(a, aOrd);
    orderDims(b, bOrd);
    for (size_t i = 0; i < aN; i++) {
        if (aOrd[i] != bOrd[i]) {
            PRINT_DEBUG("Dim mismatch at %lu: %lu vs %lu", i, aOrd[i], bOrd[i]);
            return false;
        }
    }
    return true;
}
```

■ C CONCEPT: `bool` Type in C

C (before C99) had no `bool` type. C99 introduced `_Bool` and the `<stdbool.h>` header which defines 'bool', 'true' (=1), and 'false' (=0). `#include <stdbool.h> bool flag = true; // flag holds 1 bool other = false; // other holds 0 if (flag) { ... } // checks if flag != 0` Functions returning `bool` are used for predicates (yes/no questions). `doDimensionsMatch` returns true if shapes are compatible for element-wise operations.

Ran k ch eck	<code>if (aN != bN) { PRINT_ERROR(); exit(1); }</code>	Two tensors with different numbers of dimensions cannot be added/multiplied element-wise. e.g. a 2D matrix and a 3D cube have incompatible shapes regardless of individual sizes.
VLA arra ys	<code>size_t aOrd[aN], bOrd[bN];</code>	Allocate two temporary arrays on the stack. Size is runtime-determined. Note: <code>aN == bN</code> after the rank check, so <code>aOrd</code> and <code>bOrd</code> are the same size.
ord erDi ms	<code>orderDims(a, aOrd); orderDims(b, bOrd);</code>	Materialize LOGICAL dimension sizes for both tensors. This handles transposed tensors correctly.
Dim ensi on c om pari son	<code>if (aOrd[i] != bOrd[i]) return false;</code>	Every logical dimension must match. <code>PRINT_DEBUG</code> prints a message but DOES NOT exit — returns false instead, letting the caller decide whether to abort or handle the mismatch.
retu rn true	All checks passed: shapes are element-wise compatible.	The tensors can be used in point-wise arithmetic together.

Section 12.4 — calcElementIndexByIndices: The Transpose-Aware Index

calcElementIndexByIndices converts logical multi-dim indices to a physical flat index, applying the orderOfDimensions permutation. This is the function that makes transposed tensor access correct.

```
size_t calcElementIndexByIndices(size_t numberOfDims, size_t *dims,
size_t *indices, size_t *orderOfDimensions) {
    size_t offset = 1;
    for (size_t i = 0; i < numberOfDims; i++) {
        offset *= dims[i]; // total elements = product of all dims
    }
    // Apply permutation: map logical indices to physical indices
    size_t orderedIndices[numberOfDims];
    for (size_t d = 0; d < numberOfDims; d++) {
        orderedIndices[orderOfDimensions[d]] = indices[d];
    }
    // Row-major index calculation using physical indices
    size_t outputIndex = 0;
    for (size_t i = 0; i < numberOfDims; i++) {
        offset /= dims[i];
        outputIndex += orderedIndices[i] * offset;
    }
    return outputIndex;
}
```

— ■ WORKED EXAMPLE

Example: access element [i=0, j=1] of a transposed [3,4] matrix (transposed from a physical [4,3] layout): Physical tensor: dims={4,3}, orderOfDimensions={1,0} Logical indices: indices={0, 1} (logical row 0, logical col 1) Phase 1: permute indices d=0: orderedIndices[orderOfDims[0]=1] = indices[0] = 0 → orderedIndices[1]=0 d=1: orderedIndices[orderOfDims[1]=0] = indices[1] = 1 → orderedIndices[0]=1 orderedIndices = {1, 0} Phase 2: row-major index from physical dims {4,3} offset = 4×3 = 12 i=0: offset /= 4 → offset=3; outputIndex += orderedIndices[0]×3 = 1×3 = 3 i=1: offset /= 3 → offset=1; outputIndex += orderedIndices[1]×1 = 0×1 = 0 outputIndex = 3 Physical byte offset = 3 × sizeof(int32_t) = 12 This accesses the element at row 1, col 0 of the ORIGINAL (untransposed) matrix, which is exactly what row 0, col 1 of the TRANSPOSED matrix should be. ✓

Section 12.5 — int32PointWiseArithmetic: Full Line-by-Line

```

// Full implementation from Arithmetic.c:
void int32PointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor) {
if (!doDimensionsMatch(aTensor, bTensor)) {
PRINT_ERROR("Dimensions don't match!"); exit(1);
}
size_t numberOfElements = calcNumberOfElementsByTensor(aTensor);
size_t bytesPerElement = sizeof(int32_t);
size_t numberOfDims = aTensor->shape->numberOfDimensions;
size_t *aDims = aTensor->shape->dimensions;
size_t *bDims = bTensor->shape->dimensions;
size_t aOrderedDims[numberOfDims];
size_t bOrderedDims[numberOfDims];
orderDims(aTensor, aOrderedDims);
orderDims(bTensor, bOrderedDims);
for (size_t i = 0; i < numberOfElements; i++) {
size_t aIndices[numberOfDims];
size_t bIndices[numberOfDims];
calcIndicesByRowIndex(numberOfDims, aOrderedDims, i, aIndices);
size_t aIdx = calcElementIndexByIndices(numberOfDims,
aDims, aIndices, aTensor->shape->orderOfDimensions);
calcIndicesByRowIndex(numberOfDims, bOrderedDims, i, bIndices);
size_t bIdx = calcElementIndexByIndices(numberOfDims,
bDims, bIndices, bTensor->shape->orderOfDimensions);
int32_t aVal = readBytesAsInt32(aTensor->data + aIdx*bytesPerElement);
int32_t bVal = readBytesAsInt32(bTensor->data + bIdx*bytesPerElement);
int32_t result = arithmeticFunc(aVal, bVal);
size_t oIdx = calcElementIndexByIndices(numberOfDims,
outputTensor->shape->dimensions, aIndices,
outputTensor->shape->orderOfDimensions);
writeInt32ToByteArray(result,
outputTensor->data + oIdx*bytesPerElement);
}
}

```

L1-2	doDimensionsMatch check and exit on failure	Correctness guard. Both input tensors must have the same shape (after accounting for transposes). Exits immediately on mismatch because continuing with mismatched shapes would write to wrong memory locations.
L3	numberOfElements = total number of elements in tensor A	This is the product of all dimensions. We iterate i from 0 to numberOfElements-1, covering every element.
L4	bytesPerElement = sizeof(int32_t) = 4	All INT32 elements are 4 bytes. We use this to compute byte offsets.
L5-7	Cache dimension arrays as local pointers	Avoids repeatedly dereferencing tensor->shape->dimensions in the hot loop. Modern compilers may do this anyway, but explicit caching makes intent clear.
L8-11	VLA arrays + orderDims: materialize logical sizes	aOrderedDims[i] = logical size of dimension i for tensor A (accounting for transpose). Used in calcIndicesByRowIndex to convert flat index → logical indices.
L12	Main loop: for each element position i in [0, N)	We process every element of the result tensor.
L13-14	VLA index arrays – new stack allocation per iteration	aIndices[d] = logical index along dimension d for element i. These are inside the loop so a new VLA is created each iteration. Compilers typically optimize this to a single stack frame adjustment.
L15	calcIndicesByRowIndex → convert flat i to logical [row, col, ...]	For a [3,4] tensor, flat index 6 → logical [1,2]. Uses aOrderedDims (the logical sizes) so the result is in LOGICAL space.

L16	<code>calcElementIndexByIndices</code> → convert logical indices to PHYSICAL byte offset	Applies the <code>orderOfDimensions</code> permutation. For a non-transposed tensor this is the same as row-major index. For a transposed tensor, the physical memory offset differs from the logical index.
L17-18	Same process for tensor B	B may have a DIFFERENT <code>orderOfDimensions</code> than A. This correctly handles: A is row-major, B is transposed.
L19-20	<code>readBytesAsInt32</code> to read values from both tensors	The byte offset = $\text{index} \times 4 \text{ bytes}$. <code>readBytesAsInt32</code> copies 4 bytes → <code>int32_t</code> .
L21	<code>arithmeticFunc(aVal, bVal)</code> – the actual operation	This is the function pointer call. For <code>addInt32s</code> : returns <code>aVal + bVal</code> . For <code>mulInt32s</code> : returns <code>aVal × bVal</code> . The loop logic is identical for all operations.
L22	Compute output physical index from the SAME logical indices as A	The output tensor has the same logical shape as A and B. We reuse <code>aIndices</code> (= logical position of element <code>i</code>) to find where to write.
L23	<code>writeInt32ToByteArray</code> – write result	Copies 4 bytes of result into the output tensor's data buffer.

■■ IMPORTANT

Performance note: `int32PointWiseArithmetic` calls `calcIndicesByRowIndex` and `calcElementIndexByIndices` per element. For an N-element tensor: $O(N \times \text{numDims})$ work just for index calculations. This is correct but slow compared to a simple flat-index loop (which would ignore `orderOfDimensions`). If you know your tensors are non-transposed, you can use the inplace variants (`int32PointWiseArithmeticInplace`) which directly iterate flat indices.

Section 12.6 — matmulIntCore: The Matrix Multiply Engine

`matmulIntCore` is the shared implementation for all INT32 matrix multiplications. It handles 1D vectors as well as 2D matrices. An optional bias array can be pre-added to the accumulator.

```

static void matmulIntCore(tensor_t *A, tensor_t *B, tensor_t *out,
const int32_t *biasSeed) {
if (A->shape->numberOfDimensions > 2 || B->shape->numberOfDimensions > 2) {
PRINT_ERROR("Matmul only supports up to 2D Tensors"); exit(1);
}
size_t aRows, aColumns;
if (A->shape->numberOfDimensions < 2) {
aRows = 1; aColumns = getDimensionsByIndex(A, 0);
} else {
aRows = getDimensionsByIndex(A, 0);
aColumns = getDimensionsByIndex(A, 1);
}
size_t bRows = getDimensionsByIndex(B, 0);
size_t bColumns = (B->shape->numberOfDimensions < 2)
? 1 : getDimensionsByIndex(B, 1);
if (aColumns != bRows) {
PRINT_ERROR("Rows don't match Columns"); exit(1);
}
size_t resultCounter = 0;
for (size_t row = 0; row < aRows; row++) {
for (size_t col = 0; col < bColumns; col++) {
int32_t result = biasSeed ? biasSeed[col] : 0;
for (size_t k = 0; k < aColumns; k++) {
size_t aIdx[2] = {row, k};
size_t aPhys = calcElementIndexByIndices(2, A->shape->dimensions,
aIdx, A->shape->orderOfDimensions);
int32_t aV = readBytesAsInt32(A->data + aPhys*4);
size_t bIdx[2] = {k, col};
size_t bPhys = calcElementIndexByIndices(2, B->shape->dimensions,
bIdx, B->shape->orderOfDimensions);
int32_t bV = readBytesAsInt32(B->data + bPhys*4);
result += mulInt32s(aV, bV);
}
writeInt32ToByteArray(result, out->data + resultCounter*4);
resultCounter++;
}
}
}
}

```

1D handling	if (numberOfDimensions < 2) { aRows=1; aColumns=dim0; }	A 1D vector [K] is treated as a [1,K] row vector for matrix multiplication. This enables matmul to handle both vector-matrix and matrix-matrix products.
Dimension check	if (aColumns != bRows) { PRINT_ERROR; exit(1); }	The inner dimension must match: A is [M,K], B must be [K,N]. If K doesn't match, matrix multiplication is undefined.
bias Seed	result = biasSeed ? biasSeed[col] : 0;	Start the accumulator with the bias value for column 'col'. This elegantly fuses the bias addition into the matmul loop, avoiding a separate addition pass over the output.
Inner loop	for k: aPhys = calcElementIndex(A, [row,k]);	Look up A[row][k] using the transpose-aware index function. For linearForward: A has been transposed, so A[row][k] accesses the COLUMN of the weight matrix before the transpose.

result += mullnt32s(aV, bV)	Accumulate in int32_t.	For small K (e.g. 64), the maximum accumulation is $64 \times 32767^2 \approx 68B$ — which OVERFLOWS int32_t (max 2.1B). For safe INT32 accumulation, qMaxBits must be small enough that $K \times qMax^2 < INT32_MAX$. The SYM_INT32 variant uses int64_t to avoid this overflow.
resultCounter++	Output is written in row-major order.	output[row*bColumns + col] = result. resultCounter is a flat index that increments for each output element.

■ WORKED EXAMPLE

Trace matmulIntCore for A=[2,3] and B=[3,2] (no bias): A = [[1,2,3],[4,5,6]] B = [[7,8],[9,10],[11,12]] Expected C = A×B = [[1×7+2×9+3×11, 1×8+2×10+3×12], [4×7+5×9+6×11, ...]] = [[58, 64], [139, 154]] Loop trace: row=0, col=0: result=0; k=0: 1×7=7; k=1: +2×9=25; k=2: +3×11=58. Write 58. row=0, col=1: result=0; k=0: 1×8=8; k=1: +2×10=28; k=2: +3×12=64. Write 64. row=1, col=0: result=0; k=0: 4×7=28; k=1: +5×9=73; k=2: +6×11=139. Write 139. row=1, col=1: result=0; k=0: 4×8=32; k=1: +5×10=82; k=2: +6×12=154. Write 154. Output flat: [58, 64, 139, 154] ✓

Section 12.7 — matmulSymInt32TensorsWithBias: The FQT Core

```

void matmulSymInt32TensorsWithBias(tensor_t *A, tensor_t *B,
tensor_t *out, tensor_t *bias) {
symInt32QConfig_t *aCfg = A->quantization->qConfig;
symInt32QConfig_t *bCfg = B->quantization->qConfig;
symInt32QConfig_t *outCfg = out->quantization->qConfig;
symInt32QConfig_t *biasCfg = bias->quantization->qConfig;
float scaleA = aCfg->scale;
float scaleB = bCfg->scale;
float scaleOut = outCfg->scale;
float scaleBias = biasCfg->scale;
int32_t outQMax = (1 << (outCfg->qMaxBits - 1)) - 1; // e.g. 32767
int32_t outQMin = -outQMax;
size_t M = getDimensionsByIndex(A, 0);
size_t K = getDimensionsByIndex(A, 1);
size_t N = getDimensionsByIndex(B, 1);
size_t resultCounter = 0;
for (size_t row = 0; row < M; row++) {
for (size_t col = 0; col < N; col++) {
int64_t acc = 0; // 64-bit to prevent overflow
for (size_t k = 0; k < K; k++) {
// A[row][k] and B[k][col] – respecting orderOfDimensions
int32_t aV = /* readBytesAsInt32 at [row][k] */;
int32_t bV = /* readBytesAsInt32 at [k][col] */;
acc += (int64_t)aV * (int64_t)bV;
}
// Convert from mantissa-product to real value:
float realResult = (float)acc * scaleA * scaleB;
// Add bias (bias is already in real-value units × scaleBias):
float biasReal = readBytesAsInt32(bias->data + col*4) * scaleBias;
realResult += biasReal;
// Quantise to output SYM_INT32:
int32_t q = roundByMode(
clamp(realResult / scaleOut, outQMin, outQMax),
outCfg->roundingMode);
writeInt32ToByteArray(q, out->data + resultCounter*4);
resultCounter++;
}
}
}

```

Real scales	scaleA, scaleB, scaleOut, scaleBias from qConfig	Each tensor has its own scale. $\text{real_A}[i][k] = \text{A_mantissa}[i][k] \times \text{scaleA}$. The product $\text{real_A} \times \text{real_B} = \text{mantissa_A} \times \text{mantissa_B} \times \text{scaleA} \times \text{scaleB}$.
out QMax	$(1 \ll (\text{qMaxBits}-1)) - 1$	$\text{qMaxBits}=16 \rightarrow \text{outQMax} = 2^{15} - 1 = 32767$. The $\ll$ operator is a bit shift: $1 \ll 15 = 32768$. Subtract 1 = 32767. This is the maximum representable mantissa.
int64_t acc = 0	64-bit accumulator for the inner product.	Each product $\text{aV} \times \text{bV}$ fits in <code>int32_t</code> , but K products may overflow. Casting to <code>int64_t</code> before multiply prevents the overflow.
(int64_t)aV × (int64_t)bV	Cast to 64-bit BEFORE the multiply, not after.	Without the cast: $\text{int32} \times \text{int32} = \text{int32}$ (potential overflow). With the cast: $\text{int64} \times \text{int64} = \text{int64}$ (no overflow for K up to ~2M).

float realResult = acc × scaleA × scaleB	Convert mantissa-product accumulation to real value.	acc holds $\text{sum}(aM[k] \times bM[k])$. $\text{real_result} = \text{sum}(aM[k] \times \text{scaleA} \times bM[k] \times \text{scaleB}) = \text{acc} \times \text{scaleA} \times \text{scaleB}$.
bias Real = bias M × scaleBias	Bias is also in SYM_INT32. Convert to real value and add.	The bias is NOT converted to float VLA here — we read one bias element at a time.
clamp(..., outQMin, outQMax)	Clip the real value to the output's representable range.	If $\text{realResult} / \text{scaleOut} = 40000$ but $\text{outQMax} = 32767$: clamp to 32767. This saturation is INTENTIONAL — values outside the range are clipped.
roundByMode(..., roundingMode)	Round the float to nearest int32_t using the configured rounding mode.	HALF_AWAY: standard round(). SR_HALF_AWAY: add noise before rounding (stochastic rounding for gradient estimation).

Section 12.8 — TRACK_INSTRUCTIONS: Profiling Support

Matmul.c has a conditional compile feature for profiling: if TRACK_INSTRUCTIONS is defined, each matmul call increments a counter. This lets you measure how many multiply-accumulate operations your model uses.

```
// At the top of Matmul.c:
#ifdef TRACK_INSTRUCTIONS
#define MATMUL_FUNC_INT matmulIntTensorsWithInstructionCounter
#else
#define MATMUL_FUNC_INT matmulIntTensors
#endif
// The counter:
size_t matmulInstructionCounter = 0;
// Accessed via:
size_t getMatmulInstructionCounter() { return matmulInstructionCounter; }
// Add.h also has addInstructionCounter:
size_t getAddInstructionCounter() { return addInstructionCounter; }
```

■ C CONCEPT: `#ifdef` — Conditional Compilation

`#ifdef SYMBOL` compiles the following block only if `SYMBOL` is defined. `#endif` ends the conditional block. `#ifndef` compiles only if `SYMBOL` is NOT defined. In `CMakeLists.txt` or `Makefile`:

`add_definitions(-DTRACK_INSTRUCTIONS)` → defines `TRACK_INSTRUCTIONS` (Without this flag, the profiling code is completely absent from the binary) Why conditional? Profiling adds overhead (counter increment per `matmul` call). In production (deployment), you want maximum speed. `#ifdef` lets you enable profiling for benchmarking and remove it for deployment with a single build flag, without changing source code.

Chapter 13

C Language Deep Dive for Embedded Systems

Pointers, Memory Layout, Structs, and All C Patterns Used in ODT

Section 13.1 — Pointers: The Most Important C Concept

■ C CONCEPT: What is a Pointer?

A pointer is a variable that holds a MEMORY ADDRESS (a location in RAM). Instead of storing a value, it stores the address WHERE a value is stored. `int x = 42;` // x holds the VALUE 42, lives at some address (say 0x2000) `int *p = &x;` // p holds the ADDRESS 0x2000 (& = 'address of') `int y = *p;` // *p = dereference: READ the value at address 0x2000 = 42 `*p = 100;` // write 100 to address 0x2000 → x is now 100 Types of pointers in ODT: `uint8_t *data` → points to a byte array (tensor storage) `size_t *dimensions` → points to an array of sizes `tensor_t *tensor` → points to a `tensor_t` struct `void *qConfig` → generic pointer to any config struct `forwardFn_t forward` → pointer to a function

```
// Pointer examples from the ODT library:
// 1. tensor_t has a pointer to its data:
tensor_t t;
uint8_t buf[48]; // 12 float32 elements
t.data = buf; // t.data now holds the address of buf[0]
t.data[0] = 0x3F; // write byte to buf[0] (same as buf[0] = 0x3F)
// 2. Pointer arithmetic – move a pointer forward by N bytes:
uint8_t *elem3 = t.data + 3 * sizeof(float); // points to element 3
// This is equivalent to &t.data[3 * sizeof(float)]
// 3. Arrow operator → : dereference + field access in one step:
tensor_t *pt = &t;
size_t n = pt->shape->numberOfDimensions; // same as (*pt).shape->numberOfDimensions
// 4. NULL pointer: a pointer that doesn't point to anything valid:
tensor_t *nothing = NULL;
// Accessing nothing->data would crash (segfault / HardFault on STM32)
if (nothing != NULL) { // ALWAYS check before using
    size_t x = nothing->shape->numberOfDimensions;
}
// 5. const pointer: can't modify what it points to:
const uint8_t *readOnly = buf; // can read readOnly[0] but NOT write it
// readOnly[0] = 5; ← compiler error
```

■ C CONCEPT: Why Pointers are Essential for ODT

On a microcontroller with 320 KB of RAM, copying data wastes memory and time. ODT passes `tensor_t*` everywhere instead of `tensor_t` (by value) because: 1. `sizeof(tensor_t)` = ~16 bytes (4 pointers). Passing by pointer: 4 bytes on 32-bit ARM. 2. Modification: functions that update tensors (forward pass writes output) MUST receive a pointer to modify the original, not a copy. 3. Large buffers: `tensor->data` points to potentially megabytes of weight data. Copying by value would copy that too — catastrophic. Rule: if the function READS a tensor, pass `const tensor_t*`. If it MODIFIES a tensor, pass `tensor_t*`.

Section 13.2 — Structs: Grouping Related Data

```

// Struct basics:
typedef struct point {
int x;
int y;
} point_t;
// Creating and using:
point_t p; // p is on the stack, uninitialized
p.x = 3; // set field x
p.y = 4;
int dist = p.x * p.x + p.y * p.y; // = 25
// Struct on the heap (ODT avoids this, uses static instead):
point_t *p2 = malloc(sizeof(point_t));
p2->x = 5; // arrow: dereference then access
free(p2);
// Struct layout in memory (no padding for same-sized fields):
// [x: 4 bytes][y: 4 bytes] = 8 bytes total
// sizeof(point_t) = 8
// Struct with pointer fields (the ODT pattern):
typedef struct shape {
size_t numberOfDimensions; // 4 or 8 bytes
size_t *dimensions; // 4 bytes (pointer)
size_t *orderOfDimensions; // 4 bytes (pointer)
} shape_t;
// sizeof(shape_t) ≈ 12 bytes on 32-bit ARM
// The actual dimension data is ELSEWHERE, pointed to by dimensions*

```

■ C CONCEPT: Struct vs Union vs Enum

struct: ALL fields exist simultaneously. Total size = sum of field sizes (+ alignment). struct { int a; float b; } → 8 bytes, both a and b are valid. union: ONLY ONE field valid at a time. Total size = LARGEST field size. union { linearConfig_t *linear; reluConfig_t *relu; } → 4 bytes (one pointer). Reading the wrong field is UNDEFINED BEHAVIOR. enum: named integer constants. enum {A, B, C} → A=0, B=1, C=2. sizeof(enum) = sizeof(int) = 4 bytes on ARM. ODT uses all three: - tensor_t, shape_t, linearConfig_t: structs (all fields always present) - layerConfig_t: union (holds one of 11 config pointers) - qtype_t, layerType_t, lossFuncType_t: enums (small integer sets)

Section 13.3 — Arrays, Pointer Arithmetic, and Multi-Dimensional Access

```

// Arrays in C are CONTIGUOUS blocks of memory:
int arr[5] = {10, 20, 30, 40, 50};
// [0] [1] [2] [3] [4]
// Memory: 10 20 30 40 50 (each int = 4 bytes)
// Address of arr[0]: 0x1000
// Address of arr[1]: 0x1004 (0x1000 + 4)
// Address of arr[2]: 0x1008 etc.
// Pointer arithmetic: arr+i points to arr[i]
int *p = arr; // p points to arr[0]
p++; // p now points to arr[1] (advances by sizeof(int) = 4 bytes)
// *(p+2) = arr[3] (two ints forward from arr[1])
// ODT tensor access: element i at byte offset i*sizeof(int32_t):
int32_t val = readBytesAsInt32(tensor->data + i * sizeof(int32_t));
// tensor->data is uint8_t*
// Adding i*4 gives the byte address of the i-th int32_t element
// 2D array as flat 1D (row-major):
// Matrix [3][4] stored as: [row0: 4 ints][row1: 4 ints][row2: 4 ints]
// Element [r][c] at flat index r*4 + c
// In bytes: (r*4 + c) * sizeof(int32_t)
// The ODT equivalent using calcElementIndexByIndices:
size_t indices[2] = {r, c};
size_t flatIdx = calcElementIndexByIndices(2, dims, indices, order);
int32_t v = readBytesAsInt32(tensor->data + flatIdx * 4);

```

— ■ WORKED EXAMPLE

Memory layout of a [3×4] INT32 tensor with data [0,1,...,11]: Address: 0x2000 0x2004 0x2008 0x200C 0x2010 ...
Value: 0 1 2 3 4 ... Meaning: [0][0] [0][1] [0][2] [0][3] [1][0] ... tensor->data = 0x2000 (pointer to start) Element [1][2] =
flat index $1 \times 4 + 2 = 6$ Byte offset = $6 \times 4 = 24$ bytes Address = $0x2000 + 24 = 0x2018$ Value at 0x2018 = 6 ✓

Section 13.4 — Function Pointers: C's Dispatch Mechanism

```
// A function pointer stores the ADDRESS of a function.
// Step 1: define the function signature type:
typedef int32_t (*int32BinaryOp_t)(int32_t a, int32_t b);
// This says: 'int32BinaryOp_t is a pointer to any function
// that takes two int32_t and returns int32_t'
// Step 2: write concrete functions matching this signature:
int32_t addInt32s(int32_t a, int32_t b) { return a + b; }
int32_t mulInt32s(int32_t a, int32_t b) { return a * b; }
int32_t maxInt32s(int32_t a, int32_t b) { return (a > b) ? a : b; }
// Step 3: store or pass a function pointer:
int32BinaryOp_t op = addInt32s; // op now holds the address of addInt32s
int32_t result = op(3, 4); // calls addInt32s(3,4) = 7
op = mulInt32s;
result = op(3, 4); // calls mulInt32s(3,4) = 12
// Step 4: pass as argument (like int32PointWiseArithmetic):
void applyToAll(int32_t *arr, size_t n, int32_t x, int32BinaryOp_t op) {
    for (size_t i = 0; i < n; i++)
        arr[i] = op(arr[i], x);
}
// Usage:
int32_t data[4] = {1, 2, 3, 4};
applyToAll(data, 4, 10, addInt32s); // adds 10 to each: {11,12,13,14}
applyToAll(data, 4, 2, mulInt32s); // multiplies by 2: {22,24,26,28}
```

■ C CONCEPT: Function Pointer Typedef Syntax — The Tricky Part

`typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*);` Read this inside-out: `*forwardFn_t` → it's a pointer (`*forwardFn_t`) → to a function (`(layer_t*, tensor_t*, tensor_t*)`) → taking these parameters `void` → returning `void` So `forwardFn_t` is a type alias for 'pointer to function(`layer_t*, tensor_t*, tensor_t*`) returning `void`'. An alternative syntax (C23/GCC extension) using `__typeof__` or function alias macros exists but is non-standard. The typedef form above is portable C99.

Section 13.5 — Storage Classes: static, extern, and Global Variables

```
// 1. LOCAL (automatic) – lives on the stack, dies when function returns:
void foo() {
int x = 42; // created on stack entry, freed on return
}
// 2. STATIC (file-scope) – lives in BSS/data section for entire program:
static uint8_t weightData[32768]; // declared outside functions
// OR static inside a function:
void bar() {
static int count = 0; // persists between calls!
count++;
}
// 3. EXTERN – 'this variable is defined in another .c file':
// In Layer.h:
extern layerFunctions_t layerFunctions[];
// This declares that layerFunctions[] is DEFINED in Layer.c
// Any .c file that includes Layer.h can use layerFunctions[] without redefining it
// In Layer.c:
layerFunctions_t layerFunctions[] = { // ← the actual DEFINITION
[LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
// ...
};
// 4. STATIC + function scope = module-level state (hidden from other files):
// In RNG.c:
static rng32_t rng = { .state = 1 }; // only RNG.c can access this
// Other files cannot accidentally modify rng – it's encapsulated in RNG.c
```

■ C CONCEPT: Why static at File Scope?

In C, a function or variable declared static at file scope is PRIVATE to that .c file. Other files cannot see or use it (the linker doesn't export it). This is C's primary encapsulation mechanism: static rng32_t rng → only RNG.c's functions can modify the RNG state static uint32_t rngNext(...) → only RNG.c can call rngNext; public API is rngNextFloat Without static, rng would be visible to ALL .c files (global scope). Any file could accidentally reset the seed to 0 and corrupt training.

Section 13.6 — Memory Sections on ARM Cortex-M7

Location	Contents	Lifetime
Flash (1MB)	Compiled code, string literals, const arrays	Forever (read-only)
SRAM (copied from flash at startup)	Initialized global/static variables (non-zero initial value)	Forever
SRAM (zero-initialized at startup)	Uninitialized global/static variables (zeroed by startup)	Forever
SRAM (grows downward)	Function call frames, local variables, VLAs	Duration of function
SRAM (between .bss and stack)	malloc'd memory	Until free()

```

// Example: where does each variable live?
static uint8_t weightData[32768]; // → .bss (large, initialized to 0)
static float lr = 0.001f; // → .data (non-zero initial value)
const char *msg = "hello"; // msg ptr → .data; "hello" string → .text
void train() {
    int epoch = 0; // → stack
    float tmpBuf[1024]; // → stack (VLA: 4KB! careful with stack size)
    float *grad = malloc(N*4); // → heap (should use static instead!)
    // After train() returns:
    // epoch and tmpBuf are gone (stack frame freed)
    // grad is LEAKED unless free(grad) is called
    free(grad);
}
// Memory map at runtime (typical Cortex-M7):
// 0x00000000 - 0x001FFFFFF: Flash (1MB)
// .text + .rodata (.bss/.data initializers)
// 0x20000000 - 0x2004FFFF: SRAM (320KB)
// .data → .bss → heap → (gap) → stack (grows down)

```

Section 13.7 — Preprocessor Directives in Detail

```

// The preprocessor runs BEFORE compilation. It does text substitution.
// #define: simple text replacement
#define MAX_LAYERS 16
layer_t *layers[MAX_LAYERS]; // becomes: layer_t *layers[16];
// #define with arguments (function-like macro):
#define ABS(x) ((x) < 0 ? -(x) : (x))
int a = ABS(-5); // becomes: int a = ((-5) < 0 ? -(-5) : (-5)) = 5
// Note the extra parentheses around x: ABS(a+b) → ((a+b) < 0 ? -(a+b) : (a+b))
// Without them: ABS(a+b) → (a+b < 0 ? -a+b : a+b) – WRONG due to precedence!
// #include: paste file contents
#include <stdint.h> // system header (angle brackets: search system paths)
#include "Tensor.h" // project header (quotes: search current dir first)
// #ifdef / #ifndef / #endif: conditional compilation
#ifndef ODT_TENSOR_H // if ODT_TENSOR_H is NOT defined:
#define ODT_TENSOR_H // define it (marking this file as included)
// ... header content ...
#endif // end of conditional
// #pragma: compiler-specific directives (non-portable):
#pragma once // alternative to header guards (GCC/Clang extension)
// __VA_ARGS__: variadic macros
#define PRINT_DEBUG(level, fmt, ...) \
do { if (DLEVEL >= level) printf(fmt, ##__VA_ARGS__); } while(0)
// ## before __VA_ARGS__: removes the comma if no args follow fmt
PRINT_DEBUG(2, "Loss: %.4f", loss); // → printf("Loss: %.4f", loss)
PRINT_DEBUG(1, "starting"); // → printf("starting") (no comma issue)

```

Section 13.8 — Type Casting in C

```

// Explicit cast: (type)expression
// 1. Widening cast – safe, no information loss:
int32_t a = 32767;
int64_t b = (int64_t)a; // 32-bit → 64-bit, always safe
// 2. Narrowing cast – may lose information:
int64_t big = 1000000000000LL;
int32_t small = (int32_t)big; // TRUNCATION – only lower 32 bits kept
// Always clamp before narrowing!
// 3. Float ↔ integer cast:
float f = 3.14f;
int32_t i = (int32_t)f; // rounds TOWARD ZERO: i = 3
// Use roundf() to round to nearest: (int32_t)roundf(f) = 3
// 4. Pointer cast – the most dangerous!
uint8_t *bytes = ...
float *floats = (float *)bytes; // ONLY safe if bytes is properly aligned!
// On ARM, unaligned float access may trigger HardFault.
// Safe alternative: memcpy into a float variable.
// 5. void* cast – used for qConfig:
void *ptr = &symInt32QCfg;
symInt32QConfig_t *cfg = (symInt32QConfig_t *)ptr; // explicit cast
// In C (not C++), void* can be implicitly assigned to any pointer.
// But explicit cast makes intent clear.
// 6. The safe cast pattern from ODT:
// WRONG:
float *wArr = (float *)w->data; // only safe for FLOAT32 tensors!
// RIGHT:
if (w->quantization->type == FLOAT32) {
float *wArr = (float *)w->data; // now guaranteed safe
}

```

Section 13.9 — Embedded C Patterns Specific to ODT

13.9.1 Designated Initializers

```

// Designated initializer: initialize specific fields by name
static rng32_t rng = { .state = 1 };
// Equivalent to: rng.state = 1; (all other fields = 0)
// Array designated initializer (used in dispatch table):
layerFunctions_t layerFunctions[] = {
[LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
[RELU] = { reluForward, ... },
[SOFTMAX] = { softmaxForward, ... },
};
// [LINEAR] = 0, [RELU] = 1, etc. (from the enum values)
// Slots not initialized (e.g. indices between 0 and LINEAR) are zero-filled
// This syntax requires C99 or later.

```

13.9.2 Compound Literals

```

// Compound literal: create a temporary struct/array inline:
size_t aIdx[2] = {row, k}; // stack array
// Or inline (C99):
calcElementIndexByIndices(2, dims, (size_t[]){row, k}, order);
// (size_t[]){row, k} creates a temporary 2-element array
// Used in matmulIntCore:
size_t aIndices[] = {rowIndex, i}; // local array, no separate declaration

```

13.9.3 Flexible Array Members and Zero-Length Arrays

```
// The ODT dispatch tables use a flexible-size approach:
layerFunctions_t layerFunctions[]; // no size specified = extern (defined elsewhere)
// In the definition file, size is determined by the initializer:
layerFunctions_t layerFunctions[] = {
[LINEAR] = ..., [RELU] = ..., ..., [LAYERNORM] = ...
};
// The compiler counts the initializers and sets the array size automatically.
// sizeof(layerFunctions) / sizeof(layerFunctions[0]) = number of layer types
```

13.9.4 exit(1) vs Return Error Codes

```
// ODT uses exit(1) for fatal errors:
if (aColumns != bRows) {
    PRINT_ERROR("Dimension mismatch");
    exit(1);
}
// Alternative: return error code (not used in ODT for simplicity):
int matmul(...) {
    if (aColumns != bRows) return -1; // error
    // ... compute ...
    return 0; // success
}
// Caller must check: if (matmul(...) != 0) { /* handle error */ }
// On bare-metal MCU, exit(1) typically calls _exit() which loops forever
// (or resets the MCU, depending on the startup code).
// This is acceptable: a dimension mismatch is a programming error,
// not a runtime condition to recover from.
// For production code, a reset might be more appropriate:
// NVIC_SystemReset(); // ARM reset via CMSIS
```

Section 13.10 — Common C Mistakes in Embedded Systems

■ ■ IMPORTANT

MISTAKE 1: Integer overflow in accumulation. `int32_t acc += a * b;` // `a*b` may overflow before being added to `acc`
 FIX: `acc += (int64_t)a * (int64_t)b;` — cast BEFORE multiply

■ ■ IMPORTANT

MISTAKE 2: Signed/unsigned comparison. `for (int i = n-1; i >= 0; i--)` // works fine for `(size_t i = n-1; i >= 0; i--)` //
 INFINITE LOOP! `size_t` is unsigned, wraps to HUGE after 0 FIX: use 'int' for loop counters that must go negative.

■ ■ IMPORTANT

MISTAKE 3: Dangling pointer. `tensor_t *t = &localTensor;` // `t` points to a local variable `return t;` // `localTensor` is freed on return! `t` now points to garbage FIX: use static/global arrays or heap allocation for tensors that must outlive functions.

■ ■ IMPORTANT

MISTAKE 4: Missing volatile for hardware registers. `while (!(UART->SR & 0x40)) {}` // busy-wait for UART ready
 Without 'volatile', the compiler may cache `UART->SR` and loop forever. FIX: hardware registers are declared volatile in STM32 HAL headers.

■ ■ IMPORTANT

MISTAKE 5: Forgetting NULL check before pointer dereference. `tensor->sparsity->config->mask` // if `sparsity==NULL`: HardFault! FIX: always check: `if (tensor->sparsity != NULL && tensor->sparsity->config != NULL)`

Chapter 14

Complete Function Reference Index

Every Public Function in the ODT Library

Section 14.1 — DTypes Module

Function	Signature	Purpose
readBytesAsInt32	int32_t f(const uint8_t *bytes)	Copy 4 bytes into int32_t via memcpy. Used to read int32 from tensor byte buffer.
readBytesAsFloat	float f(const uint8_t *bytes)	Copy 4 bytes into float via memcpy. Used to read float32 from tensor byte buffer.
writeInt32ToByteArray	void f(int32_t value, uint8_t *bytes)	Copy int32_t into 4 bytes via memcpy. Used to write int32 results into tensor.
writeFloatToByteArray	void f(float value, uint8_t *bytes)	Copy float into 4 bytes via memcpy. Used to write float results into tensor.
writeBitsToByteArray	void f(uint8_t *ba, size_t start, size_t end, uint8_t data)	Write sub-byte bits into a byte array. Used for BOOL tensors and sparse masks.
readBitsFromByteArray	uint8_t f(const uint8_t *ba, size_t start, size_t end)	Read sub-byte bits from a byte array. Companion to writeBitsToByteArray.

Section 14.2 — Quantization Module

Function	Signature	Purpose
initInt32Quantization	void f(quantization_t *q)	Set type=INT32, qConfig=NULL.
initFloat32Quantization	void f(quantization_t *q)	Set type=FLOAT32, qConfig=NULL.
initSymInt32Quantization	void f(quantization_t *q, symInt32QConfig_t *cfg)	Set type=SYM_INT32, qConfig=cfg.
initBoolQuantization	void f(quantization_t *q, boolQConfig_t *cfg)	Set type=BOOL, qConfig=cfg.
initInt32QConfig	void f(int32QConfig_t *cfg)	Initialize empty INT32 config (no-op, placeholder).
initFloat32QConfig	void f(float32QConfig_t *cfg)	Initialize empty FLOAT32 config.
initSymInt32QConfig	void f(roundingMode_t rm, symInt32QConfig_t *cfg)	Set scale=1.0, qMaxBits=16, roundingMode=rm.
calcNumberOfBytesForData	size_t f(quantization_t *q, size_t N)	Return bytes needed for N elements of type q->type.
calcBitsPerElement	size_t f(quantization_t *q)	Return bits per element for the given quantization type.

Section 14.3 — Tensor Module

Function	Signature	Purpose
setTensorValues	void f(tensor_t*, uint8_t*, shape_t*, quantization_t*, sparsity_t*)	Link all tensor fields. No allocation — stores pointers.
setParameterValues	void f(parameter_t*, tensor_t* param, tensor_t* grad)	Link weight and gradient into parameter_t.
setShape	void f(shape_t*, size_t* dims, size_t N, size_t* order)	Link shape fields. No allocation.

setOrderOfDimsForNewTensor	void f(size_t N, size_t* order)	Fill order with identity {0,1,...,N-1}.
transposeTensor	void f(tensor_t*, size_t dim0, size_t dim1)	Swap two axes in orderOfDimensions (O(1)).
copyTensor	void f(tensor_t* dest, tensor_t* src)	memcpy data bytes from src to dest.
calcNumberOfElementsByShape	size_t f(shape_t*)	Product of all dimensions.
calcNumberOfElementsByTensor	size_t f(tensor_t*)	Calls calcNumberOfElementsByShape on tensor->shape.
calcBytesPerElement	size_t f(quantization_t*)	Bytes per element based on quantization type.
calcBytesPerTensor	size_t f(tensor_t*)	Total bytes = elements × bytesPerElement.
tensorBoolGet	bool f(tensor_t*, size_t flatIndex)	Read 1 bit from BOOL tensor at flatIndex.
tensorBoolSet	void f(tensor_t*, size_t flatIndex, bool val)	Write 1 bit to BOOL tensor at flatIndex.
getParamFromParameter	tensor_t* f(parameter_t*)	Return p->param (weight tensor).
getGradFromParameter	tensor_t* f(parameter_t*)	Return p->grad (gradient tensor).
printTensor	void f(tensor_t*)	Print all values to UART (debug use).
printShape	void f(shape_t*)	Print shape info to UART.

Section 14.4 — TensorConversion Module

Function	Signature	Purpose
convertTensor	void f(tensor_t* in, tensor_t* out)	Convert in to out's quantization type. Dispatches via conversionMatrix[fromType][toType].
requantSymInt32Tensor	void f(tensor_t* in, tensor_t* out)	Two-pass SYM_INT32 → SYM_INT32 with auto-computed scale. In-place safe.
requantSymInt32TensorToScale	void f(tensor_t* in, tensor_t* out)	SYM_INT32 → SYM_INT32 using pre-set target scale. Caller must set out->qConfig->scale first.
quantTypeToString	char* f(qtype_t t)	Return type name as string (for debug).
conversionMatrix[6][6]	conversionFunction_t table	6×6 table of type conversion functions. Index by [fromType][toType].

Section 14.5 — Arithmetic Module

Function	Signature	Purpose
getDimensionsByIndex	size_t f(tensor_t*, size_t index)	Logical dimension size at axis 'index', accounting for transpose.
orderDims	void f(tensor_t*, size_t* out)	Fill out[] with logical dimension sizes [0..N-1].
doDimensionsMatch	bool f(tensor_t* a, tensor_t* b)	True if all logical dimensions match (accounts for transpose).
calcTensorIndexByIndices	size_t f(size_t N, size_t* dims, size_t* indices)	Row-major flat index from multi-dim indices (no transpose).
calcIndicesByRawIndex	void f(size_t N, size_t* dims, size_t raw, size_t* out)	Reverse: flat raw index → multi-dim logical indices.
calcElementIndexByIndices	size_t f(size_t N, size_t* dims, size_t* idx, size_t* order)	Physical flat index from logical indices, applying orderOfDimensions permutation.
int32PointWiseArithmetic	void f(tensor_t* a, tensor_t* b, fn, tensor_t* out)	Apply binary fn to each pair of elements. Transpose-aware.
floatPointWiseArithmetic	void f(tensor_t* a, tensor_t* b, fn, tensor_t* out)	Float version of point-wise arithmetic.

int32PointWiseArithmeticInplace	void f(tensor_t* a, tensor_t* b, fn)	Inplace: $a[i] = fn(a[i], b[i])$. Faster (no output tensor).
int32ElementWithTensorArithmetic	void f(tensor_t* a, int32_t x, fn, tensor_t* out)	Apply fn to each tensor element and scalar x.

Section 14.6 — Add / Sub / Mul / Div Modules

Function	Signature	Purpose
addInt32s / addFloat32s	int32_t/float f(a, b)	Primitive element operations: return a+b.
addInt32Tensors	void f(tensor_t* a, tensor_t* b, tensor_t* out)	Element-wise INT32 add.
addFloat32TensorsInplace	void f(tensor_t* a, tensor_t* b)	$a[i] += b[i]$ for all i, FLOAT32.
addInt32TensorToSymInt32TensorInplace	void f(tensor_t* sym, tensor_t* int32)	$sym_mantissa[i] += int32[i]$ for all i (mixed-type accumulation).
addFloat32TensorToSymInt32TensorInplace	void f(tensor_t* sym, tensor_t* f32)	$sym[i] += f32[i] / scale$ (converts float grad to SYM_INT32 units).
addSymInt32Tensors / addSymInt32TensorsInplace	void f(tensor_t* a, tensor_t* b, ...)	SYM_INT32 addition with scale matching.
subInt32Tensors / subFloat32Tensors / subSymInt32Tensors	void f(a, b, out)	Element-wise subtraction variants.
mulInt32s / mulFloat32s	int32_t/float f(a, b)	Primitive element multiplication.
mulInt32Tensors / mulFloat32Tensors / mulSymInt32Tensors	void f(a, b, out)	Element-wise tensor multiplication.
divInt32Tensors / divFloat32Tensors	void f(a, b, out)	Element-wise tensor division.
getAddInstructionCounter	size_t f()	Return count of add operations (for profiling).

Section 14.7 — Matmul, MinMax, Sum, Log, Rounding, Comparison

Function	Signature	Purpose
matmulInt32Tensors	void f(a, b, out)	INT32 matrix multiply without bias.
matmulFloat32Tensors	void f(a, b, out)	FLOAT32 matrix multiply without bias.
matmulSymInt32Tensors	void f(a, b, out)	SYM_INT32 matrix multiply without bias.
matmulFloat32TensorsWithBias	void f(a, b, out, bias)	FLOAT32 $Y = X @ W + b$.
matmulSymInt32TensorsWithBias	void f(a, b, out, bias)	SYM_INT32 $Y = X @ W + b$ with scale handling.
getMatmulInstructionCounter	size_t f()	Return MACs counter.
findAbsMaxFloat	float f(uint8_t* data, size_t N)	Max $ x[i] $ over N FLOAT32 elements.
findMaxFloat / findMinFloat	float f(data, N)	Max/min over N FLOAT32 elements.
findMaxInt32 / findMinInt32	int32_t f(data, N)	Max/min over N INT32 elements.
sumint32 / sumFloat	int32_t/float f(values, N)	Sum of N values.
logFloat	float f(float x)	Natural log. Used in cross-entropy loss.
roundByMode	int32_t f(float x, roundingMode_t)	Round float to int32 with HALF_AWAY or SR_HALF_AWAY.
clamp	float f(float x, float min, float max)	Clip x to $[min, max]$.

gteFloatValue	void f(in, float thresh, float belowVal, out)	Output = input where input>=thresh, else belowVal.
gteInt32Value	void f(in, int32_t thresh, int32_t belowVal, out)	INT32 threshold gate.
gteSymInt32Zero	void f(in, out)	SYM_INT32 ReLU: zero mantissas below 0, copy scale.

Section 14.8 — Layer Module

Function	Signature	Purpose
initLayer	void f(layer_t*, layerType_t, layerConfig_t*)	Set layer->type and layer->config.
linearInitConfig	void f(linearConfig_t*, weights, bias, forwardQ, ...)	Fill all linearConfig_t fields.
linearForward / linearForwardFloat / linearForwardSymInt32	void f(layer, in, out)	Forward pass: transpose W, matmul, transpose W back.
linearBackward / backwardFloat / backwardSymInt32	void f(layer, fwdIn, loss, propLoss)	Backward pass: compute weight grads, bias grads, propagated loss.
linearCalcWeightGradsFloat32 / SymInt32	void f(loss, fwdIn, weightGrads)	dL/dW = loss^T @ forwardInput. Accumulates (+=) into weightGrads.
linearCalcBiasGradsFloat32 / SymInt32	void f(biasGrads, loss)	dL/db = sum(loss, axis=batch). Accumulates.
linearCalcPropLossFloat32 / SymInt32	void f(weights, loss, propLoss)	dL/dX = loss @ W (propagated gradient for layer below).
linearCalcOutputShape	void f(layer, inShape, outShape)	outShape[0]=inShape[0]=batch; outShape[1]=weights.dims[0]=out_features.
reluInitConfig / reluInit / reluForward / reluBackward	...	ReLU config + dispatch forward/backward.
reluCalcOutputShape	void f(layer, in, out)	Output shape = input shape (ReLU is identity shape).
softmaxInitConfig / softmaxForward / softmaxBackward	...	Softmax layer forward (exp normalization) and backward.
conv1dForward / conv1dBackward / conv1dCalcOutputShape	...	1D convolution forward and backward.
dropoutForward / dropoutBackward	...	Dropout: random mask during training, identity during eval.
flattenForward / flattenBackward / flattenCalcOutputShape	...	Flatten: reshape metadata without copying data.
initKernel / initKernelExplicit	void f(kernel_t*, size, padding, stride, dilation)	Set up convolution kernel parameters.

Section 14.9 — Training Infrastructure

Function	Signature	Purpose
defaultLossConfig	lossConfig_t f(lossFuncType_t)	Returns {funcType, REDUCTION_MEAN, NULL}. Quick setup.
lossFunctions[MSE].forward	float f(out, label, reduction)	MSE loss: mean((out-label)^2).
lossFunctions[CE].forward	float f(out, label, reduction)	Cross-entropy: -sum(label * log(out)).
lossFunctions[CE].backward	void f(out, label, result)	CE+softmax grad: result = out - label.

lossFunctions[X].computeMeanScale	float f(totalSamples, out)	Scale factor for REDUCTION_MEAN gradient.
sgdInit	void f(sgd_t*, float lr, float mom, float wd)	Initialize SGD with learning rate, momentum, weight decay.
sgdStep	void f(sgd_t*, parameter_t*, quantization_t*)	Update: $w \leftarrow lr * grad$. Dispatches float or SYM_INT32 path.
sgdStepM	void f(sgd_t*, parameter_t*, quantization_t*, state)	SGD+Momentum: $v = m*v + g$; $w \leftarrow lr*v$.
sgdZeroGrad	void f(parameter_t*)	memset grad data to 0. Reset SYM_INT32 scale to 1.0.
initDataLoader	void f(dataLoader_t*, getSample, getSize, getBatch, batchSize, ...)	Set up DataLoader with callbacks and config.
optimizerFunctions[SGD].step / .zeroGrad	stepFn_t / zeroFn_t	Dispatch table for optimizer variants.

Section 14.10 — Support Modules

Function	Signature	Purpose
rngSetSeed	void f(uint32_t seed)	Set xorshift32 state. seed=0 → UINT32_MAX (state=0 is illegal).
rngGetSeed	uint32_t f(void)	Return current RNG state.
rngNextFloat	float f(void)	Uniform float in [0,1). Uses top 24 bits of xorshift32.
rngShuffleIndices	void f(size_t* indices, size_t n)	Fisher-Yates shuffle of indices[0..n-1].
bernoulliSampleTensor	void f(tensor_t* mask, float p)	Fill BOOL mask: each bit = 1 with prob p, 0 with prob 1-p.
serializeTensor	void f(tensor_t*, FILE*)	Write tensor (quantization + shape + data) to byte stream.
serializeParameter	void f(parameter_t*, FILE*)	Serialize both param and grad tensors.
serializeModel	void f(layer_t** model, size_t N, FILE*)	Serialize all N layers' parameters.
deserializeTensor	void f(tensor_t*, FILE*)	Read tensor from byte stream into pre-allocated tensor_t.
deserializeModel	void f(layer_t** model, size_t N, FILE*)	Restore all N layers from byte stream.

Section 14.11 — Quick Lookup: Data Type Sizes

Size (ARM 32-bit)	Value Range	Used For
1 byte	0 to 255	Tensor data buffer, byte arrays
4 bytes	-2,147,483,648 to +2,147,483,647	INT32 and SYM_INT32 mantissas, indices
8 bytes	-9.2e18 to +9.2e18	Matmul accumulator (avoids overflow)
4 bytes	0 to 4,294,967,295	RNG state, bitmasks
4 bytes	~1.2e-38 to ~3.4e38 (7 sig. digits)	FLOAT32 tensors, scales, learning rate
4 bytes (32-bit)	0 to 4,294,967,295	Sizes, indices, counts
1 byte (implementation)	0 or 1	Boolean flags, BOOL tensor elements (in C)
4 bytes (32-bit ARM)	0x00000000 to 0xFFFFFFFF	tensor_t*, function pointers, all references

Section 14.12 — Quick Lookup: All Struct Fields

tensor_t — The central data structure:

Field	Type	Purpose
data	uint8_t*	Raw bytes. Size = calcBytesPerTensor(tensor).

shape	shape_t*	Dimensions and axis permutation.
quantization	quantization_t*	Storage format (type + config).
sparsity	sparsity_t*	Sparse pattern (currently a stub, NULL for dense).

shape_t — Dimension descriptor:

Field	Type	Purpose
numberOfDimensions	size_t	Tensor rank (number of axes).
dimensions	size_t*	Physical size of each dimension slot.
orderOfDimensions	size_t*	Permutation mapping logical → physical axes.

quantization_t — Quantization descriptor:

Field	Type	Purpose
type	qtype_t	INT32, FLOAT32, SYM_INT32, SYM, ASYM, or BOOL.
qConfig	void*	Pointer to type-specific config struct. NULL for INT32/FLOAT32.

symInt32QConfig_t — SYM_INT32 parameters:

Field	Type	Purpose
scale	float	real = mantissa × scale. Updated by requantSymInt32Tensor.
qMaxBits	uint8_t	Mantissa range: $[-2^{(qMaxBits-1)}+1, +2^{(qMaxBits-1)}-1]$.
roundingMode	uint8_t	HALF_AWAY or SR_HALF_AWAY.

Layer Implementations Deep Dive

Every forward and backward pass, line by line

15.1 What Are Neural Network Layers?

A neural network is nothing more than a chain of mathematical operations called **layers**. Each layer takes a tensor in, does some math, and produces a tensor out. The magic is that when you stack many of these layers and tune their internal parameters (weights and biases) using gradient descent, the network learns to map inputs to outputs.

In the ODT library, every layer follows the same interface contract defined in **Layer.h**. This means you can add any new layer type without touching the training loop — you just register three function pointers: forward, backward, and calcOutputShape.

■ C CONCEPT: Forward Pass vs Backward Pass

The **forward pass** computes the layer's output from its input — this is inference. The **backward pass** computes gradients — how much does changing each parameter affect the loss? The backward pass flows in reverse through the network, which is why it is called **backpropagation**.

15.2 Layer.h: The Interface Contract

Every layer is described by a `layer_t` struct. Let's look at each piece:

Layer.h — layer_t struct

```
typedef struct {
    layerType_t type; // Which kind of layer is this?
    layerConfig_t config; // Pointer to the layer's settings
} layer_t;
```

The `layerType_t` is an enum — an integer constant with a name. ODT defines: LINEAR, RELU, SOFTMAX, CONV1D, DROPOUT, FLATTEN, BATCHNORM, EMBEDDING, ATTENTION, POOLING, LOSS.

The `layerConfig_t` is a **union** — a single block of memory that can hold a pointer to *any* of the 11 config structs. A union is like a parking space: only one car (value) at a time, but any car that fits can park there.

layerConfig_t union — one memory slot for all layer config types

```
typedef union {
    linearConfig_t *linear;
    reluConfig_t *relu;
    softmaxConfig_t *softmax;
    conv1dConfig_t *conv1d;
    dropoutConfig_t *dropout;
    flattenConfig_t *flatten;
    batchNormConfig_t *batchNorm;
    embeddingConfig_t *embedding;
    attentionConfig_t *attention;
    poolingConfig_t *pooling;
    lossConfig_t *loss;
} layerConfig_t;
```

■ C CONCEPT: Why Use a Union Here?

All pointers on a 32-bit CPU are 4 bytes. A union that holds any pointer therefore costs exactly 4 bytes — the same as a single pointer. Without the union, you'd need 11 pointer fields (44 bytes) even though only one is ever used at a time. The union is pure memory efficiency.

15.2.1 The Dispatch Table — layerFunctions[]

The dispatch table connects a layer type (an integer) to its three function pointers. It is an array where the index is the layer type.

Layer.h — dispatch table entry and full table

```
typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*);
typedef void (*backwardFn_t)(layer_t*, tensor_t*, tensor_t*,
tensor_t*, tensor_t*);
typedef void (*calcOutputShapeFn_t)(layer_t*, shape_t*, shape_t*);
typedef struct {
forwardFn_t forward;
backwardFn_t backward;
calcOutputShapeFn_t calcOutputShape;
} layerFunctions_t;
extern layerFunctions_t layerFunctions[]; // declared in Layer.h
// In Layer.c — definition using designated initializers:
layerFunctions_t layerFunctions[] = {
[LAYER_LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
[LAYER_RELU] = { reluForward, reluBackward, reluCalcOutputShape },
[LAYER_SOFTMAX] = { softmaxForward, softmaxBackward, softmaxCalcOutputShape },
[LAYER_CONV1D] = { conv1dForward, conv1dBackward, conv1dCalcOutputShape },
[LAYER_DROPOUT] = { dropoutForward, dropoutBackward, dropoutCalcOutputShape },
[LAYER_FLATTEN] = { flattenForward, flattenBackward, flattenCalcOutputShape },
};
```

When the training loop calls `layerFunctions[layer->type].forward(layer, input, output)`, C looks up the function pointer at index `layer->type` and calls it. This is called a **vtable** pattern — it is how object-oriented languages implement virtual methods, but here we do it explicitly with C arrays.

— ■ WORKED EXAMPLE

Calling a layer generically: `layer_t myLayer; // could be linear, relu, anything // The same call works for ANY layer type: layerFunctions[myLayer.type].forward(&myLayer, &inputTensor, &outputTensor); // C resolves the correct function at runtime based on myLayer.type`

15.3 Linear Layer — Full Deep Dive

The Linear (fully connected / dense) layer is the fundamental building block of neural networks. It computes: $\text{output} = \text{input} \times W^T + b$, where W is the weight matrix and b is the bias vector.

Mathematically: if input has shape $[\text{batchSize}, \text{inputFeatures}]$ and W has shape $[\text{outputFeatures}, \text{inputFeatures}]$, then output has shape $[\text{batchSize}, \text{outputFeatures}]$. Each output neuron is a dot product of the input with one row of W .

15.3.1 linearConfig_t — All 7 Fields

Linear.h — linearConfig_t struct

```
typedef struct {  
    parameter_t weights; // {tensor_t param; tensor_t grad;}  
    parameter_t bias; // bias parameter + its gradient  
    bool hasBias; // true = add bias; false = skip bias  
    bool freezeWeights; // true = don't update weights during backward  
    bool freezeBias; // true = don't update bias during backward  
    bool isTraining; // true during training, false during inference  
    tensor_t *inputCache; // stores input for use in backward pass  
} linearConfig_t;
```

weights		A parameter_t holds both the value tensor (.param) and its gradient tensor (.grad). For a linear layer mapping 64→32, weights.param has shape [32,64].
bias		Shape [outputFeatures]. Added to each row of the output.
hasBias		If false, the bias addition is skipped entirely — useful when a BatchNorm layer follows (it would cancel the bias anyway).
freezeWeights/freezeBias		Skip gradient accumulation for these parameters. Used in transfer learning where you want to train only part of the network.
inputCache		Pointer to the forward pass input tensor. The backward pass needs this to compute the weight gradient: $dW = d\text{Output}^T \times \text{input}$.

15.3.2 linearForwardFloat — Complete Walkthrough

Linear.c — linearForwardFloat

```
void linearForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
    linearConfig_t *cfg = layer->config.linear; // Step 1
    cfg->inputCache = input; // Step 2
    // Step 3: transpose input from [batch, in] → [in, batch]
    transposeTensor(input);
    // Step 4: output = weights × inputT + bias
    // weights shape: [out, in]
    // input (transposed) shape: [in, batch]
    // result shape: [out, batch]
    matmulFloat32TensorsWithBias(
        &cfg->weights.param, // A: [out, in]
        input, // B: [in, batch] (transposed)
        cfg->hasBias ? &cfg->bias.param : NULL,
        output // C: [out, batch]
    );
    // Step 5: un-transpose input (restore caller's tensor)
    transposeTensor(input);
    // Step 6: transpose output from [out, batch] → [batch, out]
    transposeTensor(output);
}
```

Step 1		Cast the generic config union to linearConfig_t*. This is safe because we know this function is only called when layer->type == LAYER_LINEAR.
Step 2		Store the input pointer in inputCache. The backward pass will read this later to compute weight gradients. Note: only the pointer is stored, not a copy. The caller must keep the input tensor alive until backward completes.
Step 3		transposeTensor is O(1) — it just swaps orderOfDimensions[0] and orderOfDimensions[1]. Now what was [batch, in] logically looks like [in, batch] when accessed via tensor index functions.
Step 4		matmulFloat32TensorsWithBias computes A×B+bias. The shapes are [out,in] × [in,batch] = [out,batch]. If hasBias is false, NULL is passed and the matmul skips the bias addition.
Step 5		Restore the input tensor to its original orientation. The caller expects the input to be unchanged after the forward pass.
Step 6		The matmul result is [out,batch]. Transpose to [batch,out] so that the next layer receives data in the conventional [batch, features] format.

■ WORKED EXAMPLE

Shape trace through linearForwardFloat: Input: [4, 64] (batch=4, input_features=64) Weights: [32, 64] (output_features=32, input_features=64) Bias: [32] (one bias per output neuron) After transposeTensor(input): logically [64, 4] After matmul([32,64] × [64,4]): result [32, 4] After transposeTensor(output): output is [4, 32] ✓

15.3.3 linearBackward — Computing All Three Gradients

The backward pass receives `dOutput` — the gradient of the loss with respect to this layer's output. It must compute:

1. **dWeights** = $dOutput^T \times input$ (shape: [out, in])
2. **dBias** = sum over batch of dOutput (shape: [out])
3. **dInput** = $dOutput \times weights$ (shape: [batch, in]) — propagates loss upstream

Linear.c — linearBackward (simplified)

```
void linearBackward(layer_t *layer, tensor_t *input,
tensor_t *dOutput, tensor_t *dInput) {
    linearConfig_t *cfg = layer->config.linear;
    if (!cfg->freezeWeights) {
        // dWeights = dOutput^T x inputCache
        // dOutput shape: [batch, out] → transpose → [out, batch]
        // inputCache shape: [batch, in]
        // [out, batch] x [batch, in] = [out, in] ← same as weights shape
        transposeTensor(dOutput);
        matmulFloat32TensorsWithBias(dOutput, cfg->inputCache,
        NULL, &cfg->weights.grad);
        transposeTensor(dOutput); // restore
    }
    if (cfg->hasBias && !cfg->freezeBias) {
        // dBias[j] = sum over batch of dOutput[batch, j]
        for (size_t j = 0; j < biasSize; j++) {
            float sum = 0.0f;
            for (size_t b = 0; b < batchSize; b++)
                sum += dOutput[b * outFeatures + j];
            biasGradPtr[j] += sum;
        }
    }
    if (dInput != NULL) {
        // dInput = dOutput x weights (no transpose on weights)
        // [batch, out] x [out, in] = [batch, in]
        matmulFloat32TensorsWithBias(dOutput, &cfg->weights.param,
        NULL, dInput);
    }
}
```

■ C CONCEPT: Why += for gradient accumulation?

Gradients use `+=` because a single parameter (like a weight) can affect multiple outputs. Each path through the network contributes a partial gradient, and the total gradient is the sum of all contributions. This is the chain rule applied across a batch.

15.3.4 linearCalcOutputShape

Linear.c — linearCalcOutputShape

```
void linearCalcOutputShape(layer_t *layer, shape_t *inputShape,
shape_t *outputShape) {
    linearConfig_t *cfg = layer->config.linear;
    size_t outFeatures = cfg->weights.param.shape.dimensions[0];
    // Output has same batch dimensions, but last dim changes
    *outputShape = *inputShape; // copy all dims
    outputShape->dimensions[inputShape->numberOfDimensions - 1] = outFeatures;
}
```

This function is used before allocating output tensors. By knowing input shape and layer config, the training infrastructure can pre-allocate the correct output buffer.

15.4 ReLU Layer — Complete Analysis

ReLU stands for **Rectified Linear Unit**. It is the most widely used activation function in neural networks. The formula is simple: $\text{ReLU}(x) = \max(0, x)$. Negative values become 0; positive values pass through unchanged.

Why is this useful? Without non-linear activation functions like ReLU, stacking multiple linear layers is mathematically equivalent to a single linear layer — the network cannot learn complex patterns. ReLU introduces non-linearity cheaply.

15.4.1 reluForwardFloat — Line by Line

Relu.c — reluForwardFloat

```
void reluForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
    size_t n = calcNumberOfElementsByShape(&input->shape);
    float *inPtr = (float*) input->data.dataPTR;
    float *outPtr = (float*) output->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        outPtr[i] = getFloatValue(inPtr[i], 0.0f) ? inPtr[i] : 0.0f;
    }
}
```

Line 2	<code>calcNumberOfElementsByShape</code> multiplies all dimensions together. For shape [4, 32], n = 128. We loop over every element.
Line 3-4	Cast the void* data pointer to float*. This lets us use array indexing with [i] instead of byte-level pointer arithmetic.
Line 5-7	For each element: if it is ≥ 0 , copy it. Otherwise write 0. The <code>getFloatValue(a, b)</code> helper returns true if $a \geq b$.

Note that ReLU has no trainable parameters — `reluConfig_t` exists only to hold a pointer to the input tensor cached for the backward pass.

15.4.2 reluForwardSymInt32 — Quantized ReLU

Relu.c — reluForwardSymInt32

```
void reluForwardSymInt32(layer_t *layer, tensor_t *input, tensor_t *output) {
    size_t n = calcNumberOfElementsByShape(&input->shape);
    int32_t *inPtr = (int32_t*) input->data.dataPTR;
    int32_t *outPtr = (int32_t*) output->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        outPtr[i] = getSymInt32Zero(inPtr[i]) ? inPtr[i] : 0;
    }
    // Copy scale from input to output (ReLU doesn't change scale)
    symInt32QConfig_t *inQ = input->quantization.qConfig;
    symInt32QConfig_t *outQ = output->quantization.qConfig;
    outQ->scale = inQ->scale;
    outQ->qMaxBits = inQ->qMaxBits;
}
```

getSymInt32Zero	For SYM_INT32, zero is represented as the integer 0 (because the quantization is symmetric). So this is just checking if the integer mantissa ≥ 0 , which is equivalent to checking if the real value ≥ 0 .
Scale copy	ReLU does not change the magnitude of positive values, so the output has the same quantization scale as the input. This is important — if we forgot to copy the scale, the next layer would decode the tensor wrong.

■ C CONCEPT: Symmetric Quantization and Zero

In SYM_INT32: $\text{real_value} = \text{mantissa} \times \text{scale}$. Zero mantissa $\rightarrow$ zero real value. Checking $\text{mantissa} \geq 0$ is exactly the same as checking $\text{real_value} \geq 0$, because scale is always positive. This is why symmetric quantization is convenient for ReLU.

15.4.3 reluBackward

Relu.c — reluBackward

```
void reluBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
    size_t n = calcNumberOfElementsByShape(&dOutput->shape);
    float *dOutPtr = (float*) dOutput->data.dataPTR;
    float *dInPtr = (float*) dInput->data.dataPTR;
    float *cachedIn = (float*) layer->config.relu->inputCache->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        // If the forward input was > 0, gradient passes through
        // If the forward input was <= 0, gradient is blocked (= 0)
        dInPtr[i] = (cachedIn[i] > 0.0f) ? dOutPtr[i] : 0.0f;
    }
}
```

The ReLU backward pass implements the derivative of $\max(0, x)$: it equals 1 when $x > 0$ and 0 otherwise. By multiplying the upstream gradient by this derivative, gradients only flow back through neurons that were 'on' (positive) during the forward pass. Neurons that were 'off' (negative) get zero gradient — they are 'dead' for this sample.

■■ IMPORTANT

The Dying ReLU Problem: If a neuron always receives negative input, its gradient is always 0, so it never gets updated. This is called a 'dead ReLU'. Solutions include: careful weight initialization, lower learning rates, or using LeakyReLU which allows a small negative slope instead of zero.

15.5 Softmax Layer — Probability Output

Softmax converts a vector of arbitrary real numbers (called **logits**) into a probability distribution — all values are in $[0,1]$ and they sum to 1. It is used as the last layer in classification networks.

The formula: $\text{softmax}(x)[i] = \exp(x[i]) / \sum(\exp(x[j]))$ for all j

■ C CONCEPT: Numerical Stability Trick

Computing $\exp(x)$ directly can overflow for large x values (e.g., $\exp(1000) = \text{inf}$). The trick: subtract the maximum value first. Since $\text{softmax}(x) = \text{softmax}(x - \max(x))$, this shifts all values to ≤ 0 before exponentiating, preventing overflow.

Softmax.c — softmaxForward (simplified)

```
void softmaxForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    size_t batch = input->shape.dimensions[0];
    size_t n = input->shape.dimensions[1]; // number of classes
    float *in = (float*) input->data.dataPTR;
    float *out = (float*) output->data.dataPTR;
    for (size_t b = 0; b < batch; b++) {
        float *row_in = in + b * n;
        float *row_out = out + b * n;
        // Step 1: find max for numerical stability
        float maxVal = row_in[0];
        for (size_t i = 1; i < n; i++)
            if (row_in[i] > maxVal) maxVal = row_in[i];
        // Step 2: compute exp(x - max) and sum
        float sum = 0.0f;
        for (size_t i = 0; i < n; i++) {
            row_out[i] = expf(row_in[i] - maxVal);
            sum += row_out[i];
        }
        // Step 3: divide by sum to normalize
        for (size_t i = 0; i < n; i++)
            row_out[i] /= sum;
    }
}
```

b * n		Pointer arithmetic to jump to the start of batch row b. If each row has n elements, batch b starts at offset b*n from the beginning of the array.
expf		C standard library function for float exponentiation. Use expf (not exp) for float — exp works on double, which is slower and unnecessary on an MCU.
Dividing by sum		This ensures the outputs sum to exactly 1.0, making them valid probabilities.

15.5.1 Why Softmax and Cross-Entropy Are Usually Combined

If you compute softmax backward separately and cross-entropy backward separately, you get a numerically unstable and complicated gradient. But if you combine them, the gradient simplifies beautifully: $d(\text{CE after softmax})/d(\text{logits}) = \text{pred} - \text{target}$.

This is why ODT implements `crossEntropySoftmaxBackward` as a single function that assumes softmax was applied just before cross-entropy. The combined backward is both simpler and more numerically stable.

15.6 Conv1D Layer — 1D Convolution for Sequences

1D convolution processes sequential data — like audio waveforms, time series, or sequences of feature vectors. It slides a small kernel (filter) across the input and computes dot products at each position.

For speaker identification on LibriSpeech audio, Conv1D extracts local temporal patterns — things like the shape of phonemes or short acoustic events that characterize a speaker's voice.

15.6.1 conv1dConfig_t — Every Field Explained

Conv1d.h — conv1dConfig_t

```
typedef struct {
    parameter_t weights; // shape: [outChannels, inChannels, kernelSize]
    parameter_t bias; // shape: [outChannels]
    bool hasBias;
    size_t inChannels; // input feature dimension
    size_t outChannels; // output feature dimension
    kernel_t kernel; // size, padding, stride, dilation
    bool isTraining;
    tensor_t *inputCache;
} conv1dConfig_t;

typedef struct {
    size_t size; // kernel size (e.g. 3, 5, 7)
    paddingType_t paddingType; // VALID, SAME, or EXPLICIT
    size_t stride; // step between kernel positions
    size_t dilation; // spacing between kernel elements
    size_t padding; // explicit padding amount (if EXPLICIT)
} kernel_t;
```

kernel Size		How many input time steps each output value looks at. A kernel of size 3 means each output is computed from 3 consecutive inputs.
stride		How many steps to advance the kernel each time. stride=1 gives maximum overlap (most detailed). stride=2 halves the output length.
dilation		Gaps between kernel elements. dilation=2 means the kernel samples at positions [0, 2, 4] instead of [0, 1, 2]. This gives a larger receptive field without more parameters.
paddingType= SAME		Adds zeros at the edges so the output length equals the input length (assuming stride=1).
paddingType= VALID		No padding. Output length = $\text{floor}((L-K)/\text{stride})+1$ where L is input length and K is kernel size.

15.6.2 Conv1D Mathematical Formula

For each output channel c_{out} , output position t , and batch b :

Conv1D formula

```
output[b, c_out, t] = bias[c_out]
+ sum over c_in, k of:
weights[c_out, c_in, k] * input[b, c_in, t*stride + k*dilation]
```

—■ WORKED EXAMPLE

Conv1D with kernel=3, stride=1, no padding: input: [1, 1, 5] (batch=1, channels=1, length=5): [1, 2, 3, 4, 5]
kernel: [1, 1, 3] (1 out_channel, 1 in_channel, size=3): [0.5, 1.0, 0.5] output[t=0] = $1 \cdot 0.5 + 2 \cdot 1.0 + 3 \cdot 0.5 = 0.5 + 2.0 + 1.5 = 4.0$ output[t=1] = $2 \cdot 0.5 + 3 \cdot 1.0 + 4 \cdot 0.5 = 1.0 + 3.0 + 2.0 = 6.0$ output[t=2] = $3 \cdot 0.5 + 4 \cdot 1.0 + 5 \cdot 0.5 = 1.5 + 4.0 + 2.5 = 8.0$ Output shape: [1, 1, 3] (length reduced from 5 to 3 by VALID padding)

15.7 Dropout Layer — Regularization

Dropout randomly sets a fraction p of the neurons to zero during training. This prevents co-adaptation: neurons cannot rely on specific other neurons, so they must learn more robust features. During inference, dropout is disabled.

ODT uses **inverted dropout**: during training, surviving neurons are scaled by $1/(1-p)$ so that the expected value of the output is the same as without dropout. This means no adjustment is needed at inference time.

15.7.1 dropoutConfig_t Fields

Dropout.h — dropoutConfig_t

```
typedef struct {
    float p; // dropout probability (fraction to zero out)
    bool training; // true = apply dropout; false = pass-through
    tensor_t *mask; // boolean mask (1=keep, 0=drop), saved for backward
    quantization_t *forwardQ; // quantization for mask tensor
    quantization_t *backwardQ;
    bool ownsQuantizations; // true = we allocated forwardQ/backwardQ
} dropoutConfig_t;
```

15.7.2 dropoutForward — Step by Step

Dropout.c — dropoutForward

```
void dropoutForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    dropoutConfig_t *cfg = layer->config.dropout;
    size_t n = calcNumberOfElementsByShape(&input->shape);
    float *in = (float*) input->data.dataPTR;
    float *out = (float*) output->data.dataPTR;
    if (!cfg->training) {
        // Inference: copy input to output unchanged
        memcpy(out, in, n * sizeof(float));
        return;
    }
    // Training: generate random binary mask
    bernoulliSampleTensor(cfg->mask, 1.0f - cfg->p);
    // 1-p = probability of keeping a neuron
    float scale = 1.0f / (1.0f - cfg->p); // inverted dropout scale
    for (size_t i = 0; i < n; i++) {
        bool keep = tensorBoolGet(cfg->mask, i);
        out[i] = keep ? in[i] * scale : 0.0f;
    }
}
```

bernoulliSampleTensor		Fills cfg->mask with random bits: each bit is 1 with probability (1-p) and 0 with probability p. Uses the RNG module.
scale = 1/(1-p)		For p=0.5: scale=2.0. Surviving neurons are doubled, so their expected contribution equals the full (no-dropout) case.
tensorBoolGet		Reads a single bit from the packed boolean mask tensor. The mask uses 1 bit per element, so a 512-element mask costs only 64 bytes.

```
out[
i] =
0.0f
```

Dropped neurons output exactly zero. During backward, zeros also block gradient flow through dropped neurons.

15.7.3 dropoutBackward

Dropout.c — dropoutBackward

```
void dropoutBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
    dropoutConfig_t *cfg = layer->config.dropout;
    size_t n = calcNumberOfElementsByShape(&dOutput->shape);
    float *dOut = (float*) dOutput->data.dataPTR;
    float *dIn = (float*) dInput->data.dataPTR;
    float scale = 1.0f / (1.0f - cfg->p);
    for (size_t i = 0; i < n; i++) {
        bool kept = tensorBoolGet(cfg->mask, i);
        dIn[i] = kept ? dOut[i] * scale : 0.0f;
    }
}
```

The backward pass uses the SAME mask as the forward pass (saved in `cfg->mask`). This ensures that gradients only flow through neurons that were active during the forward pass. The scale factor is applied again for consistency.

15.8 Flatten Layer — Zero-Copy Reshape

Flatten converts a multi-dimensional tensor into a 1D vector (per batch item). For example, after a Conv1D layer with output [batch, channels, length], Flatten produces [batch, channels × length]. This is needed before a Linear layer which expects 2D input.

The key insight in ODT's implementation: **no data is copied**. The underlying memory is identical — we just change the shape metadata.

Flatten.c — flattenForward

```
void flattenForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    // Share the data buffer – no copy needed
    output->data.dataPTR = input->data.dataPTR;
    output->data.numberOfElements = input->data.numberOfElements;
    // Compute flat size: product of all dims except batch
    size_t flatSize = 1;
    for (size_t d = 1; d < input->shape.numberOfDimensions; d++)
        flatSize *= input->shape.dimensions[d];
    // Set output shape: [batchSize, flatSize]
    size_t batchSize = input->shape.dimensions[0];
    output->shape.numberOfDimensions = 2;
    output->shape.dimensions[0] = batchSize;
    output->shape.dimensions[1] = flatSize;
    output->shape.orderOfDimensions[0] = 0;
    output->shape.orderOfDimensions[1] = 1;
}
```

output->data.dataPTR = input->data.dataPTR

Both tensors point to the SAME memory. A write through output->data changes what input->data sees too. This is safe here because flatten's output is only read, not written.

flatSize

Multiply dimensions [1], [2], ... together. For Conv1D output [4, 16, 8] (batch=4, channels=16, length=8): flatSize = 16×8 = 128.

orderOfDimensions = {0, 1}

Reset to natural order after flattening. The 2D output is not transposed.

— WORKED EXAMPLE

Flatten shape transformation: Input shape: [4, 16, 8] (batch=4, 16 channels, length=8) Output shape: [4, 128] (batch=4, flat=16×8=128) Data in memory: unchanged — both tensors read the same bytes Memory saved: 0 bytes copied, 0 extra allocations

■ NOTE

Flatten and Memory Safety: Because flatten shares memory, the caller must not free the input tensor's data buffer while the output tensor is still in use. The training loop manages this by keeping all layer activations alive until the backward pass completes.

15.9 Layer Summary Table

uct	Has Params?	Forward Cost	Key Feature
g_t	Yes (W, b)	$O(\text{in} \times \text{out} \times \text{batch})$	Fully connected
t	No	$O(n)$	Element-wise, l
nfig_t	No	$O(\text{classes} \times \text{batch})$	Normalizes to p
nfig_t	Yes (W, b)	$O(\text{out} \times \text{in} \times K \times L \times \text{batch})$	Local pattern e
nfig_t	No	$O(n) + \text{RNG}$	Regularization,
g_t	No	$O(1)$	Zero-copy resh

■ RELATIONS WITH OTHER FILES

Chapter 15 connects to: Chapter 6 (Arithmetic) — `matmulFloat32TensorsWithBias`, `gteFloatValue` called by layers · Chapter 5 (Tensor) — `transposeTensor`, `calcNumberOfElementsByShape` used everywhere · Chapter 8 (Training) — `layerFunctions` dispatch table called by training loop · Chapter 9 (RNG) — `bernoulliSampleTensor` used by dropout · Chapter 11 (MCU) — All layers execute on-device, data flows through SRAM buffers

Chapter 16

Mathematical Foundations

The math behind every operation in the ODT library

16.1 Linear Algebra Essentials

Neural networks are, at their core, linear algebra. Every forward pass is a sequence of matrix multiplications and element-wise operations. Understanding the shapes and operations on matrices is essential to understanding the code.

16.1.1 Vectors and Matrices

A **vector** is a list of numbers: $v = [v_1, v_2, \dots, v_n]$. A **matrix** is a 2D grid of numbers with rows and columns. An $m \times n$ matrix has m rows and n columns.

Matrix notation example

```
W = [[1, 2, 3], // row 0
     [4, 5, 6]] // row 1
// W is a 2x3 matrix: 2 rows, 3 columns
// W[i][j] means row i, column j
// W[1][2] = 6
```

In C, a 2D matrix is typically stored as a flat 1D array in **row-major order**: all elements of row 0 come first, then row 1, etc. Element $W[i][j]$ is at index $i \times \text{numCols} + j$.

WORKED EXAMPLE

Row-major memory layout: Matrix W (2 rows, 3 cols) stored in memory: Index: 0 1 2 3 4 5 Value: 1 2 3 4 5 6
↑row0 ↑row1 ↑ $W[1][2] = W_{\text{flat}}[1 \times 3 + 2] = W_{\text{flat}}[5] = 6$ ✓

16.1.2 Matrix Multiplication

If A has shape $[m, k]$ and B has shape $[k, n]$, then $C = A \times B$ has shape $[m, n]$. Each element $C[i][j] = \text{sum over } t \text{ of } A[i][t] \times B[t][j]$. The inner dimension k must match — this is the 'contracted' dimension.

Matrix multiply formula

```
// C[i][j] = sum_{t=0}^{k-1} A[i][t] * B[t][j]
//
// Example: A is [2,3], B is [3,2], C is [2,2]
A = [[1,2,3], [4,5,6]] // shape [2,3]
B = [[7,8], [9,10], [11,12]] // shape [3,2]
C[0][0] = 1*7 + 2*9 + 3*11 = 7+18+33 = 58
C[0][1] = 1*8 + 2*10 + 3*12 = 8+20+36 = 64
C[1][0] = 4*7 + 5*9 + 6*11 = 28+45+66 = 139
C[1][1] = 4*8 + 5*10 + 6*12 = 32+50+72 = 154
```

In ODT, `matmulFloat32TensorsWithBias` computes exactly this, with an optional bias added to each row of the result.

16.1.3 Matrix Transpose

The transpose of A (written A^T) swaps rows and columns: $A^T[i][j] = A[j][i]$. If A has shape $[m, n]$, then A^T has shape $[n, m]$.

In ODT, `transposeTensor` is $O(1)$ — it does not move any data. It swaps the `orderOfDimensions` array, so that when the tensor is accessed logically, dimensions are looked up in swapped order.

WORKED EXAMPLE

Why transpose is needed in Linear forward pass: We want: $\text{output} = \text{input} \times W^T$ (so each row of W is a set of weights for one output neuron) input shape: $[\text{batch}=4, \text{in}=64]$ W shape: $[\text{out}=32, \text{in}=64]$ W^T shape: $[\text{in}=64, \text{out}=32]$ $\text{input} \times W^T = [4,64] \times [64,32] = [4,32]$ ← correct output shape But ODT's `matmul` expects row $\times$ column = scalar, so instead we transpose input and compute $W \times \text{input}^T = [32,64] \times [64,4] = [32,4]$, then transpose result to $[4,32]$

16.2 Gradient Descent and Backpropagation

The goal of training is to find weight values that minimize the loss function. We do this by computing the gradient (direction of steepest increase) and moving the weights in the *opposite* direction.

16.2.1 What Is a Gradient?

For a function $f(x)$, the derivative $f'(x)$ tells us: 'if I increase x by a tiny amount ϵ , f increases by approximately $f'(x) \times \epsilon$ '. The **gradient** is the multidimensional generalization: ∇f is a vector where each component is the partial derivative with respect to one variable.

Gradient intuition

```
// f(w) = w^2 → f'(w) = 2w
// At w=3: f'(3) = 6
// Meaning: if w increases by 0.01, f increases by ~0.06
// To DECREASE f, move w in direction -f'(w) = -6
// Update: w_new = w - lr * f'(w) = 3 - 0.1*6 = 2.4
// Now f(2.4) = 5.76 < f(3) = 9 ✓ loss went down
```

16.2.2 Chain Rule — The Engine of Backpropagation

Neural networks are composed functions: $\text{loss} = L(\text{softmax}(\text{linear}(\text{relu}(\text{linear}(x))))))$. To find how much the loss changes when we change a weight deep in the network, we use the **chain rule**: if $z = f(g(x))$, then $dz/dx = (dz/dg) \times (dg/dx)$.

In a network with layers $A \rightarrow B \rightarrow C \rightarrow \text{loss}$:

Chain rule in backprop

```
// Forward: x → A(x) → B(A(x)) → C(B(A(x))) → loss
// Backward (chain rule):
// dL/dA = dL/dC × dC/dB × dB/dA
//
// In code, each layer's backward receives dOutput = dL/d(this layer's output)
// and must compute dInput = dL/d(this layer's input) = dOutput × d(output)/d(input)
// This dInput becomes dOutput for the previous layer.
```

This is exactly what ODT implements: the training loop calls backward on each layer from last to first, passing the gradient signal backwards through the network.

16.2.3 Stochastic Gradient Descent (SGD)

Pure gradient descent computes gradients over the entire dataset. **Stochastic Gradient Descent (SGD)** computes gradients over small random batches (mini-batches). This is noisier but much faster per update.

SGD update rule

```
// For each parameter w:
// w_new = w - learning_rate * gradient(w)
//
// With momentum (helps escape local minima):
// velocity = momentum * velocity - learning_rate * gradient
// w_new = w + velocity
//
// With weight decay (L2 regularization):
// w_new = w - learning_rate * (gradient + weight_decay * w)
```

ODT's `sgdStepFloat` implements the basic version: $w \leftarrow w - \text{lr} \times \text{grad}$. The `sgd_t` struct stores `momentumFactor` and `weightDecay` fields for optional use in extended implementations.

16.3 Loss Functions — Measuring Error

A loss function measures how wrong the network's predictions are. It takes the network's output and the true target, and returns a single scalar number. Lower loss = better predictions.

16.3.1 Mean Squared Error (MSE)

MSE = $(1/n) \times \sum((\text{predicted} - \text{target})^2)$. Used for regression tasks (predicting a continuous value). The squared term penalizes large errors more heavily.

MSE example

```
// predictions: [2.3, 0.5, 1.8]
// targets: [2.0, 0.0, 2.0]
// errors: [0.3, 0.5, -0.2]
// squared: [0.09, 0.25, 0.04]
// MSE = (0.09 + 0.25 + 0.04) / 3 = 0.127
//
// MSE gradient w.r.t. predictions: 2/n * (pred - target)
// = 2/3 * [0.3, 0.5, -0.2] = [0.2, 0.333, -0.133]
```

16.3.2 Cross-Entropy Loss

Cross-entropy is used for classification. If the network outputs probabilities (after softmax), cross-entropy is: $CE = -\sum(\text{target} \times \log(\text{prediction}))$.

For one-hot targets (only one class is correct), this simplifies to $-\log(\text{probability of the correct class})$. Perfect prediction $\rightarrow$ loss = 0. Wrong prediction $\rightarrow$ loss = infinity.

Cross-entropy example — 3-class classification

```
// targets (one-hot): [0, 1, 0] ← class 1 is correct
// predictions (softmax output): [0.1, 0.7, 0.2]
// CE = -(0*log(0.1) + 1*log(0.7) + 0*log(0.2))
// = -log(0.7)
// = 0.357 (low loss – prediction was fairly good)
//
// If predictions were [0.4, 0.2, 0.4] (wrong):
// CE = -log(0.2) = 1.609 (high loss)
```

The backward gradient through softmax+cross-entropy combined is simply: $dL/d(\text{logits}) = \text{predictions} - \text{targets}$. This is what `crossEntropySoftmaxBackward` computes.

— ■ WORKED EXAMPLE

Cross-entropy backward example: predictions: [0.1, 0.7, 0.2] targets: [0, 1, 0] gradient: [0.1-0, 0.7-1, 0.2-0] = [0.1, -0.3, 0.2] Interpretation: the network should increase logit for class 1 (negative gradient means: decreasing this value decreases loss) and decrease logits for classes 0 and 2.

16.4 Fixed-Point Quantization — The Mathematics

Floating-point numbers (float32) are convenient but expensive on MCUs. Quantization maps floating-point values to integers, enabling faster arithmetic with less memory. FQT (Fixed-point Quantization Training) in ODT trains with quantized weights from the start.

16.4.1 SYM_INT32 Encoding

In ODT's SYM_INT32 format: $\text{real_value} = \text{mantissa} \times \text{scale}$, where mantissa is an int32_t and scale is a float. The range of representable values is approximately $[-q_{\text{Max}} \times \text{scale}, +q_{\text{Max}} \times \text{scale}]$ where $q_{\text{Max}} = 2^{(q_{\text{MaxBits}}-1)} - 1$.

SYM_INT32 quantization formulas

```
// To quantize a float to SYM_INT32:
mantissa = clamp(round(real_value / scale), -qMax, +qMax)
// To dequantize back to float:
real_value = mantissa * scale
// Example: scale=0.01, qMaxBits=8, qMax=127
// Quantize 0.735: mantissa = round(0.735/0.01) = round(73.5) = 74
// Dequantize: 74 * 0.01 = 0.74 (small rounding error)
```

16.4.2 Choosing Scale — Two-Pass Requantization

To minimize quantization error, we want the scale to be as small as possible while still representing the maximum value in the tensor. The optimal scale is: $\text{scale} = \text{absMax} / q_{\text{Max}}$.

ODT's `requantSymInt32Tensor` does this in two passes:

Two-pass requantization algorithm

```
// Pass A: find absolute maximum value in the tensor
float absMax = 0.0f;
for each element e in tensor:
float val = e.mantissa * e.scale; // dequantize
if (fabsf(val) > absMax) absMax = fabsf(val);
// Compute new scale to use the full integer range
float newScale = absMax / qMax;
// Pass B: re-encode every element with the new scale
for each element e in tensor:
float realVal = e.mantissa * e.scale; // old representation
e.mantissa = clamp(round(realVal / newScale), -qMax, qMax);
e.scale = newScale; // update scale (same for whole tensor)
```

■ NOTE

Scale is Per-Tensor, Not Per-Element: In ODT's SYM_INT32, a single scale value is shared across all elements of a tensor. This is called per-tensor quantization and is the simplest scheme. Per-channel quantization (one scale per output channel) can give better accuracy but is more complex to implement.

16.4.3 Integer Arithmetic for Matmul

When multiplying two SYM_INT32 tensors, the result has a combined scale: $\text{scale}_C = \text{scale}_A \times \text{scale}_B$.

SYM_INT32 matmul math

```
// A[i] = mantissa_A[i] * scale_A
// B[j] = mantissa_B[j] * scale_B
// C[i][j] = sum_k(A[i][k] * B[k][j])
// = sum_k(mantissa_A[i][k] * scale_A * mantissa_B[k][j] * scale_B)
// = (sum_k(mantissa_A[i][k] * mantissa_B[k][j])) * scale_A * scale_B
// = int64_accumulator * (scale_A * scale_B)
// Why int64_t? Because int32_t * int32_t can overflow int32_t!
// int32 max = ~2.1 billion
// (2.1e9)^2 = 4.4e18, which overflows int32 but fits in int64
```

This is why `matmulSymInt32TensorsWithBias` uses `int64_t acc = 0` as the accumulator before converting back to `int32_t` with the new combined scale.

16.4.4 Quantization Noise and Stochastic Rounding

Every quantization step introduces a small error (the **quantization noise**). Regular rounding (round-half-away) is deterministic: the same values always round the same way. Over many training steps, this can cause a systematic bias.

Stochastic rounding (SR_HALF_AWAY in ODT) adds random noise before rounding: **$\text{mantissa} = \text{round}(\text{real}/\text{scale} + U(-0.5, 0.5))$** . This makes rounding errors random rather than systematic, which helps training converge better with low-bit quantization.

16.5 Sparse Training Mathematics (RigL)

Sparse neural networks have many weights set to exactly zero. A network that is 90% sparse (90% zeros) uses 10× less memory and can compute 10× faster if the zeros are exploited.

16.5.1 Magnitude Pruning

The simplest pruning criterion: remove the weights with the smallest absolute value. The intuition: small weights contribute little to the output and can be set to zero without much loss in accuracy.

Magnitude pruning math

```
// Given weights W and target sparsity ratio s (e.g., 0.9 for 90% sparse):
// 1. Sort |W[i]| in ascending order
// 2. Set to zero the smallest floor(n*s) weights
// 3. Keep the remaining ceil(n*(1-s)) weights
// Example: W = [0.5, -0.1, 0.8, -0.03, 0.2], s=0.4 (prune 40%)
// |W| sorted: [0.03, 0.1, 0.2, 0.5, 0.8]
// floor(5*0.4) = 2 → prune 2 smallest: 0.03 and 0.1
// Result mask: W[-0.1]=0, W[-0.03]=0, others remain
```

16.5.2 Gradient-Based Regrowth (RigL)

Magnitude pruning only prunes. RigL (Rigging the Lottery by Evci et al., 2020) also **grows** new connections where the gradient signal is largest. The criterion for growing: weights with the largest |gradient|.

The gradient of a pruned (zero) weight is meaningful — it tells us 'if this weight were non-zero, how much would it help?' Computing this gradient requires a special backward pass that also propagates through masked weights.

RigL update algorithm

```
// At step T (e.g., every 100 batches):
//
// 1. PRUNE: find k weights with smallest |magnitude|
// mask[i] = 0 for these k weights
// W[i] = 0 (explicitly zero)
//
// 2. GROW: find k positions with largest |gradient|
// (among currently masked/pruned positions)
// mask[i] = 1 (activate these weights)
// W[i] = 0 (start at zero, will grow during training)
//
// 3. Continue normal training with the new mask
// Active weights (mask=1): updated by SGD
// Pruned weights (mask=0): skipped by SGD
```

■ RigL GAP IN THIS FILE

RigL Gap in ODT: ODT has the foundation (sparsity_t struct stub, BOOL tensor type, RNG for random init) but lacks the rigUpdate() function and the mask-aware matmul and SGD. See Chapter 10 for the complete implementation checklist.

16.5.3 Why Sparse Training Is Hard on Embedded Systems

On a CPU or GPU, sparse operations use special hardware instructions or libraries. On the STM32 MCU, there is no hardware sparse support. The options are:

1. **Dense storage with mask**: Keep W in full [out, in] memory but multiply by the mask during forward/backward. No memory savings, but computation savings if you check the mask before each multiply (skip if mask=0).
2. **Compressed Sparse Row (CSR)**: Store only non-zero values and their indices. Memory-efficient but adds index lookup overhead and more complex C code.

ODT currently targets option 1 (dense + mask) as the simpler starting point. The mask tensor uses 1 bit per weight via `tensorBoolSet/Get`, costing only $n/8$ bytes for an n -element weight matrix.

16.6 Why This Math Applies to Speaker Identification

The LibriSpeech speaker identification task requires the network to map audio features to speaker identities. Let's trace the math through one inference:

Full forward pass — mathematical view

```
// Input: x = audio feature vector, shape [1, 64] (MFCC features)
// Layer 1 (Linear 64→128):
//  $h_1 = x \times W_1^T + b_1$ 
// shape:  $[1, 64] \times [64, 128] + [128] = [1, 128]$ 
// Layer 2 (ReLU):
//  $h_2 = \max(0, h_1)$  element-wise
// shape: [1, 128]
// Layer 3 (Linear 128→num_speakers):
//  $\text{logits} = h_2 \times W_2^T + b_2$ 
// shape:  $[1, 128] \times [128, \text{num\_speakers}] + [\text{num\_speakers}] = [1, \text{num\_speakers}]$ 
// Layer 4 (Softmax):
//  $\text{probs} = \text{softmax}(\text{logits})$ 
//  $\text{probs}[k] = \exp(\text{logits}[k]) / \sum(\exp(\text{logits}[:]))$ 
// shape: [1, num_speakers], all values in (0,1), sum=1
// Prediction: speaker = argmax(probs)
```

Cross-entropy loss during training: $\text{CE} = -\log(\text{probs}[\text{true_speaker}])$. The network learns to assign high probability to the correct speaker for each audio segment.

After training is complete on the MCU, inference is just the forward pass without any gradient computation — fast and deterministic.

16.7 Numerical Concepts Reference

Concept	Value / Formula	Where Used in ODT
int32_t max	$2,147,483,647 \approx 2.1 \times 10^9$	SYM_INT32 mantissa range
int64_t max	9.2×10^{18}	matmulSymInt32 accumulator
float32 precision	~7 decimal digits	SGD, loss computation
float32 range	$\pm 3.4 \times 10^{38}$	Intermediate activations
exp overflow	$\exp(89) \approx \text{float max}$	Softmax uses max subtraction trick
qMax (8-bit SYM)	$2^7 - 1 = 127$	SYM_INT32 with qMaxBits=8
qMax (16-bit SYM)	$2^{15} - 1 = 32767$	SYM_INT32 with qMaxBits=16
UINT32_MAX	$4,294,967,295 = 2^{32} - 1$	xorshift32 state, rngBounded limit
2^{24} float normalization	16,777,216	rngNextFloat: $x > 8$ then $/2^{24} \rightarrow [0,1)$

■ RELATIONS WITH OTHER FILES

Chapter 16 connects to: Chapter 4 (Quantization) — SYM_INT32 encoding and scale computation · Chapter 6 (Arithmetic) — matmul accumulator in int64_t, roundByMode · Chapter 8 (Training) — SGD update rule, loss functions forward and backward · Chapter 10 (RigL gap) — Pruning and regrowth math that needs to be implemented · Chapter 11 (MCU) — Why integer math is preferred on Cortex-M7

Chapter 17

End-to-End Worked Example

Building, training, and running a 2-layer MLP — step by step with real numbers

17.1 The Task: 3-Class Speaker Identification

To make the example concrete, we will build a tiny network that classifies audio feature vectors into one of 3 speaker identities (A, B, C). This is a simplified version of the full LibriSpeech task.

Network architecture

Input: 4 features (simplified MFCC coefficients)

Linear 1: 4 → 8 neurons, with bias

ReLU 1: element-wise activation

Linear 2: 8 → 3 neurons (3 speaker classes), with bias

Softmax: normalize to probabilities

Loss: CrossEntropy

Parameter count: Linear1 weights = $8 \times 4 = 32$, bias = 8. Linear2 weights = $3 \times 8 = 24$, bias = 3. Total: 67 parameters. This is tiny — the full model for LibriSpeech would have thousands.

17.2 Step 1 — Memory Layout on the MCU

Before writing any code, plan where each tensor lives in SRAM. The STM32F746ZG has 320 KB RAM. For our tiny example:

	Type	Bytes	Purpose
	float32	128	Linear1 weights
	float32	128	Linear1 weight gradients
	float32	32	Linear1 bias
	float32	32	Linear1 bias gradients
	float32	96	Linear2 weights
	float32	96	Linear2 weight gradients
	float32	12	Linear2 bias
	float32	12	Linear2 bias gradients
	float32	16	Input feature vector
	float32	32	After Linear1
	float32	32	After ReLU1
	float32	12	After Linear2 (logits)
	float32	12	After Softmax
	float32	32	Gradient at h2
	float32	32	Gradient at h1
	float32	12	One-hot target
		716 B	Well within 320 KB

17.3 Step 2 — Initialization Code

All tensors and layer configs must be initialized before training begins. Here is the complete initialization sequence:

[illegible]

static		All arrays and structs are static — they live in .bss/.data sections in flash/SRAM, not on the stack. This avoids stack overflow for large buffers.
rng Set Seed(&rng, 42)		Initialize the xorshift32 RNG with seed 42 for reproducibility.
Xavier init		A well-known initialization scheme that scales initial weights to prevent activations from vanishing or exploding. Scale = $\sqrt{6/(\text{fan_in} + \text{fan_out})}$.

17.4 Step 3 — One Training Step with Real Numbers

Let's trace a complete training step with specific numbers. We use a single sample (batch_size=1) for clarity.

Initial state (after random init, simplified):

Initial weights and input

```
Input (MFCC features): x = [0.5, -0.3, 0.8, 0.1]
W1 (8x4) - showing first 3 rows for brevity:
[0.2, 0.5, -0.3, 0.1] // row 0 → neuron h1[0]
[-0.4, 0.1, 0.6, -0.2] // row 1 → neuron h1[1]
[0.3, -0.2, 0.1, 0.4] // row 2 → neuron h1[2]
... (5 more rows)
b1 (8): [0.1, -0.1, 0.05, 0.0, 0.1, -0.05, 0.02, 0.0]
Target (speaker A = class 0, one-hot): [1, 0, 0]
```

17.4.1 Forward Pass — Layer by Layer

Linear1 forward: $h1 = W1 \times x + b1$

```
h1[0] = 0.2*0.5 + 0.5*(-0.3) + (-0.3)*0.8 + 0.1*0.1 + 0.1
= 0.10 - 0.15 - 0.24 + 0.01 + 0.10
= -0.18
h1[1] = (-0.4)*0.5 + 0.1*(-0.3) + 0.6*0.8 + (-0.2)*0.1 + (-0.1)
= -0.20 - 0.03 + 0.48 - 0.02 - 0.10
= 0.13
h1[2] = 0.3*0.5 + (-0.2)*(-0.3) + 0.1*0.8 + 0.4*0.1 + 0.05
= 0.15 + 0.06 + 0.08 + 0.04 + 0.05
= 0.38
// (complete h1 for all 8 neurons in practice)
h1 = [-0.18, 0.13, 0.38, 0.05, -0.22, 0.31, -0.07, 0.44]
```

ReLU1 forward: $h2 = \max(0, h1)$

```
h2[0] = max(0, -0.18) = 0.00 ← neuron is OFF (dead for this sample)
h2[1] = max(0, 0.13) = 0.13
h2[2] = max(0, 0.38) = 0.38
h2[3] = max(0, 0.05) = 0.05
h2[4] = max(0, -0.22) = 0.00 ← neuron is OFF
h2[5] = max(0, 0.31) = 0.31
h2[6] = max(0, -0.07) = 0.00 ← neuron is OFF
h2[7] = max(0, 0.44) = 0.44
h2 = [0.00, 0.13, 0.38, 0.05, 0.00, 0.31, 0.00, 0.44]
```

Linear2 forward: $\text{logits} = W2 \times h2 + b2$

```
// W2 shape [3,8], h2 shape [8] → logits shape [3]
// Assume W2 and b2 initialized similarly to W1
logits[0] = W2[0,:] · h2 + b2[0] = 0.25 // speaker A score
logits[1] = W2[1,:] · h2 + b2[1] = 0.40 // speaker B score
logits[2] = W2[2,:] · h2 + b2[2] = 0.10 // speaker C score
logits = [0.25, 0.40, 0.10]
```

Softmax forward: $\text{probs} = \text{softmax}(\text{logits})$

```
maxVal = 0.40 (for numerical stability)
shifted = [0.25-0.40, 0.40-0.40, 0.10-0.40]
= [-0.15, 0.00, -0.30]
exp_vals = [exp(-0.15), exp(0.00), exp(-0.30)]
= [0.861, 1.000, 0.741]
sum = 0.861 + 1.000 + 0.741 = 2.602
probs = [0.861/2.602, 1.000/2.602, 0.741/2.602]
= [0.331, 0.384, 0.285]
// Network assigns 33.1% to speaker A, 38.4% to B, 28.5% to C
// True class is A (0) — not great prediction, but we just started training
```

CrossEntropy loss

target = [1, 0, 0] (speaker A)

CE = $-(1 \cdot \log(0.331) + 0 \cdot \log(0.384) + 0 \cdot \log(0.285))$

= $-\log(0.331)$

= 1.107

// High loss – the network is not confident about the correct speaker

17.4.2 Backward Pass with Real Numbers

CrossEntropy+Softmax backward: d_logits = probs - target

```
d_logits = [0.331 - 1, 0.384 - 0, 0.285 - 0]
= [-0.669, 0.384, 0.285]
// Meaning: logits[0] should increase (negative gradient → decrease loss)
// logits[1] and [2] should decrease
```

Linear2 backward: dW2, db2, dh2

```
// dW2[i][j] = d_logits[i] * h2[j]
dW2[0][:] = -0.669 * [0.00, 0.13, 0.38, 0.05, 0.00, 0.31, 0.00, 0.44]
= [0.00, -0.087, -0.254, -0.033, 0.00, -0.207, 0.00, -0.294]
dW2[1][:] = 0.384 * h2 = [0.00, 0.050, 0.146, 0.019, 0.00, 0.119, 0.00, 0.169]
dW2[2][:] = 0.285 * h2 = [0.00, 0.037, 0.108, 0.014, 0.00, 0.088, 0.00, 0.125]
db2 = d_logits = [-0.669, 0.384, 0.285]
// dh2 = W2^T * d_logits (propagate loss upstream)
dh2[j] = sum_i(W2[i][j] * d_logits[i]) for j=0..7
// This gives the gradient at each h2 neuron
```

ReLU1 backward: dh1 = dh2 * (h1 > 0)

```
// Only neurons that were ON in forward get gradient
// h1 = [-0.18, 0.13, 0.38, 0.05, -0.22, 0.31, -0.07, 0.44]
// mask = [0, 1, 1, 1, 0, 1, 0, 1]
dh1[0] = dh2[0] * 0 = 0.0 ← h1[0] was negative, gradient blocked
dh1[1] = dh2[1] * 1 = dh2[1] ← passes through
dh1[2] = dh2[2] * 1 = dh2[2]
// etc.
```

Linear1 backward: dW1, db1

```
// dW1[i][j] = dh1[i] * x[j] (input was x = [0.5, -0.3, 0.8, 0.1])
// db1[i] = dh1[i]
// (dInput computed but not used – x is the network input, no upstream layer)
```

17.4.3 SGD Weight Update

SGD update (learning_rate = 0.01)

```
// For each parameter w with gradient dw:
w_new = w - 0.01 * dw
// Example – W1 row 0 first element:
// W1[0][0]_old = 0.20
// dW1[0][0] = dh1[0] * x[0] = 0.0 * 0.5 = 0.0 (neuron 0 was OFF!)
// W1[0][0]_new = 0.20 - 0.01 * 0.0 = 0.20 (no change)
// W1[1][0] (neuron 1 was ON):
// W1[1][0]_old = -0.40
// dW1[1][0] = dh1[1] * x[0] = dh2[1] * 0.5
// W1[1][0]_new = -0.40 - 0.01 * (dh2[1] * 0.5)
// After one step, loss will be slightly lower.
// After many steps (hundreds or thousands), network converges.
```

17.5 Step 4 — The Training Loop in C

Now let's write the actual C training loop that ties everything together:

main.c — training loop

```
void trainEpoch(dataLoader_t *loader, sgd_t *sgd,
layer_t *lin1, layer_t *relu1,
layer_t *lin2, lossFunc_t *lossFn,
tensor_t *input, tensor_t *h1, tensor_t *h2,
tensor_t *output, tensor_t *dh2, tensor_t *dh1,
tensor_t *target) {
    size_t numBatches = loader->getDatasetSize(loader) / loader->batchSize;
    for (size_t batch = 0; batch < numBatches; batch++) {
        // 1. Zero all gradients
        sgdZeroGrad(sgd, lin1->config.linear);
        sgdZeroGrad(sgd, lin2->config.linear);
        // 2. Load one batch
        loader->getBatch(loader, batch, input, target);
        // 3. Forward pass
        layerFunctions[LAYER_LINEAR].forward(lin1, input, h1);
        layerFunctions[LAYER_RELU].forward(relu1, h1, h2);
        layerFunctions[LAYER_LINEAR].forward(lin2, h2, output);
        // 4. Compute loss (includes softmax inside)
        float loss = lossFn->forward(lossFn, output, target);
        // 5. Backward pass (reverse order)
        lossFn->backward(lossFn, output, target, dh2);
        layerFunctions[LAYER_LINEAR].backward(lin2, h2, dh2, dh1);
        layerFunctions[LAYER_RELU].backward(relu1, dh1, dh1); // in-place
        layerFunctions[LAYER_LINEAR].backward(lin1, input, dh1, NULL);
        // 6. Update weights
        sgdStep(sgd, lin1->config.linear);
        sgdStep(sgd, lin2->config.linear);
        if (batch % 10 == 0)
            printf('Batch %zu loss: %.4f\n', batch, loss);
    }
}
```

sgdZeroGrad		CRITICAL: must be called at the start of every batch. Otherwise gradients from previous batches accumulate, corrupting the update.
getBatch		Fills the input and target tensors with data from the dataset. In the DataLoader, this calls getSample in a loop to fill the batch.
NULL for dlinput in lin1		The first layer has no upstream layer to send gradients to, so we pass NULL. linearBackward checks for NULL before computing dInput.
printf on MCU		On the STM32, printf can be redirected to UART for debugging. This is usually done by overriding the _write() system call to use HAL_UART_Transmit.

17.6 Step 5 — Serialization and Inference

After training, the weights need to be saved to flash so they survive power cycles, or transmitted to a host PC for analysis.

Saving trained weights with `serializeTensor`

```
// Write trained weights to UART (connected to PC)
// PC-side Python script reads and saves as .npy
void saveModel(layer_t *lin1, layer_t *lin2) {
    serializeTensor(&lin1->config.linear->weights.param);
    serializeTensor(&lin1->config.linear->bias.param);
    serializeTensor(&lin2->config.linear->weights.param);
    serializeTensor(&lin2->config.linear->bias.param);
}
// For inference (no gradients needed):
void runInference(tensor_t *x, tensor_t *probs) {
    lin1cfg.isTraining = false; // disable inputCache saving
    layerFunctions[LAYER_LINEAR].forward(&lin1, x, &h1);
    layerFunctions[LAYER_RELU].forward(&relu1, &h1, &h2);
    layerFunctions[LAYER_LINEAR].forward(&lin2, &h2, &outBuf_t);
    softmaxForward(NULL, &outBuf_t, probs);
    // probs now holds probabilities for each speaker class
}
```

17.7 Common Mistakes and How to Avoid Them

Mistake	Symptom	Fix
Forgot <code>sgdZeroGrad</code>	Loss oscillates, gradients grow	Call <code>sgdZeroGrad</code> before each batch
Wrong layer order in backward	Loss doesn't decrease	Backward must be exact reverse of forward
<code>transposeTensor</code> not in pairs	Wrong matmul shapes, garbage output	Every <code>transposeTensor</code> must be undone
Scale not copied in ReLU	Next layer decodes wrong values	Copy scale from input to output quantization
<code>inputCache</code> is NULL in backward	Segfault / wrong gradients	Check <code>isTraining=true</code> and forward called first
VLA too large	Stack overflow, MCU crashes	Use static buffers for large tensors
<code>float/int32</code> mix without casting	Compiler warning, wrong results	Explicit cast: <code>(float)(int32_val)</code>
<code>log(0)</code> in cross-entropy	NaN loss	Guard: <code>max(pred, EPSILON)</code> before log

■ RELATIONS WITH OTHER FILES

Chapter 17 connects to: Chapters 7-8 (Layers + Training) — All layer and SGD functions used in the loop · Chapter 6 (Arithmetic) — Matmul and add operations underneath each layer · Chapter 9 (Support) — RNG for weight init, serialization for saving weights · Chapter 11 (MCU) — Static buffer strategy, UART printf, STM32 startup · Chapter 16 (Math) — Xavier init formula, cross-entropy gradient derivation

Chapter 18

Embedded Systems Deep Dive

STM32, ARM Cortex-M7, bare-metal C, and how the MCU executes your code

18.1 What Is an Embedded System?

An **embedded system** is a computer designed to perform a specific task, built into a larger device. Unlike a laptop or desktop, it typically has no operating system (or a very minimal one), limited resources, and must run reliably for years without human intervention.

The STM32 Nucleo-F746ZG is a development board containing an **STM32F746ZG microcontroller** — a single chip with CPU, RAM, flash, and peripherals all integrated together. Your C code runs directly on the metal.

Component	Specification	Relevance to ODT
CPU	ARM Cortex-M7 @ 216 MHz	Executes all C code
RAM (SRAM)	320 KB total	All tensors, stack, globals
Flash	1 MB	Code + constant data
FPU	Double-precision FPU	Float operations without software emulation
DSP	DSP instructions	SIMD-style operations (not used in ODT)
UART	Multiple UARTs	Serial communication, printf debugging
Timers	Multiple 16/32-bit	Training loop timing
DMA	DMA controller	Memory-to-memory transfers
I-Cache/D-Cache	16 KB instruction, 16 KB data	Speeds up repeated code/data access
Bus width	32-bit AHB/APB	All memory accesses are 32-bit aligned

18.2 ARM Cortex-M7 Architecture

The ARM Cortex-M7 is a 32-bit RISC (Reduced Instruction Set Computer) processor. Understanding its architecture helps explain why ODT makes certain design choices.

18.2.1 Registers

The CPU has 16 general-purpose 32-bit registers (R0–R15) plus status registers. All computation happens in registers — data must be loaded from memory into a register before it can be used, then stored back.

ARM Cortex-M7 key registers

R0–R3: Function arguments and return values
R4–R11: Callee-saved registers (function must preserve these)
R12: Intra-procedure-call scratch register
R13 (SP): Stack Pointer – points to top of current stack frame
R14 (LR): Link Register – return address for function calls
R15 (PC): Program Counter – address of next instruction to execute
xPSR: Status register – zero flag, negative flag, carry flag, etc.
FPU registers (for float operations):
S0–S31: 32 single-precision float registers (32-bit each)
D0–D15: 16 double-precision float registers (64-bit, alias S pairs)
FPCR: FPU status/control register

When the compiler generates code for `float a = b * c`, it emits VLDR instructions to load b and c into S0 and S1, then VMUL.F32 to multiply them and store the result in another S register, then VSTR to save back to RAM.

18.2.2 The Pipeline

The Cortex-M7 has a 6-stage pipeline: Fetch → Decode → Issue → Execute → Write-back → Retire. While one instruction is executing, the next is being decoded, and the one after that is being fetched. This overlap is called pipelining.

Pipeline hazards occur when an instruction depends on the result of the previous one. The CPU must stall (wait) for the first result. This is why tight loops with data dependencies (like accumulating a sum) can be slower than expected.

— ■ WORKED EXAMPLE

Pipeline in matrix multiply inner loop: `acc += A[i][k] * B[k][j]`
Cycle 1: Load A[i][k] into S0
Cycle 2: Load B[k][j] into S1, (waiting for S0 to be ready)
Cycle 3: VMUL.F32 S2, S0, S1 (multiply)
Cycle 4: VADD.F32 S3, S3, S2 (accumulate — must wait for VMUL result)
Cycle 5: Increment k, check loop condition
Total: ~5 cycles per inner loop iteration at 216 MHz → ~43M inner iterations/second

18.2.3 Memory Architecture

The Cortex-M7 uses a Harvard architecture variant with separate instruction and data buses. This means it can fetch the next instruction while loading/storing data for the current instruction simultaneously.

STM32F746ZG memory map (simplified)

0x00000000 - 0x000FFFFF: Flash (1 MB) – code + const data
0x20000000 - 0x2004FFFF: DTCM SRAM (320 KB) – fastest RAM
0x40000000 - 0x5FFFFFFF: Peripheral registers (UART, timers, GPIO, etc.)
0xE0000000 - 0xFFFFFFFF: System/debug registers
Stack grows DOWN from top of SRAM (0x2004FFFF → 0x20000000)
.bss + .data grow UP from bottom of SRAM (0x20000000 → ...)

18.3 Memory Sections in Detail

Your compiled C program is divided into named memory sections. Understanding these is critical for embedded programming.

18.3.1 The .text Section (Flash)

All compiled C functions and constant data go into the .text section, stored in flash memory. Flash is read-only at runtime (you can't write to it during normal execution). It survives power cycles.

What lives in .text

```
// All function code:
void matmulFloat32TensorsWithBias(...) { ... } // compiled to ARM opcodes in flash
void linearForwardFloat(...) { ... }
// String literals:
const char *name = 'ODT'; // 'ODT\0' stored in flash
// Constant arrays:
const float lut[256] = { 0.1, 0.2, ... }; // lookup table in flash
```

18.3.2 The .data Section (RAM — Initialized Globals)

Global and static variables that have initial values go into .data. The initial values are stored in flash and copied to RAM at startup.

.data example

```
float learningRate = 0.01f; // in .data — initial value 0.01
int epochCount = 0; // in .bss — zero initial value
// At startup, the MCU's reset handler copies the flash image
// of .data into RAM. This is why your globals have their initial values.
```

18.3.3 The .bss Section (RAM — Zero-Initialized)

Variables declared without an explicit initial value default to zero. These go in .bss (Block Started by Symbol). At startup, the reset handler calls `memset(0)` over the entire .bss region. This is cheap in flash because we only store the size, not 320KB of zeros.

.bss example — ODT static buffers

```
// All of these go in .bss — zero at startup, no flash storage needed
static float w1Data[8*4]; // 128 bytes in .bss
static float g1Data[8*4]; // 128 bytes in .bss
static tensor_t w1; // tensor metadata in .bss
static linearConfig_t lin1cfg; // layer config in .bss
// Cost in flash: ~4 bytes to record the size
// Cost in RAM: 32*4 = 128 bytes at runtime
```

18.3.4 The Stack

Every function call creates a **stack frame** — a region of memory for local variables and saved registers. The stack grows downward from the top of SRAM. Stack frames are automatically created on function entry and destroyed on return.

Stack frame example

```
void myFunction(int x) {  
    float localArray[64]; // 256 bytes on stack (VLA style if runtime-sized)  
    int count = 0; // 4 bytes on stack  
    // ...  
} // stack frame is automatically freed here  
// Total stack depth example for ODT training loop:  
main() → 64 bytes  
trainEpoch() → 32 bytes  
layerFunctions[].forward() → 32 bytes  
matmulFloat32TensorsWithBias() → 128 bytes  
matmulIntCore() → 48 bytes  
// Total stack depth: ~304 bytes  
// STM32 has 320KB RAM – stack and static buffers must not overlap
```

■■ IMPORTANT

Stack Overflow on MCUs: On a desktop OS, stack overflow triggers a segfault. On the STM32, stack overflow silently corrupts other data (it overwrites .bss or .data). This causes bizarre behavior that is very hard to debug. Always calculate maximum stack depth for embedded code.

18.4 Startup Sequence — Before main()

Most developers assume C programs start at main(). On embedded systems, a lot happens before main() is called.

STM32 startup sequence

```
// 1. CPU resets: PC = address stored in vector table at 0x00000004
// (this is the Reset_Handler address)
// 2. Reset_Handler (in startup_stm32f746zg.s, written in assembly):
Reset_Handler:
// Set stack pointer to top of SRAM
LDR R0, =_estack
MSR MSP, R0
// Copy .data section from flash to RAM
LDR R0, =_sdata // destination in RAM
LDR R1, =_edata // end of .data in RAM
LDR R2, =_sidata // source in flash
CopyLoop:
LDR R3, [R2], #4
STR R3, [R0], #4
CMP R0, R1
BLT CopyLoop
// Zero out .bss section
LDR R0, =_sbss
LDR R1, =_ebss
MOV R2, #0
ZeroLoop:
STR R2, [R0], #4
CMP R0, R1
BLT ZeroLoop
// 3. Call SystemInit() – configure PLL, flash wait states
// 4. Call main()
BL main
```

This is why your static float w1Data[] array is automatically zeroed — the startup code clears .bss before main() runs.

18.5 Clock Configuration — Running at 216 MHz

The STM32F746ZG has an internal 16 MHz RC oscillator as the default clock. To reach 216 MHz, it uses a Phase-Locked Loop (PLL) which multiplies the input frequency.

SystemClock_Config (STM32CubeMX generated)

```
void SystemClock_Config(void) {
RCC_OscInitTypeDef osc = {0};
RCC_ClkInitTypeDef clk = {0};
// Enable HSE (external crystal, usually 8 or 25 MHz)
osc.OscillatorType = RCC_OSCILLATORTYPE_HSE;
osc.HSEState = RCC_HSE_BYPASS;
osc.PLL.PLLState = RCC_PLL_ON;
osc.PLL.PLLSource = RCC_PLLSOURCE_HSE;
// PLL multiplier: 8 MHz * (N/M) / P = 216 MHz
osc.PLL.PLLM = 4; // divide by 4 → 2 MHz
osc.PLL.PLLN = 216; // multiply by 216 → 432 MHz
osc.PLL.PLLP = 2; // divide by 2 → 216 MHz
osc.PLL.PLLQ = 9; // USB clock = 432/9 = 48 MHz
HAL_RCC_OscConfig(&osc);
// Set flash wait states (needed at high clock speeds)
__HAL_FLASH_SET_LATENCY(FLASH_LATENCY_7);
clk.ClockType = RCC_CLOCKTYPE_SYSCLK | ...
clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
clk.AHBCLKDivider = RCC_SYSCLK_DIV1; // AHB = 216 MHz
clk.APB1CLKDivider = RCC_HCLK_DIV4; // APB1 = 54 MHz
clk.APB2CLKDivider = RCC_HCLK_DIV2; // APB2 = 108 MHz
HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_7);
}
```

HAL

Hardware Abstraction Layer — STM32's C driver library. HAL functions have names like HAL_RCC_OscConfig, HAL_UART_Transmit. They wrap the raw register writes into readable function calls.

18.6 UART — Debugging and Data Transfer

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication protocol. On the Nucleo board, one UART is connected to the ST-Link debugger chip, which bridges to USB. This creates a virtual COM port on the PC.

This is how ODT's `serializeTensor` sends weight data to the PC, and how `printf` outputs debugging information.

UART setup and use

```
static UART_HandleTypeDef huart3; // UART3 connected to ST-Link on Nucleo
void MX_USART3_UART_Init(void) {
    huart3.Instance = USART3;
    huart3.Init.BaudRate = 115200; // 115200 bits per second
    huart3.Init.WordLength = UART_WORDLENGTH_8B;
    huart3.Init.StopBits = UART_STOPBITS_1;
    huart3.Init.Parity = UART_PARITY_NONE;
    huart3.Init.Mode = UART_MODE_TX_RX;
    HAL_UART_Init(&huart3);
}
// Send data:
uint8_t data[] = {0x01, 0x02, 0xFF};
HAL_UART_Transmit(&huart3, data, 3, HAL_MAX_DELAY);
// 115200 baud → ~11520 bytes/second
// Sending 128-byte weight matrix: ~11 ms
// Redirect printf to UART:
int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart3, (uint8_t*)ptr, len, HAL_MAX_DELAY);
    return len;
}
// Now printf works over UART!
```

115
200
baud

Baud rate = bits per second. With 8 data bits + 1 stop bit = 9 bits per byte. At 115200 baud: $115200/9 \approx 12800$ bytes/second.

HAL
_M
AX_
DEL
AY

Wait until the transmission is complete. On an MCU without an OS, blocking is usually fine for debugging.

18.7 Interrupts and the NVIC

An **interrupt** is a hardware signal that pauses the CPU's current task and calls a special function (the **Interrupt Service Routine**, or ISR). Common interrupt sources: UART receive, timer overflow, GPIO edge.

ODT's training loop is designed to run in the main loop (not in an ISR). But the data loader could use a UART receive interrupt to gather audio samples while computation runs.

Timer interrupt example — measure training loop time

```
volatile uint32_t timerTicks = 0; // volatile! Modified by ISR
// ISR — called every 1 ms by Timer 6
void TIM6_DAC_IRQHandler(void) {
    if (__HAL_TIM_GET_FLAG(&htim6, TIM_FLAG_UPDATE)) {
        __HAL_TIM_CLEAR_FLAG(&htim6, TIM_FLAG_UPDATE);
        timerTicks++;
    }
}
// In main loop:
uint32_t start = timerTicks;
layerFunctions[LAYER_LINEAR].forward(&lin1, &input, &h1);
uint32_t elapsed = timerTicks - start;
printf('Linear forward: %u ms\n', elapsed);
```

■ C CONCEPT: The volatile Keyword

When a variable can be modified by an interrupt (or any external event), it must be declared volatile. Without volatile, the compiler may cache the variable in a register and never re-read it from RAM, missing updates made by the ISR. Always use volatile for ISR-shared variables.

18.8 The FPU — Why Float Is Fast on Cortex-M7

Older Cortex-M0 and M0+ processors have no hardware FPU — float operations are emulated in software, taking ~100 cycles each. The Cortex-M7 has a double-precision FPU that handles float operations in 1-3 cycles.

FPU enabling (in system_stm32f7xx.c)

```
// Enable FPU – must be done before any float operations
#define FPU_CPACR (*((volatile uint32_t*)0xE000ED88))
void FPU_Enable(void) {
    FPU_CPACR |= (0xF << 20); // CP10 and CP11 = full access
    __DSB(); // Data Synchronization Barrier
    __ISB(); // Instruction Synchronization Barrier
}
// With FPU enabled:
// float a = b * c; → VMUL.F32 S0, S1, S2 (1-3 cycles)
// Without FPU (software emulation):
// float a = b * c; → BL __aeabi_fmul (~80-100 cycles)
```

The FPU is why ODT can run full float32 neural networks on the STM32. Without it, even a small matmul would be prohibitively slow.

18.9 Instruction and Data Cache

The Cortex-M7 has separate 16 KB instruction cache (I-cache) and 16 KB data cache (D-cache). Flash has 7-cycle wait states at 216 MHz, so without the I-cache, instruction fetches would take 7× longer.

The D-cache is especially important for ODT: the matmul inner loop repeatedly reads the same rows of the weight matrix. With D-cache, these reads come from fast cache (1-4 cycles) rather than slow SRAM (3-4 cycles, but still faster than flash at 216 MHz).

Enabling caches (in main.c)

```
SCB_EnableICache(); // Enable instruction cache
SCB_EnableDCache(); // Enable data cache
// These are usually the first two lines in main()
// Typical speedup: 2-4x for instruction-heavy code
```

■■ IMPORTANT

Cache Coherency: If you use DMA (Direct Memory Access) to transfer data without going through the CPU, the D-cache can become stale. You must explicitly invalidate or clean the cache after DMA transfers. ODT uses memcpy (CPU-driven), so cache coherency is not an issue in the current implementation.

18.10 Linking and the Linker Script

The linker script (STM32F746ZGTx_FLASH.ld) tells the linker where to place each section in the memory map. This is what ensures your code goes to flash and your buffers go to SRAM.

Linker script excerpt (STM32F746ZGTx_FLASH.ld)

```
MEMORY {
RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 320K
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K
}
SECTIONS {
/* Code and read-only data go to flash */
.text : {
KEEP(*(._isr_vector)) /* Vector table must be first */
*(.text) /* All .text sections */
*(.rodata) /* Constant data */
} > FLASH
/* Initialized globals: stored in flash, copied to RAM at startup */
.data : {
_sdata = .;
*(.data)
_edata = .;
} > RAM AT > FLASH
/* Zero-initialized globals: only in RAM */
.bss : {
_sbss = .;
*(.bss)
*(COMMON)
_ebss = .;
} > RAM
/* Stack at top of RAM */
_estack = ORIGIN(RAM) + LENGTH(RAM);
}
```

AT > FL AS H		The .data section lives in RAM at runtime (> RAM) but is stored in flash during programming (AT > FLASH). The startup code copies it.
KEE P*(. isr_ vect or))		The vector table (interrupt handler addresses) must be at address 0x08000000 (start of flash). KEEP prevents the linker from removing it even if it looks unused.
_sd ata, _ed ata, _sb ss, _eb ss, _est ack		These symbols are used by the Reset_Handler in startup assembly to know where to copy .data and zero .bss.

18.11 Summary: Why Embedded C Is Different

Feature	Desktop C	Embedded C (STM32)
Memory	Virtual memory, GBs available	Physical only, 320 KB fixed
OS	Linux/Windows manages resources	Bare-metal, you manage everything
malloc/free	Generally safe	Avoid — fragmentation causes crashes
printf	Writes to terminal	Must redirect to UART
Stack overflow	Segfault + OS recovery	Silent corruption, very hard to debug
Interrupts	Rarely need to think about	Must use volatile, avoid race conditions
Float operations	Always hardware-accelerated	Need to enable FPU explicitly
Startup	OS handles initialization	Reset_Handler copies .data, zeros .bss
Debugging	gdb, IDE, print statements	SWD/JTAG probe, UART printf, blinking LED
Power	Not a concern	May need low-power modes for battery life

■ RELATIONS WITH OTHER FILES

Chapter 18 connects to: Chapter 11 (MCU chapter) — Static buffers, startup sequence, SRAM layout · Chapter 13 (C language) — volatile, static, memory sections in C · Chapter 17 (Worked example) — printf via UART, static buffers for tensors · Chapter 9 (Support/RNG) — rng32_t using DTCM SRAM for fast access · Chapter 6 (Arithmetic) — FPU acceleration for matmul float operations

Chapter 19

Debugging, Testing, and Extending ODT

How to find bugs, validate correctness, and add new layers or features

19.1 Debugging Embedded C — General Strategy

Debugging embedded code is harder than debugging desktop code because you cannot easily inspect memory or stop the program without special hardware. The main tools are: UART printing, LED blinking, hardware debugger (SWD/JTAG), and the ODT-specific PRINT_DEBUG/PRINT_ERROR macros.

19.1.1 PRINT_DEBUG and PRINT_ERROR

ODT's `Common.h` defines conditional print macros. When `#define DEBUG` is set in the build, they expand to printf. In release builds (no DEBUG), they expand to nothing — zero overhead.

Common.h — debug macros

```
#ifndef DEBUG
#define PRINT_DEBUG(fmt, ...) printf('[DEBUG] ' fmt, ##__VA_ARGS__)
#define PRINT_ERROR(fmt, ...) printf('[ERROR] ' fmt, ##__VA_ARGS__)
#else
#define PRINT_DEBUG(fmt, ...) do { } while(0)
#define PRINT_ERROR(fmt, ...) do { } while(0)
#endif
// Usage in code:
PRINT_DEBUG('matmul: A[%zu,%zu] B[%zu,%zu]\n',
A->shape.dimensions[0], A->shape.dimensions[1],
B->shape.dimensions[0], B->shape.dimensions[1]);
```

**#if D
EB
UG**

The compiler checks if DEBUG was defined. If using GCC: 'gcc -DDEBUG file.c' defines it. In STM32CubeIDE: add DEBUG to Project Properties → C/C++ Build → Settings → Preprocessor → Defined symbols.

**##_
_VA
_AR
GS_
—**

The ## before __VA_ARGS__ suppresses the trailing comma if no variadic arguments are passed — allows PRINT_DEBUG('message') without ','.

19.1.2 Adding Debug Prints to Trace a Bug

When a layer produces wrong output, add debug prints to trace the data flow:

Debug tracing example

```
// Add to linearForwardFloat to trace the matmul inputs:
void linearForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
    linearConfig_t *cfg = layer->config.linear;
    PRINT_DEBUG('Linear forward: input[%zu,%zu] weights[%zu,%zu]\n',
input->shape.dimensions[0], input->shape.dimensions[1],
cfg->weights.param.shape.dimensions[0],
cfg->weights.param.shape.dimensions[1]);
    float *inPtr = (float*) input->data.dataPTR;
    PRINT_DEBUG(' input[0] = %.4f, input[1] = %.4f\n',
inPtr[0], inPtr[1]);
    // ... rest of function
    float *outPtr = (float*) output->data.dataPTR;
    PRINT_DEBUG(' output[0] = %.4f\n', outPtr[0]);
}
```

Common causes of wrong output: wrong shape in setShape (e.g., rows and cols swapped), data pointer pointing to wrong buffer, scale not initialized (stays 0 → all values 0), or transposeTensor called odd number of times.

19.2 The TRACK_INSTRUCTIONS Macro

ODT includes a special debug mode for counting arithmetic operations. When `#define TRACK_INSTRUCTIONS` is active, the code counts multiplications and additions per forward pass.

Arithmetic.c — TRACK_INSTRUCTIONS usage

```
#ifdef TRACK_INSTRUCTIONS
static size_t g_numMul = 0;
static size_t g_numAdd = 0;
#endif
// In matmul inner loop:
for (size_t k = 0; k < K; k++) {
    acc += (int64_t)A_val * B_val;
    #ifdef TRACK_INSTRUCTIONS
    g_numMul++;
    g_numAdd++;
    #endif
}
// After forward pass:
#ifdef TRACK_INSTRUCTIONS
PRINT_DEBUG('Operations: %zu mul, %zu add\n', g_numMul, g_numAdd);
#endif
```

This is used to estimate how many sparse operations RigL would save (if mask zeros are skipped, that many multiplications are avoided).

19.3 Hardware Debugger — SWD and GDB

The Nucleo board includes an ST-Link debugger. It connects to the STM32's SWD (Serial Wire Debug) pins and lets you use GDB to inspect the MCU's state while it is running.

GDB commands for embedded debugging

```
# Connect to target (in STM32CubeIDE or arm-none-eabi-gdb):
(gdb) target remote localhost:3333 # openocd bridge
(gdb) monitor reset halt
# Set breakpoints:
(gdb) break linearForwardFloat
(gdb) break Matmul.c:45
# Inspect variables:
(gdb) print input->shape.dimensions[0]
(gdb) print *(float*)input->data.dataPTR @ 8 # print 8 floats
# Step through code:
(gdb) next # step over (don't enter functions)
(gdb) step # step into (enter function calls)
(gdb) continue # run until next breakpoint
# Inspect memory:
(gdb) x/32f w1Data # print 32 floats starting at w1Data
(gdb) x/8xb &cfg # print 8 bytes in hex at cfg address
```

19.4 Testing ODT Code — Unit Tests on Desktop

A key advantage of ODT's design: the C code is standard C99 with no MCU-specific headers in the core library. This means you can compile and test it on a desktop Linux/Mac/Windows machine using gcc.

19.4.1 Writing a Unit Test for matmul

test_matmul.c — unit test

```
#include <stdio.h>
#include <assert.h>
#include <math.h>
#include 'Tensor.h'
#include 'Matmul.h'

void test_matmulFloat_2x3_times_3x2(void) {
    // A = [[1,2,3],[4,5,6]] shape [2,3]
    // B = [[7,8],[9,10],[11,12]] shape [3,2]
    // Expected C = A*B = [[58,64],[139,154]] shape [2,2]
    float aData[] = {1,2,3, 4,5,6};
    float bData[] = {7,8, 9,10, 11,12};
    float cData[4] = {0};
    tensor_t A, B, C;
    setShape(&A.shape, 2, (size_t[]){2,3});
    setShape(&B.shape, 2, (size_t[]){3,2});
    setShape(&C.shape, 2, (size_t[]){2,2});
    A.data.dataPTR = aData; A.data.numberOfElements = 6;
    B.data.dataPTR = bData; B.data.numberOfElements = 6;
    C.data.dataPTR = cData; C.data.numberOfElements = 4;
    matmulFloat32TensorsWithBias(&A, &B, NULL, &C);
    float *result = (float*) C.data.dataPTR;
    assert(fabsf(result[0] - 58.0f) < 0.001f);
    assert(fabsf(result[1] - 64.0f) < 0.001f);
    assert(fabsf(result[2] - 139.0f) < 0.001f);
    assert(fabsf(result[3] - 154.0f) < 0.001f);
    printf('test_matmulFloat_2x3_times_3x2: PASS\n');
}

int main(void) {
    test_matmulFloat_2x3_times_3x2();
    return 0;
}
```

Compiling and running on desktop

```
# Compile:
gcc -o test_matmul test_matmul.c Matmul.c Tensor.c Arithmetic.c DTypes.c -lm
# Run:
./test_matmul
# Output: test_matmulFloat_2x3_times_3x2: PASS
```

Testing on desktop first is much faster than flashing to the MCU for every change. Only deploy to the MCU when the logic is proven correct on desktop.

19.4.2 Comparing MCU Output to Python

A powerful validation technique: run the same computation in Python (with NumPy) and compare results with the MCU output.

Python reference computation (run on PC)

```
import numpy as np
# Same weights as MCU (loaded from serialized output)
W1 = np.array([[0.2, 0.5, -0.3, 0.1],
               [-0.4, 0.1, 0.6, -0.2]], dtype=np.float32)
b1 = np.array([0.1, -0.1], dtype=np.float32)
x = np.array([0.5, -0.3, 0.8, 0.1], dtype=np.float32)
# Forward pass in NumPy
h1_py = x @ W1.T + b1
h2_py = np.maximum(0, h1_py) # ReLU
print('h1:', h1_py)
print('h2:', h2_py)
# Compare with MCU UART output
# If they match (within float32 precision), implementation is correct
```

x @
W1.
T

NumPy matrix multiply with transpose. Equivalent to ODT's transpose-matmul-untranspose sequence. Should give identical float32 results.

19.5 Adding a New Layer to ODT

ODT's dispatch table design makes it easy to add new layer types. Here is the complete procedure for adding a new layer (e.g., Tanh activation).

Step 1 — Add to layerType_t enum (Layer.h)

```
typedef enum {
    LAYER_LINEAR = 0,
    LAYER_RELU,
    LAYER_SOFTMAX,
    LAYER_CONV1D,
    LAYER_DROPOUT,
    LAYER_FLATTEN,
    LAYER_TANH, // ← ADD HERE
    LAYER_COUNT
} layerType_t;
```

Step 2 — Add config struct and union field (Layer.h)

```
typedef struct {
    tensor_t *inputCache; // for backward pass
} tanhConfig_t;
// In layerConfig_t union, add:
tanhConfig_t *tanh;
```

Step 3 — Create Tanh.c and Tanh.h

```
// Tanh.h
void tanhForward(layer_t *layer, tensor_t *input, tensor_t *output);
void tanhBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput);
void tanhCalcOutputShape(layer_t *layer, shape_t *in, shape_t *out);
// Tanh.c
#include <math.h>
void tanhForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    tanhConfig_t *cfg = layer->config.tanh;
    cfg->inputCache = input;
    size_t n = calcNumberOfElementsByShape(&input->shape);
    float *in = (float*) input->data.dataPTR;
    float *out = (float*) output->data.dataPTR;
    for (size_t i = 0; i < n; i++)
        out[i] = tanhf(in[i]); // tanhf = float version of tanh
}
void tanhBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
    tanhConfig_t *cfg = layer->config.tanh;
    size_t n = calcNumberOfElementsByShape(&dOutput->shape);
    float *dOut = (float*) dOutput->data.dataPTR;
    float *dIn = (float*) dInput->data.dataPTR;
    float *cachedIn = (float*) cfg->inputCache->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        float t = tanhf(cachedIn[i]);
        dIn[i] = dOut[i] * (1.0f - t * t); // d/dx tanh(x) = 1 - tanh(x)^2
    }
}
void tanhCalcOutputShape(layer_t *l, shape_t *in, shape_t *out) {
    *out = *in; // tanh doesn't change shape
}
```

Step 4 — Register in layerFunctions[] dispatch table (Layer.c)

```
layerFunctions_t layerFunctions[] = {
    [LAYER_LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
    [LAYER_RELU] = { reluForward, reluBackward, reluCalcOutputShape },
    [LAYER_SOFTMAX] = { softmaxForward, softmaxBackward, softmaxCalcOutputShape },
    [LAYER_TANH] = { tanhForward, tanhBackward, tanhCalcOutputShape }, // NEW
    // ...
};
```

After these 4 steps, you can use `LAYER_TANH` anywhere in the training loop exactly like `LAYER_RELU` — the dispatch table handles the routing.

19.6 Adding a New Optimizer

ODT currently only implements SGD. Adding Adam (Adaptive Moment Estimation) is a common next step. Adam requires storing two additional state vectors per parameter (first and second moment estimates).

Adam optimizer struct

```
typedef struct {
    float lr; // learning rate (default 0.001)
    float beta1; // first moment decay (default 0.9)
    float beta2; // second moment decay (default 0.999)
    float epsilon; // numerical stability (default 1e-8)
    size_t t; // step counter (for bias correction)
    tensor_t m; // first moment (mean of gradients)
    tensor_t v; // second moment (mean of squared gradients)
} adam_t;

// Adam update step:
void adamStep(adam_t *adam, parameter_t *param) {
    adam->t++;
    float *w = (float*) param->param.data.dataPTR;
    float *g = (float*) param->grad.data.dataPTR;
    float *m = (float*) adam->m.data.dataPTR;
    float *v = (float*) adam->v.data.dataPTR;
    size_t n = param->param.data.numberOfElements;
    // Bias correction factors
    float bc1 = 1.0f - powf(adam->beta1, adam->t);
    float bc2 = 1.0f - powf(adam->beta2, adam->t);
    for (size_t i = 0; i < n; i++) {
        // Update biased first moment
        m[i] = adam->beta1 * m[i] + (1.0f - adam->beta1) * g[i];
        // Update biased second moment
        v[i] = adam->beta2 * v[i] + (1.0f - adam->beta2) * g[i] * g[i];
        // Bias-corrected moments
        float m_hat = m[i] / bc1;
        float v_hat = v[i] / bc2;
        // Update weight
        w[i] -= adam->lr * m_hat / (sqrtf(v_hat) + adam->epsilon);
    }
}
```

■ NOTE

Adam Memory Cost: Adam requires 2 extra float tensors per parameter tensor (m and v). For a 64×128 weight matrix: $2 \times 64 \times 128 \times 4 \text{ bytes} = 65,536 \text{ extra bytes}$. This is significant on the STM32's 320 KB RAM. SGD needs no extra state.

19.7 Adding a New Loss Function

The `lossFunctions[]` dispatch table works the same way as `layerFunctions[]`. Here's how to add Mean Absolute Error (MAE) loss:

MAE loss implementation

```
// MAE.c
float maeForward(lossFunc_t *loss, tensor_t *pred, tensor_t *target) {
    size_t n = pred->data.numberOfElements;
    float *p = (float*) pred->data.dataPTR;
    float *t = (float*) target->data.dataPTR;
    float sum = 0.0f;
    for (size_t i = 0; i < n; i++)
        sum += fabsf(p[i] - t[i]);
    return sum / n;
}

void maeBackward(lossFunc_t *loss, tensor_t *pred, tensor_t *target,
    tensor_t *dPred) {
    size_t n = pred->data.numberOfElements;
    float *p = (float*) pred->data.dataPTR;
    float *t = (float*) target->data.dataPTR;
    float *dp = (float*) dPred->data.dataPTR;
    for (size_t i = 0; i < n; i++) {
        // d/dp |p-t| = sign(p-t)
        dp[i] = (p[i] > t[i]) ? 1.0f/n : -1.0f/n;
    }
}
```

19.8 Performance Profiling on the STM32

To measure exactly how long each part of the training loop takes, use the DWT (Data Watchpoint and Trace) cycle counter built into Cortex-M7.

Cycle counter profiling

```
// Enable DWT cycle counter (one-time setup in main.c):
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
// Measure a function:
#define CYCLES_START() (DWT->CYCCNT)
#define CYCLES_END(start) (DWT->CYCCNT - (start))
uint32_t t0 = CYCLES_START();
matmulFloat32TensorsWithBias(&w, &x, NULL, &output);
uint32_t cycles = CYCLES_END(t0);
// At 216 MHz: 1 cycle = 4.63 ns
float time_us = (float)cycles / 216.0f; // microseconds
printf('matmul: %u cycles (%.1f us)\n', cycles, time_us);
```

— ■ WORKED EXAMPLE

Measured performance on STM32F746ZG (estimated): Operation | Cycles | Time (216 MHz) Linear forward [64→32], f32 | ~18,000 | ~83 μs ReLU forward [32], f32 | ~200 | ~1 μs Softmax forward [10 classes] | ~300 | ~1.4 μs Cross-entropy backward | ~80 | ~0.37 μs SGD step [64×32 weights] | ~2,200 | ~10 μs Full training step (1 sample) | ~42,000 | ~194 μs Throughput: ~5,000 training samples/second

19.9 Checklist: Before Flashing to MCU

Check	How to Verify
All shapes correct	Add PRINT_DEBUG for shape in each layer init
Scale initialized	Check qConfig->scale != 0.0f before first use
sgdZeroGrad called	Add assert(gradBuffer[0] == 0) after sgdZeroGrad
transposeTensor pairs	Count calls: must be even for each tensor
No VLA larger than ~4KB	Calculate: sizeof(type) * maxDim1 * maxDim2
Stack depth estimated	Add up all local variables in the call chain
Loss decreasing	Print loss every 10 batches — should trend down
Gradients not NaN/inf	Print grad[0] every batch; NaN means overflow
Weights updated	Print W[0] before/after sgdStep — should differ

■ RELATIONS WITH OTHER FILES

Chapter 19 connects to: Chapter 2 (Common.h) — PRINT_DEBUG, PRINT_ERROR, TRACK_INSTRUCTIONS macros · Chapter 7 (Layers) — Adding a new layer follows the dispatch table pattern · Chapter 8 (Training) — Adding new optimizers and loss functions · Chapter 13 (C language) — assert, volatile, test compilation on desktop · Chapter 18 (Embedded) — DWT cycle counter, SWD debugger, UART printf

Chapter 20

SGD, DataLoader, and the Training Infrastructure

How the optimizer, data pipeline, and training loop work together

Section 20.1 — SGD Optimizer: Inside sgdStep

The Stochastic Gradient Descent optimizer is the simplest but most fundamental optimizer in deep learning. ODT implements it in Sgd.h and Sgd.c. Understanding every line of sgdStep reveals what 'training' really means at the lowest level.

Sgd.h — sgd_t struct

```
// sgd_t: the optimizer's persistent state
typedef struct {
    float learningRate; //  $\alpha$  — how large each update step is
    float momentumFactor; //  $\beta$  — fraction of velocity to carry forward (0 = no momentum)
    float weightDecay; //  $\lambda$  — L2 regularization strength (0 = no decay)
} sgd_t;
```

Three fields, three different training behaviours. The learning rate (α) controls the size of each gradient step. Momentum (β) smooths updates by combining the current gradient with a running average of past gradients — like a ball rolling downhill, it builds speed. Weight decay (λ) adds a penalty proportional to the weight magnitude, preventing any single weight from becoming too large (L2 regularisation).

Sgd.c — sgdStepFloat line-by-line

```
// Sgd.c — sgdStepFloat (annotated)
void sgdStepFloat(sgd_t *sgd, parameter_t *param) {
    size_t n = param->param.data.numberOfElements;
    float *w = (float*) param->param.data.dataPTR; // weight array
    float *g = (float*) param->grad.data.dataPTR; // gradient array
    for (size_t i = 0; i < n; i++) {
        // Core SGD update:  $w = w - lr * grad$ 
        w[i] -= sgd->learningRate * g[i];
        // Optional weight decay:  $w = w - lr * \lambda * w$ 
        if (sgd->weightDecay != 0.0f)
            w[i] -= sgd->learningRate * sgd->weightDecay * w[i];
    }
}
```

n	param.data.numberOfElements	Total number of elements in this parameter tensor. For a [32,64] weight matrix: n=2048.
w	(float*) param.param.data.dataPTR	Cast the void* buffer pointer to float* so we can index it as an array. This is the actual weight values that get updated.
g	(float*) param.grad.data.dataPTR	Cast the gradient buffer. These were computed and accumulated during backward pass.
w[i] -= lr * g[i]	w[i] -= sgd->learningRate * g[i];	THE CORE OF TRAINING. Subtract (learning rate $\times$ gradient) from each weight. If the gradient is positive (increasing weight $\rightarrow$ increasing loss), we subtract to decrease the weight, which should decrease the loss.
weight Decay	w[i] -= lr * weightDecay * w[i];	If non-zero, also subtract $lr \times \lambda \times w[i]$. This is equivalent to adding a term $\lambda \times w ^2$ to the loss function, which penalises large weights and encourages simpler (smaller-weight) models — a form of regularisation.

Notice what sgdStepFloat does NOT do: it does not zero the gradients. That is the job of sgdZeroGrad, which must be called at the start of every batch.

Sgd.c — sgdZeroGrad: why scale must be reset

```
// Sgd.c — sgdZeroGrad
void sgdZeroGrad(sgd_t *sgd, linearConfig_t *cfg) {
    // Zero the weight gradient buffer
    size_t wBytes = cfg->weights.grad.data.numberOfElements * sizeof(float);
    memset(cfg->weights.grad.data.dataPTR, 0, wBytes);
    // Reset scale to 1.0 (important for SYM_INT32 — memset alone sets scale to 0)
    if (cfg->weights.grad.quantization.type == SYM_INT32) {
        symInt32QConfig_t *qcfg = cfg->weights.grad.quantization.qConfig;
        qcfg->scale = 1.0f;
    }
    // Zero the bias gradient buffer if present
    if (cfg->hasBias) {
        size_t bBytes = cfg->bias.grad.data.numberOfElements * sizeof(float);
        memset(cfg->bias.grad.data.dataPTR, 0, bBytes);
        if (cfg->bias.grad.quantization.type == SYM_INT32) {
            symInt32QConfig_t *qcfg = cfg->bias.grad.quantization.qConfig;
            qcfg->scale = 1.0f;
        }
    }
}
```

■■ IMPORTANT

The scale=1.0 reset after memset: memset(ptr, 0, n) sets all bytes to zero. For a float, 0x00000000 = 0.0f — correct. But for SYM_INT32, the scale field is also a float stored in the qConfig struct. If the qConfig struct is inside the zeroed region, scale becomes 0.0f. Then the first dequantize call divides by scale = 0.0f → infinity → NaN → disaster. Always reset scale to 1.0f after zeroing quantized tensors.

Section 20.2 — `sgdStepSymInt32`: Training with Quantized Weights

When weights are stored as `SYM_INT32` (fixed-point quantised), the SGD step must convert to float, update, and re-quantise. This is the core of FQT (Fixed-point Quantisation Training) — training happens in float, but weights are stored and used for forward/backward passes in integer format.

`Sgd.c` — `sgdStepSymInt32`: dequantise → update → requantise

```
// Sgd.c — sgdStepSymInt32 (simplified)
void sgdStepSymInt32(sgd_t *sgd, parameter_t *param) {
    size_t n = param->param.data.numberOfElements;
    int32_t *w = (int32_t*) param->param.data.dataPTR;
    float *g = (float*) param->grad.data.dataPTR; // grad always float
    symInt32QConfig_t *qcfg = param->param.quantization.qConfig;
    float scale = qcfg->scale;
    int32_t qMax = (1 << (qcfg->qMaxBits - 1)) - 1;
    // VLA: temporary float buffer on the stack
    float wFloat[n]; // e.g. 2048 floats = 8 KB if n=2048
    // Step 1: dequantise all weights to float
    for (size_t i = 0; i < n; i++)
        wFloat[i] = (float)w[i] * scale;
    // Step 2: SGD update in float
    for (size_t i = 0; i < n; i++)
        wFloat[i] -= sgd->learningRate * g[i];
    // Step 3: re-quantise back to int32
    // Find new abs-max for optimal scale
    float absMax = findAbsMaxFloat(wFloat, n);
    float newScale = (absMax > 0) ? absMax / (float)qMax : scale;
    qcfg->scale = newScale;
    for (size_t i = 0; i < n; i++) {
        int32_t q = roundByMode(wFloat[i] / newScale, qcfg->roundingMode);
        w[i] = clamp(q, -qMax, qMax);
    }
}
```

■ C CONCEPT: VLA — Variable Length Array

`float wFloat[n];` declares a VLA. The size `n` is only known at runtime. VLAs are allocated on the STACK — the compiler adjusts the stack pointer by `n*4` bytes. For `n=2048`: 8KB of stack. On the STM32 with 320KB RAM, the stack might only be 8-16KB total. This is why large tensors MUST use static buffers instead of VLAs. For small layers (`n ≤ 256`), VLAs are safe.

— ■ WORKED EXAMPLE

`SYM_INT32` SGD step trace (`scale=0.01`, `qMaxBits=8`, `qMax=127`, `lr=0.001`) `w[0] = 73` (real value = $73 \times 0.01 = 0.73$) `g[0] = 2.5` (gradient) Step 1 (dequantise): `wFloat[0] = 73 × 0.01 = 0.730` Step 2 (update): `wFloat[0] = 0.730 - 0.001 × 2.5 = 0.7275` Step 3 (requantise): `absMax` found = 0.9 → `newScale = 0.9/127 = 0.00709` `q = round(0.7275 / 0.00709) = round(102.6) = 103` `w[0] = clamp(103, -127, 127) = 103` Real value stored: $103 \times 0.00709 = 0.730$ (very close to 0.7275 — tiny quantisation error)

Section 20.3 — DataLoader: Every Field Explained

The DataLoader is the bridge between the raw dataset and the training loop. It handles batching, shuffling, and calling user-provided data retrieval functions. ODT's DataLoader uses function pointers so that any data source (SD card, UART, hardcoded array, etc.) can be plugged in without modifying the training loop.

DataLoader.h — complete dataLoader_t struct

[illegible]

getSample	<code>void (*)(dataLoader_t*, size_t idx, tensor_t* out, tensor_t* target)</code>	Fetches a single sample at position <code>idx</code> and writes it into <code>out</code> and <code>target</code> . The user implements this — it might read from a hardcoded array, an SD card, or a ring buffer filled by a UART interrupt.
getDatasetSize	<code>size_t (*)(dataLoader_t*)</code>	Returns the total count of samples. Used to compute the number of batches per epoch: <code>numBatches = getDatasetSize() / batchSize</code> .
getBatch	<code>void (*)(dataLoader_t*, size_t batchIdx, tensor_t* out, tensor_t* target)</code>	Higher-level: fills a full batch. The default implementation calls <code>getSample</code> <code>batchSize</code> times, stacking results into the batch tensor.
batchSize	<code>size_t</code>	How many samples per gradient update. Larger batches = stabler gradients but more memory and slower per-update. On the STM32 with limited RAM, small batches (1-8) are typical.
dropLast	<code>bool</code>	If the dataset size is not divisible by <code>batchSize</code> , the last batch is incomplete. <code>dropLast=true</code> discards it; <code>dropLast=false</code> uses it (smaller gradients for last batch).
transform	<code>void (*)(tensor_t*)</code>	Optional: applied to each input after <code>getSample</code> . E.g., normalise audio features to zero mean. If <code>NULL</code> , no transform is applied.
shuffle	<code>bool</code>	If true, the indices array is shuffled using <code>rngShuffleIndices</code> at the start of each epoch. This randomises the order in which samples are seen, preventing the network from memorising the presentation order.
indices	<code>size_t*</code>	Array of size <code>getDatasetSize()</code> , holding sample indices. When <code>shuffle=true</code> , Fisher-Yates permutes this array. <code>getSample</code> then accesses <code>dataset[indices[i]]</code> instead of <code>dataset[i]</code> . Caller must allocate and pass this buffer.

Complete DataLoader setup example

```
// Typical DataLoader initialisation for a small hardcoded dataset
// User-defined dataset
#define NUM_SAMPLES 120
static float audioFeatures[NUM_SAMPLES][64]; // 120 samples, 64 MFCC features
static int speakerLabels[NUM_SAMPLES]; // 0, 1, or 2 for 3 speakers
static size_t shuffleIndices[NUM_SAMPLES];
// Callback: get one sample
void myGetSample(dataLoader_t *dl, size_t idx, tensor_t *x, tensor_t *y) {
    float *xPtr = (float*) x->data.dataPTR;
    float *yPtr = (float*) y->data.dataPTR;
    // Copy features
    memcpy(xPtr, audioFeatures[idx], 64 * sizeof(float));
    // One-hot encode label
    memset(yPtr, 0, 3 * sizeof(float));
    yPtr[speakerLabels[idx]] = 1.0f;
}
// Callback: dataset size
size_t myGetSize(dataLoader_t *dl) { return NUM_SAMPLES; }
// Initialise DataLoader
dataLoader_t loader;
initDataLoader(&loader, myGetSample, myGetSize, defaultGetBatch,
4, // batchSize = 4
true, // dropLast = true
NULL, NULL, // no transforms
true, // shuffle = true
42, // shuffleSeed
shuffleIndices);
```

Section 20.4 — Training Loop Analysis: Every Line Explained

The training loop ties everything together. Here is the complete loop with annotations explaining WHY each step is required and what happens if you skip it.

Complete annotated training loop

```
void trainOneEpoch(dataLoader_t *loader, sgd_t *sgd,
layer_t *layers[], size_t numLayers,
lossFunc_t *loss,
tensor_t *activations[], tensor_t *gradients[]) {
size_t N = loader->getDatasetSize(loader);
size_t numBatches = N / loader->batchSize;
if (!loader->dropLast && (N % loader->batchSize) != 0)
numBatches++;
// Shuffle indices for this epoch
if (loader->shuffle) {
rng32_t rng;
rngSetSeed(&rng, loader->shuffleSeed);
rngShuffleIndices(&rng, loader->indices, N);
loader->shuffleSeed = rng.state; // advance seed for next epoch
}
float epochLoss = 0.0f;
for (size_t b = 0; b < numBatches; b++) {
// ■■ Step 1: Zero gradients ████████████████████████████████████████
for (size_t l = 0; l < numLayers; l++)
if (layers[l]->type == LAYER_LINEAR)
sgdZeroGrad(sgd, layers[l]->config.linear);
// ■■ Step 2: Load batch ████████████████████████████████████████████
loader->getBatch(loader, b, activations[0], gradients[numLayers]);
// ■■ Step 3: Forward pass ████████████████████████████████████████████
for (size_t l = 0; l < numLayers; l++)
layerFunctions[layers[l]->type].forward(
layers[l], activations[l], activations[l+1]);
// ■■ Step 4: Compute loss ████████████████████████████████████████████
float batchLoss = loss->forward(loss,
activations[numLayers], gradients[numLayers]);
epochLoss += batchLoss;
// ■■ Step 5: Backward pass ████████████████████████████████████████████
loss->backward(loss,
activations[numLayers], gradients[numLayers],
gradients[numLayers-1]);
for (int l = (int)numLayers - 1; l >= 0; l--)
layerFunctions[layers[l]->type].backward(
layers[l], activations[l],
gradients[l], (l>0) ? gradients[l-1] : NULL);
// ■■ Step 6: Update weights ████████████████████████████████████████████
for (size_t l = 0; l < numLayers; l++)
if (layers[l]->type == LAYER_LINEAR)
sgdStep(sgd, layers[l]->config.linear);
}
PRINT_DEBUG('Epoch loss: %.4f\n', epochLoss / numBatches);
}
```

Step 1 — SGDZero Grad	Gradient buffers accumulate via +=. Without zeroing, gradients from all previous batches add up. After 1000 batches, the effective gradient would be 1000× too large, causing wildly incorrect weight updates. MUST be the very first step.
-----------------------	---

Step 2 — getBatch		Loads batchSize samples from the dataset into the input activation buffer. The target tensor is also filled (one-hot encoded labels). With shuffle=true, sample order is randomised each epoch.
Step 3 — Forward pass		Sequentially calls each layer's forward function: Linear→ReLU→Linear→Softmax (example). Each activation is stored in activations[+1]. The final activation is the network output.
Step 4 — Compute loss		Compares the network output to the target. Returns a scalar loss value. For cross-entropy: $\text{loss} = -\log(\text{probability assigned to true class})$. Lower is better. Also computes the initial gradient (dL/d_{output}).
Step 5 — Backward pass		Propagates gradients in REVERSE layer order. Each layer receives dOutput (gradient flowing in) and computes: dInput (to pass upstream) and dWeights/dBias (accumulated into gradient buffers). The chain rule in action.
Step 6 — Update weights		For each LINEAR layer: $w -= \text{lr} * \text{gradient}$. This is the actual learning step. Note: only LINEAR layers have learnable parameters. ReLU, Softmax, Flatten have none.

■ NOTE

Why the Loop Iterates Batches, Not Individual Samples A batch of 4 samples produces 4 sets of gradients. The backward pass accumulates them all (via +=) into the gradient buffers. The SGD step then updates weights based on the AVERAGE gradient over the batch. This is more numerically stable than updating after every single sample (1-sample batches = pure stochastic gradient descent, very noisy).

Section 20.5 — Loss Functions: Forward and Backward in Detail

ODT's loss function dispatch table works exactly like the layer dispatch table. A `lossFunctions_t` struct holds forward, backward, and `computeMeanScale` function pointers.

LossFunction.h — dispatch table

```
// LossFunction.h
typedef float (*lossFwdFn_t)(lossFunc_t*, tensor_t* pred, tensor_t* target);
typedef void (*lossBwdFn_t)(lossFunc_t*, tensor_t* pred, tensor_t* target,
tensor_t* dPred);
typedef struct {
    lossFwdFn_t forward;
    lossBwdFn_t backward;
    // Optional: scale gradient by batch mean (mean reduction)
    computeMeanScaleFn_t computeMeanScale;
} lossFunctions_t;
extern lossFunctions_t lossFunctions[]; // indexed by lossFuncType_t
// Registration:
lossFunctions_t lossFunctions[] = {
    [LOSS_MSE] = { mseForward, mseBackward, ... },
    [LOSS_CROSS_ENTROPY] = { crossEntropyFwdFloat, crossEntropySMBBackFloat, ... },
};
```

crossEntropyForwardFloat — annotated

```
// LossFunction.c - crossEntropyForwardFloat
float crossEntropyForwardFloat(lossFunc_t *loss,
tensor_t *pred, tensor_t *target) {
    size_t batch = pred->shape.dimensions[0];
    size_t classes = pred->shape.dimensions[1];
    float *p = (float*) pred->data.dataPTR;
    float *t = (float*) target->data.dataPTR;
    float totalLoss = 0.0f;
    for (size_t b = 0; b < batch; b++) {
        for (size_t c = 0; c < classes; c++) {
            float predVal = p[b * classes + c];
            float targVal = t[b * classes + c];
            if (targVal > 0.0f) {
                // Guard against log(0) = -infinity
                float safeP = (predVal < 1e-7f) ? 1e-7f : predVal;
                totalLoss -= targVal * logf(safeP);
            }
        }
    }
    return totalLoss / (float)batch; // mean over batch
}
```

targ
Val
>
0.0f
che
ck

For one-hot targets, only one class is non-zero. This check skips the log computation for all zero-target classes (saves $n-1$ logf calls per sample).

pre
dVa
l < 1
e-7f
gua
rd

$\log(0) = -\text{infinity}$. If the network is very confident and wrong (predicts 0 for the true class), we clamp to $1e-7$ to prevent NaN loss. In practice, after softmax all outputs are > 0 , but numerical precision can push them to ~ 0 .

/ (float) batch

Divide by batch size to get mean loss. This makes the loss comparable across different batch sizes (a batch of 4 and a batch of 8 give similar loss magnitudes).

crossEntropySoftmaxBackwardFloat — the 1-line gradient

```
// LossFunction.c – crossEntropySoftmaxBackwardFloat
// This implements the combined softmax + cross-entropy backward pass
// Math result: dL/d(logit_i) = softmax_output_i - target_i
void crossEntropySoftmaxBackwardFloat(
    lossFunc_t *loss, tensor_t *pred, tensor_t *target, tensor_t *dPred) {
    size_t batch = pred->shape.dimensions[0];
    size_t classes = pred->shape.dimensions[1];
    float *p = (float*) pred->data.dataPTR; // softmax outputs
    float *t = (float*) target->data.dataPTR; // one-hot targets
    float *dp = (float*) dPred->data.dataPTR; // output gradient buffer
    for (size_t i = 0; i < batch * classes; i++)
        dp[i] = (p[i] - t[i]) / (float)batch;
    // That's it. One line. The combined derivative is beautiful.
}
```

■ C CONCEPT: Why softmax+CE backward is so simple

Separately, softmax backward is complex (involves the Jacobian matrix). Cross-entropy backward is $-\text{target}/\text{pred}$. Combined, almost everything cancels: $d(\text{CE} + \text{Softmax})/d(\text{logit}_i) = \text{softmax}_i - \text{target}_i$. This is why ODT (and all major frameworks) fuse these two operations. The result: if the network predicted 0.7 for the true class (target=1.0), the gradient is $0.7 - 1.0 = -0.3 \rightarrow$ the logit should increase.

■ RELATIONS WITH OTHER FILES

Chapter 20 uses: Sgd.h/c (sgdStep, sgdZeroGrad) · DataLoader.h/c (initDataLoader, getBatch) · LossFunction.h/c (crossEntropyForward, crossEntropySoftmaxBackward) · Layer.h (layerFunctions dispatch) · Rng.h/c (rngShuffleIndices for shuffle) · Tensor.h (tensor_t for all data) · Chapter 17 worked example shows these functions called together.